

**Pengembangan Arsitektur DataOps dan MLOps
Terintegrasi pada Kubernetes dengan Pemanfaatan
Open Source Tools:
Studi Kasus Jual Beli Saham dan Kripto**

Proposal Tugas Akhir

Oleh

**Bryan P. Hutagalung
18222130**



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Desember 2025**

LEMBAR PENGESAHAN

Pengembangan Arsitektur DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*: Studi Kasus Jual Beli Saham dan Kripto

Proposal Tugas Akhir

Oleh

Bryan P. Hutagalung
18222130

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Proposal Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 6 Desember 2025

Pembimbing

Achmad Imam Kistijantoro, S.T, M.Sc., Ph.D.
NIP. 197308092006041001

DAFTAR ISI

DAFTAR ISI	v
DAFTAR GAMBAR	vi
DAFTAR TABEL	vii
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	6
I.3 Tujuan	7
I.4 Batasan Masalah	8
I.5 Metodologi	9
II STUDI LITERATUR	12
II.1 <i>Machine Learning Operations</i> (MLOps)	12
II.1.1 Tingkat Kematangan MLOps	12
II.1.2 Komponen Inti Arsitektur DataOps dan MLOps	18
II.2 Kubernetes untuk <i>Machine Learning Workloads</i>	20
II.2.1 Arsitektur Kubernetes	21
II.2.2 Kubernetes <i>Primitives</i> untuk <i>Machine Learning</i>	21
II.3 <i>Feature Store</i> dan Feast	22
II.3.1 Komponen Arsitektur Feast	22
II.3.2 <i>Feature Materialization</i> dan <i>Point-in-Time Correctness</i>	23
II.3.3 <i>Feature Serving</i> dan <i>Retrieval</i>	23
II.4 Manajemen Siklus Hidup Model	24
II.4.1 MLflow untuk <i>Experiment Tracking</i> dan <i>Model Registry</i>	24
II.4.2 Kubeflow untuk Orkestrasi <i>ML Pipeline</i>	25
II.4.3 Alternatif Platform Orkestrasi	26
II.4.4 KServe untuk Penyajian Model	26
II.5 Pemrosesan dan <i>Streaming Data</i>	27
II.5.1 Apache Spark untuk <i>Batch Processing</i>	27
II.5.2 Apache Flink untuk <i>Stream Processing</i>	28
II.5.3 Apache Kafka untuk <i>Event Streaming</i>	28
II.6 Sistem Penyimpanan	29
II.6.1 ClickHouse sebagai <i>Offline Feature Store</i>	29
II.6.2 Redis Stack untuk <i>Online Feature Store</i> dan Pencarian Vektor	30

II.6.3	MinIO untuk <i>Object Storage</i>	31
II.7	Tata Kelola Data dan <i>Observability</i>	31
II.7.1	DataHub untuk <i>Metadata Management</i>	32
II.7.2	Prometheus dan Grafana untuk <i>Monitoring</i>	33
II.7.3	Keamanan, Autentikasi, dan Manajemen Akses	33
II.8	GitOps dan CI/CD untuk Platform DataOps dan MLOps	34
II.9	Penelitian dan Praktik Terkait	34
II.9.1	Tinjauan Arsitektur MLOps dan Tingkat Kematangan	35
II.9.2	Tantangan Siklus Hidup Data dan Validitas Temporal	35
II.9.3	Deteksi <i>Drift</i> dan Degradasi Model	36
II.9.4	<i>Feature Store</i> dan Arsitektur <i>Dual-Store</i>	36
II.9.5	<i>Streaming</i> dan <i>Near Real-Time</i> MLOps	37
II.9.6	Kubernetes untuk <i>ML Workloads</i>	37
II.9.7	Operasionalisasi dan <i>Deployment</i> ML	38
II.9.8	<i>Governance</i> dan Reprodusibilitas	38
II.9.9	Analisis Celah Riset dan Kontribusi Penelitian Ini	38
III	ANALISIS MASALAH	40
III.1	Analisis Kondisi Saat Ini	40
III.1.1	Implementasi MLOps di Institusi Finansial	40
III.1.2	Arsitektur Umum yang Diterapkan	42
III.1.3	Tantangan Fragmentasi dan Kesenjangan Fungsional	43
III.1.4	Tantangan Spesifik Domain Perdagangan	45
III.1.5	Kesenjangan Kritis yang Teridentifikasi	46
III.2	Analisis Kebutuhan	47
III.2.1	Identifikasi Masalah Pengguna	48
III.2.2	Sintesis Kebutuhan dari Analisis <i>Persona</i>	53
III.2.3	Kebutuhan Fungsional	53
III.2.4	Kebutuhan Nonfungsional	55
III.3	Analisis Pemilihan Solusi	57
III.3.1	Alternatif Solusi	58
III.3.2	Evaluasi Kuantitatif Berdasarkan Pemenuhan Kebutuhan	61
III.3.3	Analisis Penentuan Solusi	62
III.3.4	Mitigasi Keterbatasan	64
III.3.5	Evaluasi dan Pemilihan Komponen Spesifik	65
IV	DESAIN KONSEP SOLUSI	70
IV.1	Perbandingan Arsitektur Sistem	70

IV.1.1	Arsitektur Sistem Saat Ini: Paradigma Terfragmentasi	70
IV.1.2	Arsitektur Sistem yang Diusulkan: Platform Terintegrasi	77
IV.1.3	Analisis Perbedaan Kunci	86
IV.2	Desain Detail Lapisan Arsitektur	92
IV.2.1	Lapisan Infrastruktur dan Orkestrasi	93
IV.2.2	Lapisan Ingesti dan <i>Streaming</i>	96
IV.2.3	Lapisan Pemrosesan Data	98
IV.2.4	Lapisan Layanan Fitur (<i>Feature Service</i>)	100
IV.2.5	Lapisan Layanan Model (<i>Model Service</i>)	102
IV.2.6	Lapisan <i>Observability</i> dan Pemantauan	104
IV.2.7	Lapisan Tata Kelola Data	106
IV.3	Alur Kerja <i>End-to-End</i>	107
IV.3.1	<i>Data Ingestion</i> hingga <i>Feature Materialization</i>	108
IV.3.2	<i>Model Training Workflow</i>	108
IV.3.3	<i>Model Deployment</i> dan <i>Serving</i>	109
IV.3.4	Deteksi <i>Drift</i> dan <i>Continuous Training</i>	109
IV.3.5	<i>Governance</i> dan <i>Audit Workflow</i>	110
V	RENCANA SELANJUTNYA	111
V.1	Tahapan Implementasi	111
V.1.1	Tahap 1: Persiapan Infrastruktur dan Platform Dasar	111
V.1.2	Tahap 2: Implementasi Lapisan Ingesti dan Penyimpanan Data	113
V.1.3	Tahap 3: Implementasi Lapisan Layanan Fitur dengan Feast	115
V.1.4	Tahap 4: Implementasi Lapisan Layanan Model dan Siklus Hidup	116
V.1.5	Tahap 5: Implementasi Penyajian Model dan Pola <i>Deployment</i>	117
V.1.6	Tahap 6: Implementasi Pemrosesan Data dan Deteksi <i>Drift</i>	118
V.1.7	Tahap 7: Implementasi Tata Kelola Data dan Integrasi Final	119
V.2	Jadwal Pengerjaan	120
V.2.1	Tinjauan Linimasa	121
V.2.2	Rincian Jadwal Detil	122
V.2.3	Analisis Jalur Kritis	124
V.3	Kebutuhan Sumber Daya	125
V.3.1	Infrastruktur dan Sumber Daya Komputasi	126
V.3.2	Perangkat Lunak dan Alat	127
V.3.3	Layanan <i>Cloud</i> dan Pertimbangan Anggaran	131
V.3.4	Sumber Daya Manusia dan Keahlian	132
V.4	Analisis Risiko dan Mitigasi	134
V.4.1	Risiko Teknis	134

V.4.2	Risiko Operasional	136
V.4.3	Risiko Personal dan Organisasi	137
DAFTAR PUSTAKA		140

DAFTAR GAMBAR

II.1	Arsitektur Google Cloud MLOps Level 0	16
II.2	Arsitektur Google Cloud MLOps Level 1	16
II.3	Arsitektur Google Cloud MLOps Level 2	17
IV.1	Arsitektur Terfragmentasi Saat Ini (General)	72
IV.2	Arsitektur Terfragmentasi Saat Ini (Data Engineer)	73
IV.3	Arsitektur Terfragmentasi Saat Ini (Data Scientist)	74
IV.4	Arsitektur Terfragmentasi Saat Ini (ML Engineer)	75
IV.5	Arsitektur Terfragmentasi Saat Ini (Business Users)	75
IV.6	Arsitektur Terintegrasi yang Diusulkan (General)	78
IV.7	Arsitektur Terintegrasi yang Diusulkan (Data Engineer)	80
IV.8	Arsitektur Terintegrasi yang Diusulkan (Data Scientist)	82
IV.9	Arsitektur Terintegrasi yang Diusulkan (ML Engineer)	84
IV.10	Arsitektur Terintegrasi yang Diusulkan (Business Users)	85

DAFTAR TABEL

II.1 Perbandingan Framework Tingkat Kematangan MLOps	13
III.1 Kebutuhan Fungsional Sistem	54
III.2 Kebutuhan Nonfungsional Sistem	56
III.3 Evaluasi Pemenuhan Kebutuhan Fungsional	62
III.4 Evaluasi Pemenuhan Kebutuhan Nonfungsional	62
III.5 Evaluasi Alternatif <i>Feature Store Open-Source</i>	65
III.6 Evaluasi Alternatif Orkestrasi ML <i>Open-Source</i>	65
III.7 Evaluasi Alternatif <i>Experiment Tracking</i> dan <i>Model Registry</i>	66
III.8 Evaluasi Alternatif <i>Model Serving Open-Source</i>	66
III.9 Evaluasi Alternatif Manajemen Metadata dan <i>Governance</i>	66
III.10Evaluasi Alternatif <i>Monitoring</i> dan <i>Observability</i>	66
III.11Evaluasi Alternatif Penyimpanan <i>Offline</i> (OLAP)	66
III.12Evaluasi Alternatif Penyimpanan <i>Online</i> (Latensi Rendah)	67
III.13Evaluasi Alternatif <i>Framework</i> Pemrosesan Data	67
IV.1 Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan	87
V.1 Linimasa Pengerjaan Penelitian	122
V.2 Jadwal Detil Oktober - November 2025	123
V.3 Jadwal Detil Desember 2025 - Januari 2026	123
V.4 Jadwal Detil Februari - Maret 2026	124
V.5 Jadwal Detil April - Mei 2026	124
V.6 Ringkasan Biaya Utama untuk Skala Tugas Akhir	132

BAB I

PENDAHULUAN

I.1 Latar Belakang

Pemanfaatan *machine learning* (ML) tidak lagi berhenti pada tahap eksperimen di lingkungan riset, tetapi semakin banyak dioperasikan sebagai sistem yang berjalan stabil di lingkungan produksi. Pada tahap ini, sistem ML tidak hanya berisi kode model, melainkan bergantung pada rangkaian komponen pendukung yang saling terhubung dan perlu dikelola secara konsisten. Polyzotis dkk. (2018) menekankan bahwa sistem ML modern harus dipahami sebagai bagian dari ekosistem data yang kompleks, bukan sekadar kumpulan algoritma. Sejalan dengan itu, Sculley dkk. (2015) menunjukkan bahwa **porsi kode *machine learning* hanya merupakan bagian kecil dari keseluruhan sistem**, sementara sekitar 95% sisanya berupa infrastruktur seperti pengumpulan data, verifikasi, ekstraksi fitur, *monitoring*, dan *serving*. Kondisi ini menuntut perancangan arsitektur yang sejak awal terkelola, terdokumentasi, dan siap menghadapi kebutuhan produksi.

Tantangan tersebut menjadi lebih berat pada domain *algorithmic trading*, karena setiap keputusan model secara langsung memengaruhi pergerakan modal dengan latensi yang sangat rendah dalam hitungan milidetik. **Pertumbuhan pasar *algorithmic trading* global dari USD 19,95 miliar (2024) menjadi USD 21,89 miliar (2025)** menunjukkan meningkatnya ketergantungan industri finansial terhadap sistem ML (The Business Research Company September 2025). Ketergantungan ini membuat kesalahan kecil pada arsitektur data atau proses operasional dapat berujung pada kerugian finansial yang signifikan dalam waktu singkat. Dalam konteks ini, arsitektur DataOps dan MLOps yang *production-grade* diperlukan untuk menjaga akurasi model, tata kelola data, *monitoring*, dan keandalan operasional secara menyeluruh.

Machine Learning Operations (MLOps) dan *Data Operations* (DataOps) mengadaptasi praktik DevOps ke dalam konteks *engineering* ML dan manajemen data untuk mengotomatisasi siklus hidup ML secara *end-to-end* (Kreuzberger, Kühl, dan Hirschl 2023). Pendekatan ini membentuk kerangka kerja bagi proses ML yang efisien, terukur, dan aman pada skala produksi (Jain dan Das 2025; Rella 2022; Ahmad dkk. June 2024). Berbagai solusi telah muncul, mulai dari *managed platform* hingga

rangkaian komponen *open-source* seperti *feature store*, *workflow orchestrator*, dan *serving platform*. Namun, penerapan DataOps dan MLOps secara terpadu masih menghadapi empat tantangan utama yang memengaruhi *reliability* sistem, terutama pada perdagangan finansial yang dinamis, yaitu: (1) tata kelola data dan metadata, (2) degradasi model akibat pergeseran distribusi, (3) kebocoran data temporal pada *feature engineering*, dan (4) keterbatasan arsitektur penyimpanan untuk data modern berdimensi tinggi. Keempat tantangan ini menjadi dasar perumusan arsitektur yang diusulkan dalam proposal tugas akhir ini.

1. Krisis Tata Kelola Data dan Fragmentasi Metadata

Dalam sistem ML berskala besar, ratusan hingga ribuan fitur dapat terbentuk dari beragam sumber data tanpa mekanisme pengelolaan yang konsisten. Tanpa tata kelola yang jelas, repositori fitur berisiko berubah menjadi *data swamp* yang sulit ditelusuri dan sukar dimanfaatkan kembali. Literatur menempatkan *feature store* dan katalog data sebagai komponen utama untuk mengurangi fragmentasi metadata serta memudahkan *discovery* dan *reuse* fitur (Lamer dkk. 2024). Pentingnya tata kelola data juga diperkuat oleh studi lintas bidang yang menunjukkan korelasi positif antara praktik pengelolaan data dan peningkatan inovasi serta kualitas manajemen organisasi (Bernardo dkk. 2024).

Survei Anaconda terhadap 2.400 profesional data menunjukkan bahwa *data scientist* justru menghabiskan porsi waktu terbesar pada tahap persiapan data, bukan pada pemodelan (Anaconda, Inc. July 2020). Pada saat yang sama, organisasi menanggung kerugian besar akibat kualitas data yang buruk (Chien February 2022). Tanpa katalog dan tata kelola yang eksplisit, aset data dan fitur sulit ditemukan, sulit ditafsirkan, dan rentan disalahgunakan. Kondisi ini menghambat eksperimen, mendorong duplikasi pekerjaan, dan meningkatkan risiko kesalahan yang seharusnya dapat dihindari.

Diperlukan katalog metadata terpusat sebagai *single source of truth* yang merekam *lineage* dari *raw data* hingga model produksi, menyediakan *semantic search* untuk *discovery*, dan mendukung dokumentasi otomatis. Implementasi *automated discovery* melalui *data profiling* dan *schema inference* dapat menurunkan beban dokumentasi manual yang biasanya memakan banyak waktu. Selain itu, *data quality monitoring* perlu memberikan *alert* proaktif ketika kualitas data menurun dan berpotensi memengaruhi model (Stoyanovich, Howe, dan Jagadish 2020). Kombinasi mekanisme ini membentuk fondasi tata kelola data yang dapat diskalakan dan diaudit.

2. Degradasi Model Senyap melalui Pergeseran Distribusi

Pasar finansial bersifat non-stasioner sehingga distribusi data input dan hubungan antara fitur dengan target berubah dari waktu ke waktu. Pergeseran distribusi (*data/feature distribution shift*) dan *concept drift* secara bertahap menurunkan kinerja model tanpa selalu menimbulkan *error* eksplisit (Lu dkk. 2019). Studi oleh Vela dkk. (2022) menunjukkan adanya degradasi temporal yang signifikan pada berbagai model dan dataset, sehingga permasalahan ini tidak dapat disederhanakan sebagai *data drift* biasa. Pendekatan *model-based concept drift explanation* juga menegaskan bahwa perubahan dapat terjadi pada hubungan kompleks antar fitur, bukan hanya pada distribusi setiap fitur secara terpisah (Hinder dkk. 2023).

Deteksi *drift* memerlukan metrik statistik yang sensitif terhadap perubahan distribusi, seperti *Population Stability Index* (PSI) dan uji Kolmogorov–Smirnov (Céspedes Sisniega dkk. 2024). Metrik tersebut membandingkan distribusi referensi dengan distribusi aktual sehingga gejala pergeseran dapat diidentifikasi secara kuantitatif. Pada domain finansial yang bergejolak, volume dan kecepatan data membuat inspeksi manual tidak lagi memadai. Tanpa pendekatan yang terukur, degradasi kinerja model mudah terlewat hingga dampaknya terlihat pada metrik bisnis.

Model yang dilatih pada periode volatilitas rendah sering kali gagal ketika pasar memasuki fase *crash* atau *regime change*. Krisis COVID-19 pada Maret 2020 memicu penurunan tajam indeks saham AS (Mazur, andg, dan Vega 2021), sedangkan keruntuhan Terra/LUNA pada Mei 2022 menghapus sekitar USD 50 miliar kapitalisasi pasar hanya dalam tiga hari (Liu, Makarov, dan Schoar 2023). Pada situasi ekstrem seperti ini, model tetap dapat menghasilkan prediksi dengan *confidence* tinggi meskipun akurasinya turun drastis (*silent degradation*). Risiko bisnis meningkat tanpa segera terdeteksi apabila *monitoring* tidak dirancang secara menyeluruh.

Karena itu, dibutuhkan ***monitoring*** statistik *near-real-time* dan periodik terhadap distribusi fitur serta keluaran prediksi, disertai *alerting* proaktif dan *automated retraining* ketika ambang *drift* terlampaui (Bayram, Ahmed, dan Kassler 2022). *Drift detection pipeline* perlu membedakan antara *noise* yang wajar dan pergeseran sistematis yang memerlukan pembaruan model. Integrasi antara *statistical testing* dan *business logic* membantu menekan *false positive alerts* tanpa mengurangi sensitivitas terhadap perubahan yang kritis. Dengan demikian, sistem dapat merespons dinamika pasar secara lebih terkendali dan konsisten.

3. Kebocoran Data Temporal dalam Inovasi *Feature Engineering*

Inovasi fitur untuk membangun keunggulan kompetitif dalam *trading* rentan terhadap kebocoran temporal, yaitu ketika informasi masa depan tanpa sengaja digunakan untuk memprediksi masa lalu. Kebocoran ini membuat hasil *backtest* terlihat sangat baik, tetapi gagal direplikasi di lingkungan produksi (Wang dan Ruf 2021). Tanpa pengendalian eksplisit, kebocoran temporal mengganggu validitas evaluasi model dan membuat metrik performa menjadi menyesatkan. Dalam jangka panjang, hal ini dapat menurunkan kepercayaan terhadap keseluruhan proses pengembangan model.

Kebocoran temporal biasanya muncul melalui tiga pola utama, yaitu: (1) agregasi temporal yang keliru, (2) asumsi ketersediaan data yang tidak tepat, dan (3) kesalahan penyelarasan waktu, termasuk *timezone alignment errors* dan batas sesi perdagangan yang tidak konsisten. Ketiga pola tersebut dapat menggeser informasi masa depan ke masa lalu akibat *join* atau transformasi yang kurang tepat. Jika tidak terdeteksi, kesalahan ini akan terbawa ke seluruh siklus pelatihan dan evaluasi model. Oleh karena itu, pencegahan perlu dilakukan secara sistematis pada level *feature engineering* dan *retrieval*.

Konsep *point-in-time correctness* menuntut *temporal query semantics* yang memverifikasi bahwa setiap fitur memang sudah tersedia pada *timestamp* yang digunakan. Feast menyediakan *point-in-time joins* untuk memastikan *feature retrieval* hanya menggunakan data yang secara historis tersedia pada saat prediksi dilakukan (Feast Project 2024). Mekanisme ini mencakup *timestamp validation*, *feature versioning*, dan *lag enforcement* sehingga kebocoran data temporal dapat dihindari baik pada tahap pelatihan maupun inferensi. Dengan pendekatan ini, evaluasi dan penggunaan model menjadi lebih konsisten terhadap kondisi operasional sebenarnya.

Validasi kebocoran temporal memerlukan *automated testing framework* yang memeriksa integritas waktu secara konsisten. Great Expectations menyediakan *expectation suite* yang dapat digunakan untuk menguji integritas temporal, misalnya memastikan tidak ada fitur dengan *timestamp* yang lebih baru daripada *prediction timestamp* (Great Expectations 2024). Validasi ini perlu diterapkan baik pada *feature engineering* manual maupun otomatis agar integritas temporal terjaga di seluruh jalur data. Dengan cara tersebut, risiko kebocoran dapat dicegah sejak awal perancangan *pipeline*, bukan hanya dideteksi di akhir.

4. Keterbatasan Arsitektur untuk Data Modern Berdimensi Tinggi

Sinyal dari *alternative data* seperti sentimen berita dan media sosial sering direpresentasikan sebagai *vector embeddings* berdimensi tinggi. BERT-Base menghasilkan vektor 768 dimensi (Devlin dkk. 2019, Table 1), sedangkan OpenAI *text-embedding-ada-002* menghasilkan vektor 1.536 dimensi (OpenAI December 2022). *Online store key-value* tradisional tidak dirancang untuk *similarity search* yang efisien, padahal kebutuhan latensi ultra-rendah dengan persentil ke-99 (p_{99}) < 10 ms menjadi penting untuk *embeddings retrieval*. Untuk menjembatani kebutuhan tersebut, indeks vektor seperti Hierarchical Navigable Small World (HNSW) diperlukan untuk *approximate nearest neighbor search* yang cukup cepat sekaligus akurat (Malkov dan Yashunin 2020).

Arsitektur HNSW mengorganisasikan vektor dalam bentuk graf berlapis sehingga pencarian dilakukan secara hierarkis dengan latensi rendah (Malkov dan Yashunin 2020). Lapisan atas berfungsi sebagai jalur navigasi kasar menuju area ruang vektor yang relevan, sedangkan lapisan bawah melakukan pencarian yang lebih rinci. Struktur ini mengurangi jumlah perbandingan jarak yang diperlukan untuk menemukan tetangga terdekat secara aproksimatif. Dengan demikian, HNSW lebih sesuai untuk kebutuhan *real-time* dibandingkan pencarian eksak yang mahal secara komputasi.

Pada pasar kripto, sentimen dapat berubah dalam hitungan menit sebagai respons terhadap *tweet*, pernyataan publik, atau perubahan regulasi. *Nearest-neighbor search* yang lambat dapat menyebabkan peluang transaksi bernilai tinggi hilang sebelum dieksekusi. Karena itu, diperlukan **integrasi vector database dengan approximate nearest neighbor search** yang mendukung *hybrid retrieval* menggunakan *dense vectors* dan *sparse vectors*. Integrasi ini tetap harus mempertahankan target $p_{99} < 10$ ms agar sistem mampu merespons perubahan pasar secara konsisten dalam skenario dunia nyata.

Proposal tugas akhir ini mengusulkan **arsitektur DataOps dan MLOps terintegrasi berbasis komponen open-source di atas Kubernetes** sebagai satu kesatuan sistem untuk menangani keempat tantangan tersebut secara *end-to-end*. Arsitektur MLOps *open-source* yang diorkestrasi dengan Kubernetes dan didukung oleh observabilitas Prometheus/Grafana serta *drift detectors* telah terbukti layak diterapkan (Burgueno-Romero, Cánovas Izquierdo, dan García-García 2025). Dengan perdagangan saham dan *cryptocurrency* sebagai domain uji yang sangat volatil (Rodrigues dkk. 2025), arsitektur ini ditujukan untuk menghasilkan platform yang andal, *matu-re*, dan dapat digunakan kembali lintas *use case*. Rancangan ini diharapkan berfungsi

si sebagai *blueprint* arsitektural yang adaptif bagi organisasi lain yang menghadapi tantangan serupa.

I.2 Rumusan Masalah

Berdasarkan tantangan implementasi DataOps dan MLOps yang telah diuraikan pada bagian latar belakang, proposal tugas akhir ini menyusun seperangkat pertanyaan riset yang terfokus dan saling terkait. Pertanyaan tersebut diarahkan pada perancangan arsitektur, mekanisme *monitoring*, jaminan integritas temporal, dan dukungan untuk data modern berdimensi tinggi sehingga kelemahan kondisi saat ini dapat diatasi secara sistematis.

1. Bagaimana merancang arsitektur yang mengintegrasikan *feature serving*, *model serving*, dan katalog data secara otomatis di atas platform orkestrasi Kubernetes untuk membentuk *single source of truth* yang terkelola, mudah ditemukan, dan tepercaya, sehingga fragmentasi metadata dan **asimetri informasi** dalam ekosistem DataOps dan MLOps dapat dikurangi, duplikasi pekerjaan menurun, dan risiko pengambilan keputusan berbasis data yang tidak konsisten dapat ditekan?
2. Bagaimana mengimplementasikan *pipeline monitoring* DataOps dan MLOps yang terotomatisasi dan proaktif dalam mendeteksi *feature drift* dan *concept drift* secara *near-real-time* dan periodik di lingkungan pasar finansial yang non-stasioner, menggunakan metrik statistik seperti *Population Stability Index* (PSI) dan uji Kolmogorov–Smirnov (Céspedes Sisniega dkk. 2024), sehingga degradasi model tidak lagi terjadi secara senyap dan dapat ditindaklanjuti sebelum menimbulkan kerugian bisnis yang material?
3. Bagaimana arsitektur *feature serving* dapat secara inheren menjamin validitas temporal atau *point-in-time correctness* pada proses *feature engineering*, baik yang dilakukan secara manual oleh *data scientist* maupun secara otomatis melalui *feature pipeline*, sehingga kebocoran data temporal dapat dicegah sejak tahap perancangan, akurasi hasil *backtest* lebih mencerminkan kondisi produksi, dan kepercayaan pemangku kepentingan terhadap metrik evaluasi dapat ditingkatkan?
4. Bagaimana memperluas arsitektur penyimpanan fitur tradisional agar mampu menyimpan, mengelola, dan menyajikan tipe data modern seperti *vector embeddings* berdimensi tinggi secara efisien dengan latensi ultra-rendah ($p99 < 10$ ms), sehingga kebutuhan *similarity search* untuk *alternative data* se-

perti sentimen berita dan media sosial dapat dipenuhi tanpa mengorbankan *throughput* maupun ketersediaan layanan pada skenario *algorithmic trading* berfrekuensi tinggi?

I.3 Tujuan

Proposal tugas akhir ini bertujuan untuk merancang, mengimplementasikan, dan memvalidasi **arsitektur *prototype* layanan DataOps dan MLOps terintegrasi** yang secara langsung menjawab pertanyaan riset dan keberhasilannya dapat diukur melalui kriteria teknis dan operasional yang jelas. Secara khusus, tujuan tugas akhir dirinci sebagai berikut.

1. Merancang dan mengimplementasikan **arsitektur terintegrasi yang *cloud-native*** dengan *technology stack* yang telah dispesifikasikan. Seluruh komponen diintegrasikan di atas Kubernetes (Lakka 2025) untuk membentuk *unified metadata flow* yang konsisten dan dapat ditelusuri lintas *pipeline*, dengan kriteria keberhasilan berupa prototipe yang dapat di-*deploy* dan dioperasikan secara stabil pada kluster uji.
2. Membangun *prototype automated drift monitoring pipeline* yang menganalisis distribusi fitur secara periodik dan memicu peringatan dini berbasis PSI serta uji Kolmogorov–Smirnov (Céspedes Sisniega dkk. 2024), dengan kriteria keberhasilan berupa kemampuan mendeteksi *synthetic drift* pada skenario uji yang dirancang dan memicu alur penanganan (misalnya *retraining* atau *rollback*) sesuai ambang yang ditetapkan.
3. Mengimplementasikan layanan fitur yang menerapkan ***point-in-time correct joins*** pada pengambilan data pelatihan sehingga validitas temporal terjamin dan kebocoran data temporal terhindar pada pelatihan maupun inferensi, dengan kriteria keberhasilan berupa tidak ditemukannya pelanggaran integritas temporal pada serangkaian uji berbasis *temporal validation framework* dan *automated testing* terhadap integritas *timestamp*.
4. Mendesain dan memvalidasi **arsitektur *hybrid online store*** untuk fitur tabular dan *vector embeddings serving* dengan target $p99 < 10$ ms pada *similarity search*, menggunakan Redis untuk fitur tabular, *vector database* (Weaviate atau Qdrant) dengan HNSW, serta *API Gateway* untuk orkestrasi *hybrid search*. Kriteria keberhasilan berupa tercapainya target latensi p99 dan *throughput* minimum yang ditetapkan pada dokumen kebutuhan.
5. Mencapai **Level 2 (ML pipeline automation)** menurut Google Cloud (Google

Cloud 2024) melalui otomasi Continuous Integration/Continuous Delivery/Continuous Training (CI/CD/CT) berbasis GitOps, mencakup *drift-triggered re-training*, validasi otomatis sebelum *deployment*, *continuous monitoring*, serta *rollback capabilities*, dengan kriteria keberhasilan berupa rangkaian *pipeline* otomatis yang dapat dieksekusi dari komit kode hingga *deployment* dan *rollback* terkontrol pada lingkungan uji.

I.4 Batasan Masalah

Untuk menjaga fokus dan kedalaman pembahasan, tugas akhir yang diusulkan dalam proposal ini menetapkan batasan yang mendefinisikan ruang lingkup secara eksplisit. Batasan ini dirancang agar beban implementasi tetap terkelola, tetapi tetap representatif terhadap tantangan MLOps yang muncul di praktik. Ruang lingkup yang jelas juga mendukung replikasi dan evaluasi karena asumsi dan keputusan desain terdokumentasi secara transparan. Dengan demikian, hasil yang diperoleh dapat diinterpretasikan secara tepat sesuai konteks yang ditetapkan.

1. Fokus Arsitektur dibanding Optimisasi Model

Tugas akhir berfokus pada desain, implementasi, dan evaluasi infrastruktur DataOps dan MLOps, bukan pada optimisasi model ML. Model diperlakukan sebagai *workload* untuk memvalidasi arsitektur, bukan sebagai objek utama perbandingan algoritma. Model yang digunakan dapat berupa regresi linear, *gradient-boosted trees*, atau *neural networks* sehingga arsitektur yang dirancang terbukti fleksibel untuk berbagai jenis model tanpa perubahan berarti. Dengan demikian, evaluasi diarahkan pada kemampuan arsitektur dalam mendukung berbagai jenis *workload*, bukan pada performa algoritma tertentu.

2. Domain Finansial sebagai Studi Kasus Terkendali

Data saham dan kripto digunakan sebagai studi kasus untuk menguji arsitektur pada data yang dinamis dan volatil. Data historis Yahoo Finance (1 tahun), CoinGecko (6 bulan), serta skenario simulasi *market shock* dimanfaatkan sebagai dasar pengujian. Implementasi tidak dihubungkan ke *live trading* dan tidak melibatkan transaksi nyata agar eksperimen berjalan terkendali, replikatif, dan bebas risiko finansial. Dengan cara ini, perilaku sistem dapat diamati secara menyeluruh tanpa menimbulkan konsekuensi langsung terhadap pasar.

3. Eksklusivitas Open-Source Stack

Solusi hanya menggunakan *technology stack open-source* yang telah dispesifikasikan: Kubernetes (Lakka 2025), Kubeflow (Kubeflow Contributors 2024), Feast, MLflow (MLflow Contributors 2024), KServe (KServe Contributors

2024), DataHub, Spark (Apache Software Foundation 2024d), Flink (Apache Software Foundation 2024b), Kafka (Apache Software Foundation 2024c), Redis Stack, dan ClickHouse (ClickHouse Inc 2024). Solusi *proprietary* atau layanan *managed cloud* di luar IaaS dasar tidak digunakan untuk menjaga *reproducibility*, *transparency*, *customizability*, dan *cost-effectiveness*. Pemilihan ini juga memudahkan adopsi oleh organisasi yang memiliki kebijakan ketat terkait ketergantungan terhadap vendor tertentu.

4. Cakupan *Data Pipeline* Transformasi

Data diasumsikan berasal dari *pipeline* eksternal dalam format terstruktur seperti CSV, Parquet, dan JSON. Desain untuk *raw data ingestion* dijelaskan secara konseptual, tetapi implementasi lengkap *raw data extraction* seperti *web scraping*, API bursa *real-time*, dan *parsing* dokumen PDF/HTML berada di luar cakupan. Fokus utama berada pada transformasi dan operasionalisasi data yang sudah tersedia agar pembahasan aspek DataOps dan MLOps dapat digali lebih dalam. Dengan pembatasan ini, kompleksitas teknis tetap terkendali tanpa mengurangi relevansi hasil.

5. Evaluasi Terkendali dibanding *Production Deployment*

Evaluasi dilakukan pada lingkungan terkendali menggunakan data historis, data sintetis, dan *synthetic load patterns* untuk mensimulasikan kondisi produksi. Skalabilitas diuji melalui *load testing* dan berbagai *failure scenarios* seperti *node failures*, *network partitions*, dan *storage unavailability*. *Production deployment* 24/7 dengan trafik nyata tidak termasuk dalam cakupan agar kompleksitas operasional tetap terkelola dan evaluasi teknis tetap terukur. Walau demikian, skenario uji dirancang sedekat mungkin dengan situasi produksi agar temuan yang diperoleh tetap relevan.

I.5 Metodologi

Tugas akhir yang diusulkan dalam proposal ini menggunakan *Design Science Research Methodology* (DSRM) yang mengintegrasikan teori dan praktik dalam perancangan serta evaluasi *artifact* teknologi. *Artifact* yang dikembangkan berupa arsitektur sistem dan implementasi *prototype* yang dirancang untuk memenuhi kebutuhan operasional ML pada lingkungan yang menuntut. Metodologi ini dipilih karena menyediakan kerangka sistematis untuk mengidentifikasi masalah, merancang solusi, dan menguji efektivitasnya. DSRM kemudian diadaptasi ke dalam enam fase yang saling terkait secara sistematis dan iteratif.

1. Fase 1: *Systematic Literature Review* dan Analisis *State-of-the-Art* (4 ming-

gu)

Kajian literatur dilakukan secara terstruktur terhadap publikasi akademik dan dokumentasi teknis untuk membangun landasan teoretis. Literatur diambil dari Scopus, IEEE Xplore, ACM Digital Library, dan arXiv dengan kriteria relevansi, kebaruan (2019 ke atas), dan dampak sitasi. *Search strategy* menggunakan *keyword* seperti “MLOps”, “DataOps”, “feature store”, “model serving”, “drift detection”, “vector database”, dan “Kubernetes ML”, serta mengkaji dokumentasi resmi komponen *open-source*.

Dilakukan *analisis komparatif* terhadap berbagai pendekatan arsitektur (Amou Najafabadi dkk. 2024) untuk mengidentifikasi celah riset dan peluang kontribusi (Paleyes, Urma, dan Lawrence 2022). Evaluasi mencakup perbandingan solusi *open-source* dan *enterprise*, *cloud-native* dan *on-premises*, serta pendekatan *best-of-breed* dan *end-to-end platform*. Luaran fase ini berupa *gap analysis*, *positioning statement*, dan landasan teoretis yang akan digunakan pada Bab II.

2. Fase 2: Analisis Masalah dan Spesifikasi Kebutuhan (2 minggu)

Fase ini menerjemahkan pertanyaan riset menjadi kebutuhan teknis yang konkret. Proses diawali dengan *gap analysis* antara kondisi eksisting dan kondisi target berdasarkan empat tantangan pada latar belakang. Identifikasi *use case* dilakukan melalui *user persona* (Data Engineer, ML Engineer, ML Ops Engineer, Data Scientist) serta *user journey mapping* dari *data ingestion* hingga *model monitoring*.

Kebutuhan fungsional dan nonfungsional dispesifikasikan secara terukur, meliputi target latensi $p99 < 10$ ms, *availability* 99,9%, *horizontal scalability*, modularitas, dan *security*. Setiap kebutuhan ditautkan ke pertanyaan riset untuk menjaga *traceability*. Luaran fase ini berupa *requirements matrix*, *use case matrix*, dan definisi Service Level Agreement (SLA) yang akan digunakan pada Bab III.

3. Fase 3: Desain Solusi Arsitektur Berlapis (3 minggu)

Desain arsitektur memadukan pendekatan *top-down* dan *bottom-up* berdasarkan kebutuhan yang telah diidentifikasi. Alternatif solusi dieksplorasi melalui *trade-off analysis* dan *decision matrix* dengan kriteria biaya, fleksibilitas, risiko *vendor lock-in*, dukungan komunitas, dan kematangan teknis.

Arsitektur dirancang secara berlapis: *Infrastructure Layer*, *Data Layer*, *Feature Layer*, *Model Layer*, *Serving Layer*, *Observability Layer*, dan *Governance Layer*. Setiap lapisan memiliki titik integrasi, alur komunikasi, dan

mekanisme *failure handling* yang jelas. Luaran fase ini mencakup diagram arsitektur (C4), urutan interaksi antarkomponen, dan diagram alur data yang kemudian akan dijabarkan pada Bab IV.

4. Fase 4: Implementasi *Prototype* Iteratif (8 minggu)

Implementasi dilakukan melalui empat *sprint* dua mingguan. **Sprint 1** membangun klaster Kubernetes, MinIO, Kafka, dan Spark. **Sprint 2** mengintegrasikan Feast, Redis, Prometheus/Grafana, dan *basic drift detection*. **Sprint 3** mengimplementasikan Kubeflow *pipelines*, MLflow, KServe, dan *model registry*. **Sprint 4** mengintegrasikan DataHub, *drift-triggered retraining*, Great Expectations, dan pengujian *end-to-end*.

Implementasi CI/CD berbasis GitOps menggunakan Argo CD (Argo Project 2024), dengan pemisahan *infrastructure code*, *application code*, dan konfigurasi YAML. Luaran fase ini berupa *working prototype*, *deployment scripts*, serta panduan *setup* dan *troubleshooting* yang menjadi dasar penyusunan rencana implementasi dan pengembangan lanjutan pada Bab V.

5. Fase 5: Evaluasi dan Pengujian Sistematis (4 minggu)

Evaluasi memvalidasi pemenuhan kebutuhan melalui *load testing* dengan alat seperti JMeter atau Locust untuk mengukur p50/p95/p99 dan *throughput*, dengan fokus pada SLA $p99 < 10$ ms. Pengujian juga mencakup skenario kegagalan seperti *node failures*, *network partitions*, dan *storage unavailability*.

Validasi fungsional mencakup *point-in-time correctness*, akurasi *drift detection* melalui *synthetic drift injection*, verifikasi otomatisasi *end-to-end*, dan pengujian *vector embeddings* terhadap latensi serta akurasi. Data uji meliputi Yahoo Finance (1 tahun), CoinGecko (6 bulan), data sintesis ekstrem, dan *synthetic embeddings*. Luaran fase ini menjadi bahan utama analisis hasil, *lessons learned*, serta perumusan rencana pengembangan lanjutan yang akan didokumentasikan pada Bab V.

6. Fase 6: Dokumentasi Komprehensif dan Analisis Hasil (2 minggu)

Fase terakhir menyintesis temuan melalui analisis terhadap kekuatan arsitektur, manfaat terukur, keterbatasan, dan rasional *trade-off* yang diambil. Pembahasan juga mencakup perbandingan solusi *open-source* dengan *managed services* dan implikasinya terhadap konsistensi serta *availability* pada sistem terdistribusi. Dokumentasi disusun agar dapat menjadi referensi praktis sekaligus akademik bagi pengembangan lanjutan dan menjadi penghubung menuju rencana selanjutnya pada Bab V.

BAB II

STUDI LITERATUR

II.1 *Machine Learning Operations* (MLOps)

Machine Learning Operations (MLOps) adalah pendekatan terintegrasi untuk mengelola sistem *machine learning* di lingkungan produksi. Pendekatan ini menggabungkan prinsip DevOps untuk otomasi *deployment* dan operasi perangkat lunak (Kim dkk. 2016), praktik DataOps untuk pengelolaan *pipeline* dan kualitas data, serta *Machine Learning Engineering* untuk pengembangan dan penguatan model. Melalui studi kasus industri, Paleyes, Urma, dan Lawrence (2022) menunjukkan bahwa *deployment* ML menghadirkan tantangan khusus, seperti kerumitan infrastruktur, pengelolaan *dependency* model, dan kebutuhan *monitoring* pada sistem yang bersifat stokastik. Kebutuhan akan MLOps muncul ketika organisasi menuntut kesiapan produksi yang lebih baik, penurunan *technical debt*, serta praktik pengujian dan *monitoring* yang konsisten. Kreuzberger, Kühn, dan Hirschl (2023) merumuskan sembilan komponen inti arsitektur MLOps dan kerangka tingkat kematangan yang menegaskan pentingnya integrasi dan otomasi siklus hidup ML bagi keandalan, reproduksibilitas, serta kemudahan pemeliharaan sistem.

Landasan MLOps berangkat dari karakter sistem ML yang berbeda dengan perangkat lunak tradisional karena bergantung pada data dan model yang bersifat stokastik. Sculley dkk. (2015) menyoroti bahwa perangkat lunak konvensional bersifat lebih deterministik, sedangkan performa ML sangat dipengaruhi kualitas data pelatihan, *hyperparameter*, dan elemen acak dalam proses pelatihan. Perbedaan ini menambah kompleksitas pada aspek reproduksibilitas, pengujian, *monitoring*, dan *deployment* yang tidak dapat ditangani sepenuhnya oleh praktik DevOps standar. MLOps berperan menjembatani kesenjangan antara eksperimen model dan operasi produksi melalui proses yang terstruktur dan terdokumentasi, sehingga perubahan data maupun kode dapat dikelola secara sistematis dan dampaknya terhadap sistem dapat ditelusuri sepanjang siklus hidup model.

II.1.1 Tingkat Kematangan MLOps

Model tingkat kematangan MLOps menggambarkan sejauh mana proses ML di sebuah organisasi telah terotomatisasi dan terintegrasi ke dalam *pipeline* produksi. Berbagai penyedia teknologi menawarkan kerangka kematangan dengan sudut

pandang yang berbeda, tetapi semuanya memetakan perjalanan dari proses yang sepenuhnya manual hingga otomatis yang stabil dan dapat diskalakan. Pemetaan ini memberi acuan bagi organisasi untuk menilai posisi saat ini dan merencanakan peningkatan bertahap sesuai kebutuhan sistem dan kesiapan infrastruktur (Google Cloud 2024; Microsoft Azure 2023; Kartakis dkk. June 2022; McKnight January 2020). Dengan memahami tingkat kematangan, organisasi dapat menyusun prioritas pengembangan platform dan investasi secara lebih fokus.

Tabel II.1 menyajikan perbandingan empat kerangka kematangan utama dari Google Cloud, Microsoft Azure, AWS, dan GigaOm. Perbandingan dilakukan pada dimensi DevOps, otomatisasi *ML pipeline*, integrasi *Continuous Integration/Continuous Delivery* (CI/CD), *Continuous Training* (CT), *monitoring* dan *feedback loop*, serta *governance* dan *lineage*. Dimensi tersebut dipilih sebagai indikator praktis untuk menilai struktur dan kedalaman implementasi MLOps di suatu organisasi. Kerangka yang digunakan dalam penelitian ini dipilih secara pragmatis berdasarkan kesesuaian dengan kebutuhan sistem, bukan sebagai standar normatif yang harus diikuti oleh semua organisasi.

Tabel II.1 Perbandingan Framework Tingkat Kematangan MLOps

FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
GCP	L0	Tidak ada (silo tim)	Manual (script, notebook)	Tidak ada	Tidak ada	Tidak ada (<i>model untracked</i>)	Tidak ada (<i>no lineage</i>)
	L1	Parsial (<i>operational symmetry</i>)	Ya (<i>end-to-end</i>)	Parsial (<i>pipeline</i> saja)	Ya (<i>automated</i>)	Ya (<i>drift detection</i>)	Ya (<i>metadata store, feature store</i>)
	L2	Ya (<i>fully integrated</i>)	Ya (<i>fully automated</i>)	Ya (CI-/CD/CT penuh)	Ya (<i>fully automated</i>)	Ya (<i>integrated</i>)	Ya (<i>automated</i> melalui sistem CI/CD)
Azure	L0	Tidak ada (<i>disparate teams</i>)	Manual (<i>ad hoc</i>)	Tidak ada	Tidak ada	Tidak ada (<i>black box</i>)	Tidak ada (<i>no tracking</i>)

FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
	L1	Ya (aplikasi saja)	Manual (<i>model handoff</i>)	Aplikasi saja (bukan untuk ML)	Tidak ada	Minimal (<i>limited feedback</i>)	Minimal (sulit ditelusuri)
	L2	Ya (DevOps aplikasi)	Ya (pelatihan saja)	Tidak ada (<i>release manual</i>)	Ya (pelatihan otomatis)	Parsial (pelacakan terpusat)	Ya (<i>version control</i> , manajemen model)
	L3	Ya (terintegrasi)	Ya (latih dan <i>deploy</i>)	Ya (CD model)	Ya (pelatihan otomatis)	Parsial (<i>A/B testing</i>)	Ya (pengujian terintegrasi)
	L4	Ya (<i>fully integrated</i>)	Ya (<i>fully automated</i>)	Ya (CI/CD/CT penuh)	Ya (<i>auto retrain</i>)	Ya (<i>verbose metrics, closed loop</i>)	Ya (komprehensif)
AWS	Initial	Tidak ada	Manual (<i>notebooks</i> , lokal)	Tidak ada	Tidak ada	Tidak ada	Tidak ada (<i>no standardization</i>)
	Repeatable	Parsial (infrastruktur)	Ya (<i>training pipeline</i>)	Tidak ada (<i>deploy manual</i>)	Ya (<i>automated</i>)	Minimal (infrastruktur eksperimen)	Ya (<i>model registry, version control</i>)
	Reliable	Ya (terintegrasi)	Ya (<i>automated</i>)	Ya (CI/CD terintegrasi)	Ya (<i>automated</i>)	Ya (<i>prod monitoring, rollback</i>)	Ya (IaC, pengujian otomatis)
	Scalable	Ya (multi-account/region)	Ya (<i>fully automated</i> , multi-region)	Ya (otomasi penuh)	Ya (<i>auto retrain</i> melalui <i>drift detection</i>)	Ya (<i>drift detection</i> , optimasi otomatis)	Ya (<i>centralized feature store, full governance</i>)
Giga Om	L0	Tidak ada (<i>no standards</i>)	Manual (inkonsisten)	Tidak ada	Tidak ada	Minimal	Minimal (keamanan minimal)
	L1	Parsial (<i>basic practices</i>)	Parsial (sebagian otomatis)	Tidak ada (sebagian besar manual)	Tidak ada	Minimal (<i>basic tracking</i>)	Parsial (<i>limited governance</i>)

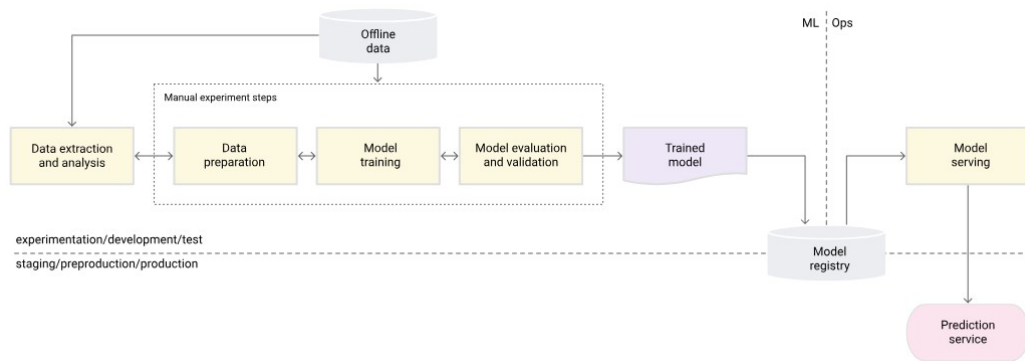
FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
	L2	Ya (diimplementasikan)	Ya (pipeline CI/CD)	Ya (diimplementasikan)	Parsial (sebagian otomatis)	Parsial (basic monitoring)	Ya (formal governance dimulai)
	L3	Ya (fully integrated)	Ya (fully automated)	Ya (komprehensif)	Ya (automated)	Ya (comprehensive monitoring/alerting)	Ya (automated governance, self-service)
	L4	Ya (fully adaptive)	Ya (AI-optimized)	Ya (self-optimizing)	Ya (continuous learning)	Ya (continuous learning dari produksi)	Ya (platform terintegrasi dan adaptif penuh)

Keterangan:

1. Merah : Tidak ada implementasi atau sepenuhnya manual
2. Kuning : Implementasi parsial atau terbatas
3. Hijau : Implementasi penuh atau otomatis

Dalam penelitian ini, **framework Google Cloud digunakan sebagai rujukan utama** untuk mengarahkan implementasi MLOps. Kerangka tiga tingkat (0–2) yang ditawarkan menitikberatkan pada otomasi *end-to-end* dan memberikan definisi yang jelas mengenai CI, CD, dan CT. Kerangka tersebut dirancang untuk organisasi yang membangun sistem ML dari awal dan menempatkan CT sebagai komponen penting sejak Level 1. Penekanan ini membuat hubungan antara perubahan data produksi dan siklus pelatihan model menjadi eksplisit sehingga relevan dengan karakter data finansial yang non-stasioner dan mudah berubah. Gambar II.1 sampai dengan Gambar II.3 mengilustrasikan arsitektur referensi Google Cloud MLOps Level 0 hingga Level 2 yang dijadikan acuan pemetaan kebutuhan platform pada penelitian ini.

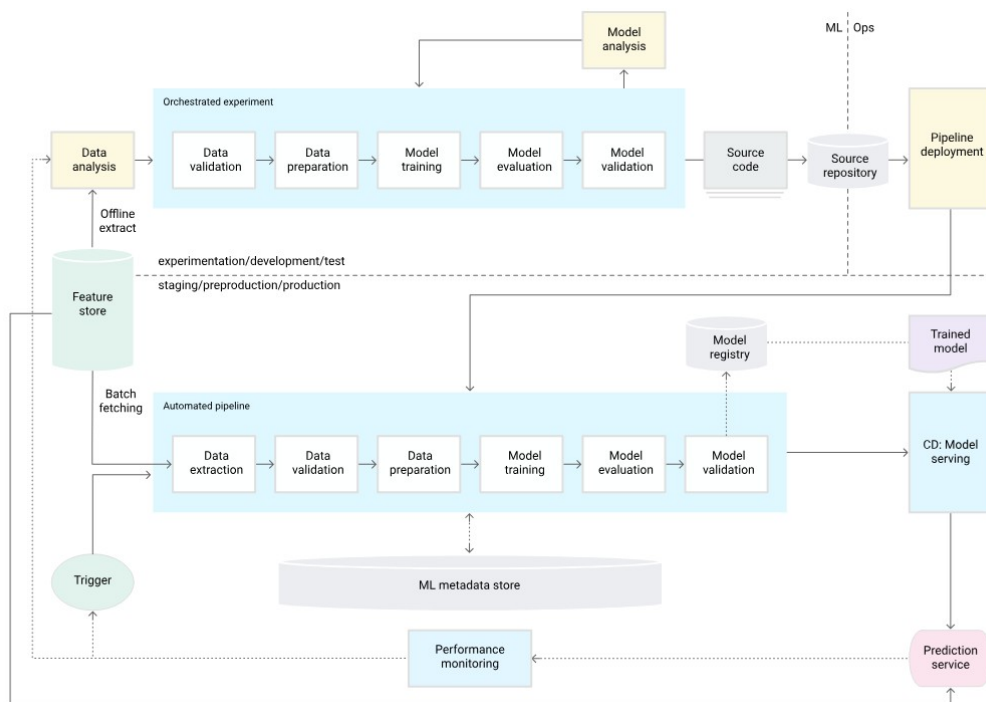
1. Level 0: *Manual Process*



Gambar II.1 Arsitektur Google Cloud MLOps Level 0

Pada Level 0, pengembangan model dan proses *deployment* masih dilakukan secara manual tanpa dukungan *pipeline* otomatis. Tim *data scientist* dan tim operasional bekerja dalam *silo* sehingga koordinasi sangat bergantung pada komunikasi informal dan prosedur tidak baku. CI/CD maupun CT belum tersedia, dan model biasanya dibangun melalui *notebook* atau skrip yang tidak terdokumentasi dengan baik. Kondisi ini menyulitkan pengulangan eksperimen, meningkatkan risiko kesalahan manusia, dan menghambat penskalaan ketika kompleksitas sistem bertambah.

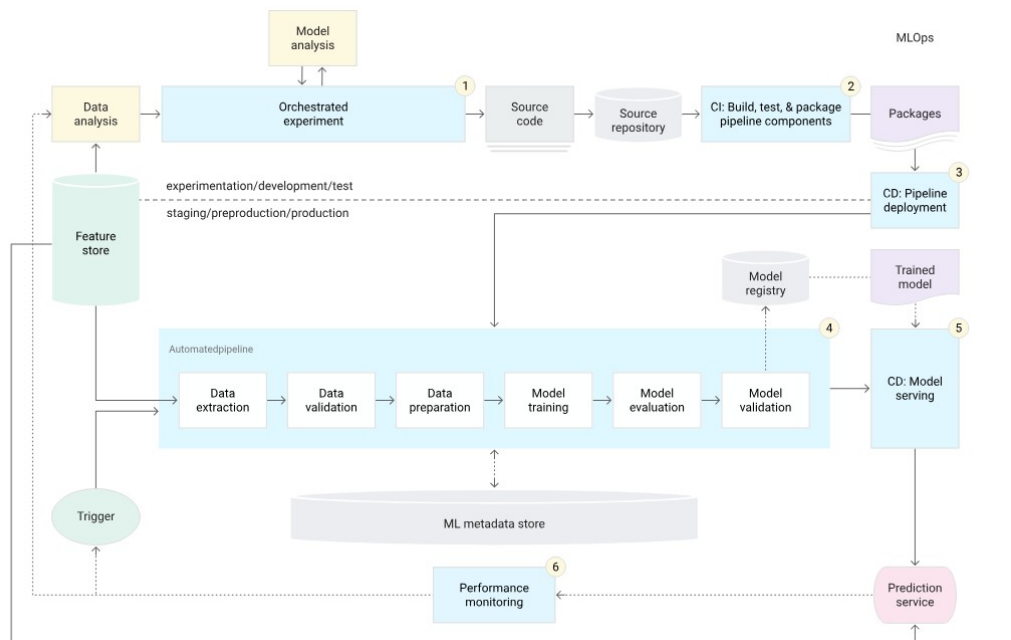
2. Level 1: *ML Pipeline Automation*



Gambar II.2 Arsitektur Google Cloud MLOps Level 1

Pada Level 1, organisasi mulai menerapkan otomasi *pipeline* ML secara *end-to-end*. Otomasi ini umumnya mencakup validasi data, *feature engineering*, pelatihan, validasi model, dan pelatihan ulang otomatis berdasarkan jadwal tertentu. *Tracking* eksperimen dan *metadata* mulai dipusatkan sehingga iterasi pemodelan menjadi lebih cepat dan keputusan teknis dapat ditelusuri. Model dideploy sebagai layanan prediksi, dan sistem mulai memantau *data drift* maupun *model drift* secara lebih terstruktur. Integrasi CI/CD dan CT pada tahap ini masih terbatas dan belum sepenuhnya menyatu dengan praktik rekayasa perangkat lunak yang lebih luas.

3. Level 2: CI/CD Pipeline Automation



Gambar II.3 Arsitektur Google Cloud MLOps Level 2

Pada Level 2, seluruh *pipeline* ML dikelola oleh CI/CD yang terotomatisasi dan terintegrasi untuk pengujian data, model, dan kode. CI, CD, dan CT berjalan sebagai satu rangkaian yang utuh, termasuk *automated retraining* yang dipicu oleh *feedback loop* dari produksi dan metrik bisnis. Level ini juga mencakup praktik *governance* yang kuat melalui proses *approval*, *compliance check*, dan pengaturan hak akses yang jelas. Selain itu, terdapat *proactive monitoring* untuk metrik ML seperti *data drift*, *concept drift*, dan degradasi kinerja model (Google Cloud 2024). Dengan struktur ini, perubahan di lingkungan produksi dapat direspons melalui aksi otomatis yang terukur dan terdokumentasi.

Berdasarkan kerangka kematangan tersebut, penelitian ini menargetkan implementasi MLOps pada **Level 2: CI/CD Pipeline Automation**. Pada tingkat ini, sistem mencakup *automated ML pipeline*, CT yang aktif, integrasi CI/CD penuh, pengelolaan *metadata* terpusat, serta *monitoring* yang terhubung dengan *alerting* dan mekanisme pemicu otomatis. Tingkat kematangan ini sejalan dengan kebutuhan sistem perdagangan finansial yang menuntut kualitas data dan model tetap konsisten sepanjang siklus hidup sistem, sekaligus mempermudah penerapan *continuous improvement* berbasis data.

II.1.2 Komponen Inti Arsitektur DataOps dan MLOps

Arsitektur DataOps dan MLOps tersusun dari sekumpulan komponen yang bersama-sama menopang siklus hidup sistem ML dari hulu ke hilir. Pemahaman yang jelas terhadap fungsi dan interaksi setiap komponen menjadi dasar untuk merancang sistem yang dapat diskalakan dan mudah dipelihara. Integrasi yang tepat membantu mengurangi risiko kegagalan di produksi dan mempercepat transisi dari eksperimen ke *deployment*, sehingga kemampuan analitik dapat diselaraskan dengan kebutuhan bisnis tanpa mengorbankan stabilitas operasional.

Komponen Data Management bertanggung jawab mengelola data mulai dari *ingestion*, validasi, *versioning*, hingga penyimpanan. Polyzotis dkk. (2018) menunjukkan bahwa masalah pada siklus hidup data sering menjadi sumber kegagalan sistem ML di produksi. *Data validation* memastikan data mengikuti *schema* yang disepakati dan memenuhi standar kualitas, sedangkan *data versioning* menyimpan *snapshot dataset* agar eksperimen dapat direproduksi dan dianalisis ketika terjadi anomali. Pada arsitektur yang diusulkan, komponen ini diperkuat oleh kombinasi *streaming platform*, *object storage*, dan alat validasi kualitas data yang terintegrasi dalam satu alur.

Komponen Feature Engineering mengubah data mentah menjadi fitur yang siap digunakan model. Tantangan utama tahap ini adalah menjaga keselarasan logika dan data antara pelatihan dan *serving* untuk menghindari *training-serving skew*. *Feature Store* menyediakan repositori terpusat yang menyimpan definisi dan nilai fitur sehingga dapat diakses kembali oleh *training pipeline* maupun *serving*. Bagian II.3 membahas *Feature Store* secara lebih rinci karena komponen ini berperan langsung dalam menjaga validitas temporal dan konsistensi fitur. Dengan repositori terpusat, tim dapat mengurangi duplikasi logika dan mempermudah audit perubahan fitur.

Komponen Model Training mencakup pemilihan algoritme, pengaturan *hyperpa-*

parameter, dan pelaksanaan proses pelatihan. Praktik MLOps menekankan *experiment tracking* yang mencatat parameter, *dataset*, versi kode, konfigurasi infrastruktur, dan metrik hasil pelatihan. Pada skala besar, *distributed training* memerlukan orkestrasi khusus yang dibahas pada Bagian II.4.2. Keluaran proses pelatihan dihubungkan dengan *model registry* untuk pengelolaan versi sehingga setiap model yang dihasilkan memiliki konteks dan rekam jejak yang jelas.

Komponen *Model Analysis* dan *validasi* memastikan model memenuhi standar kualitas sebelum dipromosikan ke produksi. Tahap ini meliputi evaluasi metrik, perbandingan dengan *baseline* atau *champion model*, serta analisis tambahan seperti *fairness*, *robustness*, dan *interpretability*. Hasil evaluasi menjadi dasar keputusan apakah model layak digunakan atau perlu disesuaikan. Integrasi tahap analisis ke dalam *pipeline* membantu menurunkan risiko penerapan model yang tidak stabil di lingkungan nyata dan memudahkan pembuktian kualitas model kepada pemangku kepentingan.

Komponen *Deployment* dan *Model Serving* menangani penyajian model sebagai layanan prediksi. Infrastruktur *serving* harus mampu memberikan latensi rendah, *throughput* tinggi, skala elastis, dan *high availability*. Strategi seperti *canary release* dan *A/B testing* membantu peluncuran model baru secara bertahap dan aman. Fowler dan Lewis (2014) menunjukkan bahwa pola *microservices* sesuai untuk pemisahan tanggung jawab dan *rollout* bertahap. Implementasi KServe sebagai komponen utama *model serving* di atas Kubernetes dijelaskan pada Bagian II.4.4 dan diintegrasikan dengan komponen lain dalam platform MLOps.

Komponen *Monitoring* dan *Observability* memantau kesehatan sistem dan kinerja model saat berjalan di produksi. Pemantauan mencakup metrik sistem seperti latensi, *throughput*, dan *error rate*, serta metrik ML seperti distribusi fitur, distribusi prediksi, dan *confidence score*. Deteksi *data drift* dan *concept drift* menjadi penting (Bayram, Ahmed, dan Kassler 2022) karena degradasi kinerja sering terjadi secara bertahap dan tidak langsung terlihat dari metrik infrastruktur. *Monitoring* perlu terhubung dengan *feedback loop* otomatis agar pelatihan ulang dapat dipicu ketika degradasi teridentifikasi. Pada arsitektur penelitian ini, metrik dari *Feature Store*, *stream processing*, dan *model serving* dikumpulkan oleh *monitoring stack* yang akan dijelaskan pada Bagian II.7.2.

Komponen *Governance* dan *Compliance* memastikan praktik ML sejalan dengan kebijakan organisasi, regulasi, dan prinsip etika (Bernardo dkk. 2024). Stoyanovich, Howe, dan Jagadish (2020) menyoroti pentingnya *lineage tracking*, *access control*,

dan *audit logging* untuk menjaga akuntabilitas. Pada sektor finansial, *governance* yang kuat menjadi prasyarat sebelum sistem ML dioperasikan secara luas. Dalam implementasi, konsep tersebut diwujudkan melalui platform *metadata management* seperti DataHub yang digabungkan dengan lapisan autentikasi dan otorisasi terpusat serta manajemen rahasia yang aman. Aspek ini dibahas lebih lanjut pada Bagian II.7 sebagai bagian dari fondasi DataOps dan MLOps yang tepercaya.

II.2 Kubernetes untuk *Machine Learning Workloads*

Kubernetes telah berkembang menjadi *de facto standard* untuk *container orchestration* (Lakka 2025) dan banyak dimanfaatkan untuk menjalankan *workload* ML. Kreuzberger, Köhl, dan Hirschl (2023) menunjukkan bahwa Kubernetes mendominasi implementasi MLOps modern baik di industri maupun penelitian. Kubernetes menyediakan fondasi deklaratif untuk *deploying*, *scaling*, dan *managing* aplikasi, termasuk *workload* ML yang memerlukan komputasi intensif dan dinamis. Dengan pendekatan deklaratif, keadaan klaster yang diinginkan didefinisikan melalui *manifest* dan dipertahankan oleh *control plane*, sehingga replikasi lingkungan menjadi lebih mudah dan konfigurasi manual yang rentan kesalahan dapat dikurangi.

Portabilitas Kubernetes memungkinkan *workload* dalam bentuk kontainer dijalankan secara konsisten di lingkungan *on-premise*, *public cloud*, maupun *edge*. Adusumilli (2025) menegaskan bahwa abstraksi ini menurunkan hambatan transisi dari pengembangan ke produksi dan mendukung strategi *hybrid* atau *multi-cloud*. **Skalabilitas** Kubernetes penting untuk *training* dan *serving* yang kebutuhan sumber dayanya dapat berubah dengan cepat. *Horizontal Pod Autoscaler* menyesuaikan jumlah *pod* berdasarkan metrik, sedangkan *Cluster Autoscaler* mengatur jumlah *node* agar kapasitas klaster selaras dengan beban dan biaya tetap terkendali. Kombinasi fitur tersebut menjadikan Kubernetes fondasi yang fleksibel untuk platform MLOps yang terukur.

Resource management dan isolasi di Kubernetes memungkinkan skenario *multi-tenant* dikelola secara aman dan adil. *Namespaces*, *resource quota*, dan *limit range* mengatur pemakaian sumber daya oleh setiap tim atau aplikasi, sedangkan *device plugin* mengelola penjadwalan *Graphics Processing Unit* (GPU) untuk *workload* pelatihan dan inferensi berat. Fitur *priority* dan *preemption* memastikan *workload* kritis tetap berjalan meskipun sumber daya sedang terbatas. Pendekatan GitOps memanfaatkan Kubernetes sebagai *control plane* yang dikonfigurasi melalui deklarasi di repositori Git. Alat seperti Argo CD memantau repositori Git (termasuk yang

di-host di Gitea) dan menyelaraskan keadaan klaster dengan *manifest* yang didefinisikan. Pola ini memperkuat praktik CI/CD dan audit konfigurasi, dan akan dibahas pada Bagian II.8.

II.2.1 Arsitektur Kubernetes

Sebuah klaster Kubernetes terdiri dari *control plane* dan satu atau lebih *worker node*. *Control plane* mencakup *API server*, *etcd*, *scheduler*, dan *controller manager* yang bersama-sama menjaga *desired state* klaster. *API server* berfungsi sebagai gerbang utama untuk seluruh interaksi dengan klaster, baik dari pengguna maupun komponen internal. *Etcd* menyimpan *state* klaster secara konsisten sehingga setiap perubahan konfigurasi dapat disinkronkan dengan andal. *Scheduler* memilih *node* yang akan menjalankan *pod*, sedangkan *controller manager* menjalankan loop rekonsiliasi untuk memastikan kondisi aktual mengikuti spesifikasi.

Di sisi lain, *worker node* menjalankan *kubelet*, *container runtime*, dan *kube-proxy*. Untuk *workload* ML, *node* sering dilengkapi GPU dan memori besar, sehingga pelabelan *node* dan pengaturan kendala *scheduling* menjadi penting. Konsep *node pool* memungkinkan pengelompokan *node* berdasarkan spesifikasi sumber daya, sehingga organisasi dapat menyeimbangkan kebutuhan kinerja dengan efisiensi biaya. Pemahaman terhadap komponen arsitektur ini membantu perancang sistem ML memanfaatkan Kubernetes secara optimal, khususnya saat mengatur prioritas *job* pelatihan dan layanan inferensi yang peka terhadap latensi dan harus tetap tersedia.

II.2.2 Kubernetes *Primitives* untuk *Machine Learning*

Pod merupakan unit terkecil *deployment* yang dapat memuat *model server* dan *sidecar* seperti *logger* dan *metrics exporter*. *Deployment* sesuai untuk layanan *inference* yang bersifat *stateless*, sedangkan *StatefulSet* digunakan untuk *workload* yang memerlukan identitas dan penyimpanan persisten. Dengan memanfaatkan kedua pola ini, layanan inferensi dan komponen data dapat diatur secara terpisah namun tetap terkoordinasi dalam satu klaster yang sama. Pola tersebut memudahkan pengelolaan siklus hidup layanan tanpa mengganggu komponen lain.

Job dan *CronJob* mendukung *batch workload* seperti pelatihan berkala atau prediksi massal yang terjadwal. *ConfigMap* dan *Secret* memisahkan konfigurasi dan data sensitif dari *image*, sehingga pengelolaan konfigurasi *pipeline* lebih aman dan tidak mengharuskan pembangunan ulang *image* saat konfigurasi berubah. *Service* dan *Ingress* menyediakan *endpoint* stabil di dalam dan di luar klaster, dan mengatur rute

HTTP/HTTPS untuk aplikasi. Dalam konteks *inference*, konfigurasi yang tepat pada komponen ini penting agar akses ke model tetap andal dan terlindungi, termasuk integrasi dengan *Application Programming Interface (API) gateway* dan mekanisme autentikasi di lapisan luar.

II.3 *Feature Store* dan Feast

Feature Store adalah komponen penting MLOps yang menyediakan mekanisme terpusat untuk mengelola dan *serving* fitur secara konsisten. Tanpa repositori terpusat, fitur cenderung dihitung dan diakses dengan cara berbeda antara pelatihan dan *serving*, sehingga memicu *training-serving skew*. *Feature Store* mengatasi masalah ini dengan menyediakan definisi, penyimpanan, dan penyajian fitur menggunakan logika yang sama di seluruh siklus hidup ML. Selain itu, *Feature Store* memperkaya *metadata* fitur yang dapat dimanfaatkan untuk *governance* dan audit, sehingga proses penelusuran dan penjelasan fitur menjadi lebih mudah.

Feast merupakan salah satu *open-source Feature Store* yang banyak diadopsi (Feast Project 2024). Proyek ini awalnya dikembangkan oleh Gojek dan menawarkan arsitektur *dual-store* untuk memisahkan fitur historis di *offline store* dan fitur terkini di *online store*. Desain Feast menekankan fleksibilitas dan kemudahan integrasi dengan infrastruktur data yang telah ada di organisasi. Dengan pendekatan ini, Feast dapat diterapkan pada berbagai kombinasi *stack* data modern tanpa memaksa perubahan arsitektur yang drastis. Posisi Feast dalam arsitektur penelitian ini menjadi penghubung antara *pipeline* data, penyimpanan, dan layanan *serving*.

II.3.1 Komponen Arsitektur Feast

Feature Registry menyimpan definisi fitur, *feature view*, dan sumber data sebagai *metadata* terpusat. *Registry* biasanya diletakkan di repositori Git sehingga mendukung *version control* dan praktik GitOps (Limoncelli 2022). Selain definisi teknis, *registry* juga memuat tipe data, batasan, *owner*, dan dokumentasi yang relevan. Informasi ini meningkatkan *discoverability* dan *governance* fitur serta memudahkan audit perubahan definisi, sehingga evolusi fitur dapat dikelola secara lebih terkontrol.

Offline Store digunakan untuk melayani fitur historis yang dibutuhkan pada pelatihan atau *batch inference*. Pada penelitian ini, ClickHouse dipilih sebagai *offline store* (Bagian II.6.1). Feast melakukan *point-in-time correct join* antara *feature view* dan *entity timestamp* untuk mencegah kebocoran temporal. *Online Store* me-

nyediakan fitur berlatensi rendah untuk *real-time inference*. Redis Stack digunakan sebagai *online store* (Bagian II.6.2). Fitur dapat dimaterialisasikan secara terjadwal dari *offline store* atau dialirkan *real-time* dari *streaming pipeline*. Pemisahan *offline* dan *online* ini memungkinkan setiap lapisan dioptimalkan sesuai pola beban tanpa mengorbankan konsistensi definisi fitur.

II.3.2 *Feature Materialization dan Point-in-Time Correctness*

Feature materialization adalah proses menghitung nilai fitur dan menuliskannya ke *Feature Store*. Feast mendukung *batch materialization* terjadwal untuk data historis dan *stream materialization* secara *real-time* dari sumber seperti Kafka (Bagian II.5.3). Kombinasi kedua pendekatan ini memungkinkan organisasi menyesuaikan strategi pemrosesan dengan kebutuhan *freshness* fitur. Dengan demikian, fitur yang sama dapat dimanfaatkan secara konsisten untuk pelatihan maupun inferensi tanpa logika yang terpecah di berbagai layanan.

Point-in-time correctness memastikan bahwa fitur historis hanya dibangun dari data yang benar-benar tersedia pada waktu tertentu. Properti ini sangat penting pada data *time-series* karena kesalahan waktu dapat menimbulkan kebocoran informasi masa depan ke masa lalu. Polyzotis dkk. (2018) menekankan bahwa kebocoran temporal merupakan salah satu penyebab utama penurunan kinerja model ketika masuk ke produksi. Feast menerapkan prinsip ini melalui *join* berbasis *entity key* dan *event timestamp* sehingga integritas temporal tetap terjaga. Dengan mekanisme ini, evaluasi dan *backtesting* di domain finansial dapat memberikan gambaran performa yang lebih realistis terhadap kondisi dunia nyata.

II.3.3 *Feature Serving dan Retrieval*

Feast menyediakan *feature retrieval* melalui Python SDK atau gRPC API untuk pola *online* maupun *batch*. *Online retrieval* mengambil nilai fitur terbaru dari *online store* dengan latensi rendah dan cocok untuk *real-time inference*. *Batch retrieval* mengambil fitur historis dari *offline store* untuk pelatihan atau prediksi dalam volume besar. Konsistensi antara kedua pola ini mengurangi duplikasi logika perhitungan fitur dan menekan *technical debt* yang timbul akibat implementasi ad hoc, sehingga tim data dapat mengandalkan satu sumber kebenaran untuk definisi dan perhitungan fitur.

Feature transformation memungkinkan penerapan transformasi saat pengambilan fitur dilakukan. Melalui *on-demand transformation*, *derived feature* dihitung keti-

ka *retrieval* berlangsung sehingga nilai fitur tetap *fresh* tanpa memperbesar beban penyimpanan. Mekanisme ini mendukung iterasi desain fitur yang lebih cepat dan fleksibel karena perubahan logika transformasi dapat diterapkan tanpa perlu mengulang proses materialisasi besar. Pendekatan ini juga mempermudah penyelarasan antara eksplorasi data di tahap riset dan kebutuhan operasional di produksi.

II.4 Manajemen Siklus Hidup Model

Manajemen siklus hidup model mencakup tahapan pengembangan, *deployment*, *monitoring*, pemeliharaan, hingga *decommissioning*. Pengelolaan yang terstruktur memastikan model di produksi tetap akurat, stabil, dan relevan dengan perubahan data maupun kebutuhan bisnis. Selain itu, pengaturan siklus hidup yang jelas membantu organisasi menjelaskan keputusan yang diambil oleh sistem otomatis kepada pemangku kepentingan dan regulator. Pada domain finansial, kemampuan menjelaskan proses pengambilan keputusan ini menjadi penting untuk menjawab pertanyaan terkait risiko dan kepatuhan.

Di samping menjaga kualitas teknis, manajemen siklus hidup model juga berfungsi memastikan kesesuaian dengan kebijakan internal dan regulasi eksternal. Platform DataOps dan MLOps menyediakan alat untuk mengotomatisasi alur promosi model, pencatatan *metadata*, dan kontrol perubahan. Dengan adanya otomasi tersebut, setiap tahap dalam siklus hidup model dapat ditelusuri dan diaudit kembali jika diperlukan. Pendekatan ini mengurangi risiko *model drift*, kesalahan konfigurasi, dan penggunaan model yang sudah tidak sesuai dengan kondisi terkini di lapangan.

II.4.1 MLflow untuk *Experiment Tracking* dan *Model Registry*

MLflow merupakan standar *open-source* yang banyak digunakan untuk pengelolaan siklus hidup ML (MLflow Contributors 2024). **MLflow Tracking** mencatat parameter, metrik, *tag*, dan *artifact* untuk setiap eksperimen sehingga variasi pendekatan pemodelan dapat dibandingkan secara sistematis. Kemampuan ini memudahkan tim untuk mengidentifikasi konfigurasi yang memberikan hasil terbaik tanpa kehilangan jejak eksperimen sebelumnya. Dukungan *autologging* untuk berbagai *framework* umum dan *manual logging* untuk kebutuhan khusus menjadikan pencatatan eksperimen lebih fleksibel dan dapat disesuaikan dengan pola kerja tim data.

Antarmuka *user interface* (UI) MLflow mempermudah penelusuran *run*, perbandingan metrik, dan pengunduhan *artifact* sehingga eksperimen dapat dikelola secara terstruktur dan mudah direproduksi. **MLflow Model Registry** mengelola versi

model, status siklus hidup, dan menjadi titik serah antara pelatihan dan *deployment*. Mekanisme *versioning* memungkinkan beberapa versi model hidup berdampingan sehingga *rollout* dan *A/B testing* dapat dilakukan secara terkendali. Promosi model antartahap dapat diotomatisasi melalui integrasi CI/CD dan praktik GitOps, dengan anotasi dan *tag* yang memperkaya konteks pada setiap versi model yang disimpan.

MLflow Models menyediakan format pengemasan model yang portabel lintas berbagai infrastruktur *serving*. Dalam skenario produksi, MLflow umumnya diintegrasikan dengan alat *serving* khusus seperti KServe (Bagian II.4.4) sehingga model dapat dijalankan pada lingkungan yang terstandar. Untuk menyimpan *artifact*, MLflow membutuhkan *object storage*; dalam arsitektur penelitian ini, MinIO dipakai sebagai *artifact store* (Bagian II.6.3). Dengan konfigurasi ini, seluruh artefak pelatihan dapat disimpan secara terpusat, tahan lama, dan mudah ditelusuri kembali ketika diperlukan evaluasi atau audit.

II.4.2 Kubeflow untuk Orkestrasi *ML Pipeline*

Kubeflow dirancang untuk menyederhanakan orkestrasi *workflow* ML di atas Kubernetes agar tetap portabel dan mudah diskalakan (Kubeflow Contributors 2024). **Kubeflow Pipelines** memungkinkan definisi dan eksekusi *pipeline* multi-langkah yang diterjemahkan menjadi Argo *Workflow* dan dijalankan di dalam kluster. Setiap *pipeline* terdiri dari *component* yang dikontainerisasi dengan *input-output* eksplisit sehingga alur data dan *dependency* antar langkah dikelola secara otomatis. Pendekatan ini mengurangi kompleksitas skrip orkestrasi manual dan membuat proses lebih mudah direproduksi.

Kubeflow Pipelines UI menyediakan tampilan untuk melacak *run*, membandingkan hasil, melihat *lineage*, serta menjadwalkan *recurring run* untuk tugas seperti pelatihan ulang atau *batch inference*. **Kubeflow Notebooks** menawarkan lingkungan Jupyter di Kubernetes sehingga eksperimen dapat memanfaatkan sumber daya kluster, termasuk GPU, tanpa konfigurasi manual yang rumit. **Kubeflow Training Operator** menyediakan *Custom Resource Definition* (CRD) seperti *TFJob* dan *PyTorchJob* untuk menjalankan *distributed training job* secara terkelola. Dengan kombinasi komponen ini, Kubeflow memposisikan Kubernetes sebagai platform utama untuk menjalankan *ML pipeline* dalam skala produksi.

II.4.3 Alternatif Platform Orkestrasi

Selain Kubeflow, terdapat sejumlah platform lain yang relevan untuk orkestrasi ML dan data. **Apache Airflow** adalah orkestrator matang berbasis *Directed Acyclic Graph* (DAG) yang mendukung penjadwalan kompleks, banyak integrasi, dan berbagai *executor* termasuk *KubernetesExecutor*. Keunggulan Airflow terletak pada fleksibilitas dan kemampuan *error handling* untuk *data pipeline* yang melibatkan banyak sistem berbeda. Banyak organisasi memanfaatkannya sebagai tulang punggung orkestrasi data pada *data warehouse* dan *data lake*.

Beberapa platform seperti **Optimus** memanfaatkan Airflow sebagai basis dan menambahkan lapisan abstraksi untuk memudahkan orkestrasi transformasi data secara deklaratif dan standar. Pendekatan ini membantu mengelola dependensi, kualitas data, dan jadwal eksekusi untuk *warehouse* dan *data lake* secara konsisten. **Tekton Pipelines** menawarkan CI/CD *cloud-native* berbasis CRD yang *vendor-agnostic*. Dalam konteks MLOps, Tekton dapat digunakan untuk mengotomatisasi alur mulai dari *code commit* hingga *deployment* model di kluster. Pemilihan orkestrator bergantung pada kebutuhan dan konteks organisasi, dan kombinasi beberapa platform tetap memungkinkan selama integrasi dan batas tanggung jawabnya diatur secara jelas.

II.4.4 KServe untuk Penyajian Model

KServe merupakan platform *Kubernetes-native* untuk *serverless inference* yang menyediakan *auto-scaling* dan strategi *deployment* lanjutan (KServe Contributors 2024). KServe berstatus *incubating* di CNCF dan dibangun di atas Knative, sehingga memanfaatkan kemampuan *scale-to-zero* dan pengelolaan lalu lintas bawaan dari ekosistem tersebut. **InferenceService** menjadi CR utama yang menggambarkan model, *runtime*, kebutuhan sumber daya, dan strategi *deployment* yang digunakan. Melalui satu definisi deklaratif, tim dapat mengatur cara model dijalankan, diskalakan, dan diekspos.

Predictor menjalankan proses *inference* dengan dukungan bawaan untuk berbagai *framework*, serta *custom predictor* jika diperlukan. **Transformer** memungkinkan *preprocessing* dan *postprocessing* sebagai layanan terpisah, sedangkan **Explainer** menyediakan fungsi interpretabilitas seperti Anchor, LIME, atau SHAP. KServe juga mendukung *canary deployment* dan *A/B testing* untuk meluncurkan model baru secara bertahap sambil memantau dampaknya. Pembagian trafik dan kemampuan *rollback* yang cepat menjadikan penerapan model di produksi lebih aman, terutama

ketika diintegrasikan dengan *observability stack* untuk memantau kualitas layanan secara berkelanjutan.

II.5 Pemrosesan dan *Streaming Data*

Infrastruktur pemrosesan data bertugas mengubah data mentah menjadi fitur yang siap digunakan untuk pelatihan maupun *inference*. Sistem perlu mendukung *batch processing* untuk data historis dan *stream processing* untuk komputasi fitur secara *real-time*. Integrasi antara pemrosesan, orkestrasi, dan *Feature Store* membentuk alur hulu–hilir yang konsisten, sehingga definisi fitur dan *training dataset* dapat dikelola sebagai sasaran bersama dan perubahan di hulu dapat ditelusuri dampaknya sampai ke model produksi.

Pada studi kasus perdagangan saham dan kripto, pemrosesan *batch* dibutuhkan untuk membangun *training dataset* historis yang representatif, sementara *streaming* digunakan untuk menghasilkan fitur yang peka terhadap waktu. Selain *engine* pemrosesan seperti Apache Spark dan Apache Flink, arsitektur penelitian ini memanfaatkan ekosistem alat *open-source* seperti Kafka dan komponen pendukungnya (misalnya Raccoon (Raystack 2025h), Stencil (Raystack 2025j), Firehose (Raystack 2025c), serta Optimus dan Dagger (Raystack 2025g, 2025b)). Alat-alat ini digunakan untuk menyederhanakan proses *ingest*, transformasi, dan distribusi data sehingga pengelolaan aliran data dapat dilakukan secara standar dan dapat diaudit.

II.5.1 Apache Spark untuk *Batch Processing*

Apache Spark menyediakan *engine* terpadu untuk *batch processing*, SQL, dan analitik berskala besar. Zaharia dkk. (2016) menjelaskan bahwa Spark berbasis pada RDD yang terdistribusi, *immutable*, dan toleran kesalahan melalui mekanisme *lineage*. Karakteristik ini memungkinkan pemrosesan data besar secara andal dan mempermudah pengulangan komputasi ketika terjadi kegagalan. Dengan abstraksi yang konsisten, tim data dapat menggunakan antarmuka yang sama untuk berbagai jenis beban kerja analitik.

Spark SQL menggunakan *DataFrame* dan optimasi otomatis sehingga kueri analitik besar dapat dijalankan dengan efisien. **Spark MLlib** menyediakan algoritme ML terdistribusi yang bermanfaat untuk *baseline*, *feature pipeline*, ataupun pelatihan skala besar tanpa perlu membangun infrastruktur tambahan. Integrasi Spark dengan Kubernetes memungkinkan *executor* dijalankan sebagai *pod* yang elastis dan dapat diskalakan. *Spark Operator* menyediakan CRD sehingga aplikasi Spark dapat

dikelola secara deklaratif dan konsisten dengan pendekatan GitOps yang diterapkan pada komponen lain.

Dalam arsitektur penelitian ini, Spark dihubungkan dengan Feast untuk menulis fitur hasil pemrosesan *batch* ke ClickHouse (Bagian II.6.1). Di tingkat orkestrasi, lapisan seperti Optimus (Raystack 2025g) dapat dimanfaatkan untuk mendefinisikan *job* Spark dan transformasi data secara deklaratif. Lapisan ini juga membantu mengatur dependensi, jadwal eksekusi, dan pengujian kualitas data sehingga praktik DataOps yang sistematis dan terukur dapat diterapkan pada jalur pemrosesan *batch*.

II.5.2 Apache Flink untuk *Stream Processing*

Apache Flink dirancang untuk komputasi *stateful stream* dengan jaminan *exactly-once* dan latensi rendah. Carbone dkk. (2015) menjelaskan bahwa Flink memproses data secara kontinu tanpa menunggu akumulasi, namun tetap mendukung *batch* sebagai kasus khusus. Pendekatan ini sangat sesuai untuk data *tick* finansial yang terus mengalir dan memerlukan respons cepat. Dengan model pemrograman yang konsisten, tim data dapat merancang *pipeline* yang sama untuk pemrosesan terus-menerus maupun perhitungan agregat.

State management Flink menjaga konsistensi komputasi meskipun terjadi kegagalan, melalui *keyed state* atau *operator state*. **Event time processing** dengan *watermark* memungkinkan analitik yang akurat meskipun data datang terlambat, sementara dukungan *windowing* memfasilitasi perhitungan berbagai metrik teknikal secara *real-time* untuk fitur. Integrasi Flink dengan Kafka menyediakan *end-to-end exactly-once semantics* melalui *checkpointing*. Flink juga dapat menulis fitur langsung ke Redis Stack sebagai *online feature store* melalui Feast (Bagian II.6.2), sedangkan kerangka konfigurasi seperti Dagger (Raystack 2025b) menyederhanakan definisi *job* Flink yang digunakan berulang pada berbagai skenario.

II.5.3 Apache Kafka untuk *Event Streaming*

Apache Kafka merupakan *event streaming platform* yang lazim digunakan untuk membangun *real-time data pipeline* (Apache Software Foundation 2024c). Kleppmann (2017) menekankan bahwa model *event log* Kafka mendukung *throughput* tinggi, latensi rendah, dan *durability*. Desain partisi dan replikasi yang digunakan membuat Kafka cocok sebagai tulang punggung *streaming* pada perdagangan berfrekuensi tinggi dan *event-driven architecture*. Dengan struktur tersebut, data dapat dialirkan secara terukur ke berbagai konsumen tanpa bergantung pada integrasi khu-

sus yang rapuh.

Topic membagi *event* ke dalam partisi agar konsumsi dapat dilakukan secara paralel, dengan replikasi untuk toleransi kesalahan. *Producer* dan *Consumer* API mendukung berbagai jaminan pengiriman, dan *consumer group* memungkinkan konsumsi paralel yang terukur. **Kafka Connect** menyediakan *connector* siap pakai untuk mengalirkan data dari dan ke sistem eksternal sehingga Kafka dapat menghubungkan proses *ingest*, pemrosesan, dan *Feature Store* dalam alur yang konsisten. Pada implementasi praktis, kolektor peristiwa seperti Raccoon (Raystack 2025h) dan *schema registry* ringan seperti Stencil (Raystack 2025j) membantu menjaga evolusi *message schema* tetap tertata. *Sink service* seperti Firehose (Raystack 2025c) memudahkan penulisan *stream* ke berbagai tujuan tanpa perlu membangun integrasi khusus berulang kali.

II.6 Sistem Penyimpanan

Lapisan penyimpanan harus mampu melayani kueri historis untuk pelatihan sekaligus *lookup* berlatensi rendah untuk *online serving*. Selain itu, penyimpanan juga perlu menampung *artifact* besar seperti model, *checkpoint*, dan dataset mentah. Karena tidak ada satu sistem yang optimal untuk seluruh pola akses, arsitektur modern cenderung menggunakan beberapa jenis penyimpanan yang berbeda. Dengan memisahkan beban kerja analitik, *serving*, dan *object storage*, setiap jenis beban dapat dioptimalkan secara terpisah tanpa saling mengganggu.

Dalam penelitian ini, ClickHouse digunakan sebagai *offline feature store*, Redis Stack sebagai *online feature store* sekaligus *vector store*, dan MinIO sebagai *object storage* (ClickHouse Inc 2024; Redis Ltd. 2024; MinIO, Inc. 2024). Kombinasi tersebut dipilih agar kebutuhan *throughput*, latensi, dan kapasitas penyimpanan pada konteks perdagangan finansial dapat terpenuhi. Bagian-bagian berikut menguraikan karakteristik masing-masing sistem dan perannya dalam arsitektur yang diusulkan sehingga pemetaan peran dapat disesuaikan dengan pola penggunaan sebenarnya di sistem.

II.6.1 ClickHouse sebagai *Offline Feature Store*

ClickHouse adalah DBMS kolumnar *open-source* yang dioptimalkan untuk *Online Analytical Processing* (OLAP) pada volume data besar (ClickHouse Inc 2024). Format kolumnar sangat sesuai untuk kueri analitik yang membaca banyak baris namun hanya membutuhkan sejumlah kolom, sehingga I/O dan penggunaan memo-

ri dapat ditekan. Fitur kompresi ClickHouse memanfaatkan *codec* kolumnar untuk mengurangi ukuran data dan pada saat yang sama meningkatkan kinerja kueri.

Query engine ClickHouse menggunakan eksekusi tervektorisasi dan paralelisme kueri untuk memaksimalkan pemanfaatan CPU. *Partitioning* dan *sparse index* mendukung *data pruning* sehingga kueri tetap efisien ketika dataset bertumbuh besar. Sebagai *offline store* Feast, ClickHouse menyimpan fitur historis untuk penyusunan *training dataset*. *Materialization job* menulis fitur ke tabel ClickHouse, sementara *point-in-time correct join* dijalankan di atas kemampuan kuerinya sehingga kebutuhan validitas temporal dapat dipenuhi tanpa menambah kerumitan besar di lapisan aplikasi.

II.6.2 Redis Stack untuk *Online Feature Store* dan Pencarian Vektor

Redis Stack memperluas Redis dengan modul seperti RedisJSON, RedisTimeSeries, RedisBloom, dan RedisSearch sehingga mendukung *full-text* dan *vector similarity search* (Redis Ltd. 2024). Kombinasi modul-modul ini memungkinkan Redis Stack berfungsi sebagai *online feature store* sekaligus *vector database* berlatensi rendah yang relevan untuk *real-time inference* di sistem perdagangan dengan tuntutan *Service Level Agreement* (SLA) ketat. Dalam arsitektur ini, Redis Stack menjadi titik utama *online retrieval* untuk fitur tabular maupun berbasis *embedding*.

1. Arsitektur *Vector Database* dan Pengindeksan

Vector database menyimpan dan mengambil *embedding* berdimensi tinggi secara efisien, lengkap dengan *metadata* yang dapat dimanfaatkan untuk *filtering* dan *governance*. Dengan memanfaatkan indeks *Approximate Nearest Neighbor* (ANN), pencarian kemiripan dapat melayani permintaan dalam jumlah besar tanpa melanggar SLA latensi. **Algoritme Hierarchical Navigable Small Worlds (HNSW)** (Malkov dan Yashunin 2020) membangun graf berlapis yang menyeimbangkan kecepatan dan akurasi. Parameter seperti M , $efConstruction$, dan ef mengatur *trade-off* antara akurasi, latensi, dan konsumsi memori sehingga pemilihannya perlu disesuaikan dengan profil *workload*.

2. Jenis dan Karakteristik *Embedding*

Embedding yang digunakan pada sistem produksi dapat berupa *sparse embeddings* untuk pencarian leksikal, *dense embeddings* untuk pencarian semantik, maupun *quantized embeddings* untuk efisiensi memori. *Product Quantization* (Jegou, Douze, dan Schmid 2011) menawarkan kompromi antara efisiensi penyimpanan dan mutu hasil pencarian. Pemilihan jenis *embedding* bergantung

pada sifat sinyal yang dimanfaatkan, seperti teks berita, percakapan di media sosial, atau sinyal teknikal yang diturunkan dari pergerakan harga.

3. Implementasi Produksi Redis Stack

Redis Stack sebagai *online store* Feast menyediakan *feature retrieval* berlatensi rendah untuk *real-time inference*. Fitur dapat dimaterialisasikan dari ClickHouse atau dialirkan langsung dari *pipeline* Flink–Kafka sesuai kebutuhan *freshness*. Redis mendukung mekanisme persistensi *Redis Database* (RDB) dan *Append-Only File* (AOF), replikasi, Sentinel untuk *failover*, serta Redis Cluster untuk skala horizontal. RedisSearch menambahkan kemampuan pencarian vektor berbasis HNSW sehingga fitur berbasis *embedding* dapat diambil dengan pendekatan semantik. Untuk menjaga kinerja pencarian stabil, konfigurasi indeks dan *monitoring* perlu diatur dan dievaluasi secara berkala.

II.6.3 MinIO untuk *Object Storage*

MinIO merupakan *object storage* kompatibel S3 yang dirancang untuk lingkungan *cloud-native* (MinIO, Inc. 2024). Paradigma *object storage* sesuai untuk data berukuran besar dan relatif *immutable*, seperti dataset, *artifact* model, dan *log*. MinIO mendukung skala horizontal melalui mode terdistribusi dan *erasure coding* untuk menyediakan toleransi kesalahan yang memadai pada lingkungan produksi. Dengan demikian, data penting dapat disimpan dalam jangka panjang tanpa bergantung pada satu *node* saja.

Fitur seperti *bucket versioning* dan *lifecycle policy* membantu audit serta pengelolaan data historis dalam jangka panjang. Aspek keamanan didukung melalui enkripsi dan kontrol akses granular bergaya *Identity and Access Management* (IAM), sehingga akses terhadap objek dapat diatur secara detail. Dalam arsitektur penelitian ini, MinIO digunakan untuk menyimpan dataset mentah, *snapshot training dataset*, dan *artifact store* MLflow (Bagian II.4.1). MinIO juga dapat digunakan untuk menyimpan *log* dan data *monitoring* jangka panjang yang tidak perlu ditampung dalam *time-series database*, sehingga beban pada sistem lain dapat dikurangi.

II.7 Tata Kelola Data dan *Observability*

Tata kelola data dan *observability* berperan menjaga kepercayaan, kepatuhan, dan kestabilan operasi sistem ML. *Governance* mengatur pengelolaan dan audit aset data maupun model agar selaras dengan kebijakan dan regulasi, sementara *observability* menyediakan visibilitas berkelanjutan untuk mendeteksi masalah secara proaktif

(Bernardo dkk. 2024; Stoyanovich, Howe, dan Jagadish 2020; Cloud Native Computing Foundation 2024; IBM 2024). Keduanya perlu direncanakan sejak awal, terutama pada domain finansial yang berada di bawah pengawasan regulator. Tanpa fondasi tersebut, risiko penyalahgunaan data dan kegagalan sistem akan lebih sulit dikendalikan dan dijelaskan.

Di atas lapisan tersebut, mekanisme keamanan dan akses (autentikasi, otorisasi, dan manajemen rahasia) perlu diselaraskan dengan *governance* agar pemanfaatan data dan model selalu sesuai hak akses serta kebijakan organisasi. Dalam arsitektur yang diusulkan, DataHub berperan sebagai platform *metadata management* terpusat. Frontier (Raystack 2025d) dan Guardian (Raystack 2025e) menyediakan fondasi autentikasi dan pengelolaan akses data. Kong (Kong Inc. 2025) dan *API gateway* lain menangani eksposur layanan ke luar sistem. Di sisi *observability*, Prometheus–Grafana, Jaeger (Jaeger Authors 2024), Loki (Grafana Labs 2024b), dan Siren (Raystack 2025i) membentuk fondasi *monitoring*, *tracing*, log terpusat, dan *alerting* sehingga perilaku sistem dapat diamati secara menyeluruh dan dipertahankan dalam batas SLA yang disepakati.

II.7.1 DataHub untuk *Metadata Management*

DataHub adalah platform *metadata open-source* yang menyajikan pandangan terpadu atas aset data organisasi (DataHub Project 2024). DataHub mengumpulkan *metadata* dari berbagai sistem, kemudian menyusunnya menjadi katalog yang memuat konteks dan relasi antar aset sehingga proses *discovery* data menjadi lebih terarah. Model *metadata* DataHub berbasis graf dan *extensible*, sehingga dapat diperluas untuk mencakup artefak ML seperti *feature view* dan model tanpa mengubah struktur dasar sistem secara drastis.

Fitur *lineage tracking* pada DataHub merekam aliran data dari sumber hingga titik konsumsi akhir, sehingga *impact analysis* dan *debugging* menjadi lebih mudah. Mekanisme pencarian menggabungkan *full-text search*, *faceted navigation*, dan *filtering* agar pengguna dapat menemukan aset yang relevan dengan cepat. DataHub juga dapat mencatat metrik kualitas seperti *freshness* dan perubahan *schema*. Dalam arsitektur ini, definisi fitur di Feast, model di MLflow, dan *pipeline* Kubeflow di *ingest* ke DataHub sehingga terbentuk *lineage* menyeluruh dari data hingga model produksi. Kolektor *metadata* generik seperti Meteor (Raystack 2025f) dapat dimanfaatkan untuk mengekstrak *metadata* dari berbagai sumber dan mengirimkannya ke katalog seperti DataHub atau Compass (Raystack 2025a).

II.7.2 Prometheus dan Grafana untuk *Monitoring*

Prometheus adalah sistem *monitoring time-series open-source* yang banyak digunakan di lingkungan produksi (Prometheus Authors 2024). Prometheus mengumpulkan metrik dengan model *pull* dan menyimpan data dalam format multidimensi berbasis *label*. Bahasa kueri **PromQL** memungkinkan agregasi metrik yang fleksibel, perhitungan tren, dan membentuk dasar aturan *alert*. Komponen *Alertmanager* mengatur pengiriman peringatan ke berbagai kanal sekaligus membantu menekan *alert fatigue* melalui pengelompokan dan deduplikasi.

Grafana menyediakan *dashboard* dengan visualisasi kaya untuk metrik Prometheus (Grafana Labs 2024a). Fitur *dashboard templating* dan variabel memungkinkan *filtering* dinamis berdasarkan *namespace*, layanan, atau atribut lain. Praktik *observability* modern menggabungkan *metrics*, *logging*, dan *tracing*. Dalam arsitektur ini, Prometheus dan Grafana diintegrasikan dengan Jaeger (Jaeger Authors 2024) untuk *distributed tracing* dan Loki (Grafana Labs 2024b) untuk agregasi log, sehingga *request path*, *latency*, dan pesan log dapat dianalisis secara terpadu. Di sisi *alerting*, sistem seperti Siren (Raystack 2025i) memanfaatkan metrik dan *alert* dari Prometheus untuk mengirim notifikasi terpusat ke kanal operasional yang digunakan tim.

II.7.3 Keamanan, Autentikasi, dan Manajemen Akses

Selain *governance* dan *observability*, platform DataOps dan MLOps di lingkungan produksi memerlukan lapisan keamanan yang kuat. Komponen autentikasi terpadu seperti *identity provider* dan *single sign-on* (SSO) memastikan bahwa pengguna dan layanan yang mengakses platform telah terverifikasi dengan benar. Di atasnya, sistem otorisasi seperti *role-based access control* dan *policy-based access* mengatur hak akses ke dataset, *feature view*, model, dan *pipeline*. Dengan cara ini, hak akses dapat disesuaikan dengan peran dan tanggung jawab masing-masing pengguna.

API gateway seperti Kong (Kong Inc. 2025) menegakkan kontrol keamanan di batas sistem dengan menyediakan *rate limiting*, autentikasi, dan *request filtering* untuk seluruh *microservices*. Manajemen rahasia (*secrets management*) melalui alat seperti Vault (HashiCorp 2025) mencegah penyimpanan kredensial secara terbuka di konfigurasi. Dalam penelitian ini, komponen autentikasi dan otorisasi seperti Frontier (Raystack 2025d) dan Guardian (Raystack 2025e) diposisikan sebagai pintu masuk ke aset data dan konfigurasi platform. Kedua komponen ini diperlakukan sebagai bagian integral dari lapisan tata kelola dan keamanan data yang menyatu dengan

arsitektur MLOps secara keseluruhan.

II.8 GitOps dan CI/CD untuk Platform DataOps dan MLOps

Praktik GitOps memanfaatkan repositori Git sebagai *single source of truth* untuk konfigurasi infrastruktur dan aplikasi. Seluruh *manifest* Kubernetes, konfigurasi *pipeline*, dan definisi layanan dikelola sebagai kode dan melalui proses *review* yang sama dengan kode aplikasi. Pendekatan ini meningkatkan auditabilitas, mengurangi risiko konfigurasi *drift*, dan memudahkan *rollback* ketika terjadi kegagalan. Setiap perubahan konfigurasi selalu tercatat dengan jelas dalam sejarah versi sehingga proses audit menjadi lebih sederhana.

Argo CD merupakan salah satu alat GitOps *cloud-native* yang memantau repositori Git dan memastikan keadaan kluster Kubernetes selaras dengan deklarasi yang ada. Argo CD menyediakan tampilan visual untuk melihat perbedaan konfigurasi, status *sync*, dan riwayat *deployment*. Fitur ini membantu tim operasi memahami perubahan yang terjadi dan menindaklanjuti jika ada penyimpangan. Untuk *source code management*, platform Git *self-hosted* seperti Gitea (Gitea Contributors 2025) menyediakan fungsi repositori Git, *pull request*, dan *issue tracking* tanpa bergantung pada layanan *hosted* komersial.

Pada sisi CI, kerangka seperti Tekton (Tekton Contributors 2024) menawarkan *pipeline* CI/CD berbasis Kubernetes yang menyusun langkah-langkah *build*, *test*, dan *image publishing* sebagai *task* dan *pipeline* deklaratif. Integrasi Tekton dengan Argo CD memungkinkan alur otomatis dari *code commit* hingga *deployment* ke kluster berjalan dengan konsisten dan dapat diaudit. Kombinasi GitOps dan CI/CD ini mendukung implementasi MLOps Level 2 yang menekankan integrasi CI/CD/CT serta menyatukan perubahan pada kode, konfigurasi, dan model ke dalam satu jejak audit yang terkelola dan dapat ditelusuri kembali.

II.9 Penelitian dan Praktik Terkait

Telaah penelitian terkait diperlukan untuk memahami *state-of-the-art* DataOps dan MLOps sekaligus mengidentifikasi celah riset yang relevan dengan studi kasus perdagangan saham dan kripto. Banyak publikasi membahas komponen individual seperti *Feature Store*, orkestrasi, atau deteksi *drift*, tetapi relatif sedikit yang menyajikan rancangan arsitektur terintegrasi dari hulu hingga hilir. Penelitian ini memanfaatkan celah tersebut dengan memfokuskan kontribusi pada rancangan arsitektur komprehensif berbasis *open-source* yang dapat direplikasi. Di sisi lain, praktik in-

dustri mulai mengadopsi ekosistem alat *cloud-native* yang saling terintegrasi, misalnya kumpulan komponen Raystack (Raystack 2025h, 2025j, 2025c, 2025g, 2025b, 2025i, 2025f, 2025a, 2025d, 2025e), *Feature Store*, orkestrator ML, dan *observability stack*. Namun, dokumentasi akademis untuk praktik ini masih terbatas dan sering tersebar sehingga perlu disusun kembali dalam bentuk yang lebih sistematis.

II.9.1 Tinjauan Arsitektur MLOps dan Tingkat Kematangan

Kreuzberger, Kühl, dan Hirschl (2023) melakukan *review* terhadap 27 publikasi dan merumuskan sembilan komponen inti arsitektur MLOps beserta tingkat kematangannya, dengan Kubernetes sebagai platform dominan. Mereka menyoroti pentingnya integrasi komponen seperti *Feature Store*, *model registry*, dan deteksi *drift* untuk membangun platform ujung-ke-ujung yang dapat dioperasikan secara konsisten. Amou Najafabadi dkk. (2024) memetakan rancangan arsitektur MLOps di literatur dan praktik industri, sementara Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mengevaluasi alat MLOps *open-source* dan pola integrasinya.

Ketiga studi tersebut menunjukkan bahwa banyak organisasi masih merangkai *stack* MLOps secara manual dengan integrasi yang relatif *loosely coupled*. Akibatnya, platform yang terbentuk memang fungsional, tetapi sulit dipelihara dan direplikasi ke konteks lain. Temuan ini menegaskan relevansi penelitian yang mengusulkan *blueprint* arsitektur terintegrasi berbasis Kubernetes dengan pemilihan komponen *open-source best-of-breed*. Rancangan yang lebih sistematis diharapkan dapat menurunkan biaya integrasi dan mempercepat adopsi MLOps yang lebih matang.

II.9.2 Tantangan Siklus Hidup Data dan Validitas Temporal

Polyzotis dkk. (2018) menyoroti bahwa kebocoran temporal merupakan salah satu penyebab utama penurunan kinerja model saat berpindah dari lingkungan eksperimen ke produksi. Mereka menekankan pentingnya *point-in-time correctness* dan *data versioning* untuk mendukung reproduksibilitas eksperimen. Tanpa mekanisme ini, evaluasi model dapat menghasilkan gambaran kinerja yang terlalu optimistis dan tidak mencerminkan kondisi produksi yang sebenarnya. Meskipun survei tersebut dilakukan sebelum *Feature Store* dan kontainer *real-time* menjadi populer, prinsip validitas temporal yang dibahas tetap relevan hingga sekarang.

Penelitian ini memperluas prinsip tersebut dengan mengaitkannya pada arsitektur *dual-store Feature Store* dan orkestrasi Kubernetes untuk menangani data finansial berkecepatan tinggi. Dengan memanfaatkan kombinasi Feast, ClickHouse, Redis

Stack, dan *stream processing*, rancangan ini berusaha menyajikan jalur konsisten dari data mentah hingga fitur produksi yang menjaga integritas temporal. Pendekatan ini diharapkan dapat mengurangi kesenjangan antara evaluasi model di laboratorium dan performa di lingkungan nyata dalam konteks perdagangan saham dan kripto.

II.9.3 Deteksi *Drift* dan Degradasi Model

Céspedes Sisniega dkk. (2024) mengusulkan pendekatan deteksi *covariate drift* yang efisien dengan memanfaatkan *threshold* adaptif pada arsitektur *serverless*. Mereka menunjukkan bahwa elastisitas komputasi membantu pengendalian biaya ketika volume data berfluktuasi, tetapi integrasi *drift detector* dengan *Feature Store* dan *model registry* belum dijelaskan secara rinci. Akibatnya, jalur operasional dari deteksi ke tindakan korektif masih longgar dan memerlukan perancangan tambahan di sisi platform.

Bayram, Ahmed, dan Kassler (2022) meninjau berbagai *drift detector* dan membedakan *feature drift* dari *concept drift*. Mereka menekankan perlunya *monitoring* proaktif berbasis metrik yang tepat, penentuan ambang *alert* yang proporsional, dan mekanisme untuk mencegah *alert fatigue*. Temuan ini menjadi rujukan dalam merancang *observability* pada penelitian ini. Arsitektur yang diusulkan menghubungkan metrik dari *Feature Store*, *streaming*, dan *model serving* dengan sistem *alerting* terpusat sehingga deteksi *drift* tidak berhenti pada identifikasi, tetapi dikaitkan dengan aksi korektif yang terintegrasi di *pipeline*.

II.9.4 *Feature Store* dan Arsitektur *Dual-Store*

Arsitektur *dual-store* telah menjadi pola umum dalam desain *Feature Store*, dengan memadukan *offline store* untuk data historis dan *online store* untuk *serving* berlatensi rendah (Li dkk. 2023). Konsistensi antara kedua lapisan dan dukungan *point-in-time correctness* sangat penting untuk mengurangi *training–serving skew*. Munappy dkk. (2022) menyoroti kebutuhan DataOps yang matang pada *deep learning*, sedangkan Lamer dkk. (2024) menempatkan *Feature Store* sebagai repositori fitur terkurasi dengan *metadata* dan *versioning*.

Prinsip-prinsip tersebut sejalan dengan kebutuhan domain finansial yang memerlukan jejak data yang jelas dan dapat diaudit. Penelitian ini memanfaatkan konsep *dual-store* dengan menggabungkan ClickHouse dan Redis Stack yang diorkestrasi oleh Feast, sehingga fitur historis dan *real-time* dapat dikelola sebagai bagian dari satu platform DataOps dan MLOps. Dengan demikian, pengelolaan fitur menjadi

lebih konsisten dan mudah ditelusuri dalam lingkungan yang sangat dinamis dan peka terhadap waktu.

II.9.5 *Streaming dan Near Real-Time MLOps*

Rodrigues dkk. (2025) mengusulkan arsitektur MLOps untuk *near real-time distributed stream learning* yang mengintegrasikan pemrosesan *streaming* dan pembaruan model cepat. Mereka menunjukkan bahwa frekuensi pembaruan model yang lebih tinggi dapat meningkatkan responsivitas terhadap perubahan distribusi data dan mengurangi *concept drift*. Namun, arsitektur tersebut belum mengulas secara mendalam konsistensi fitur *dual-store*, integrasi *governance*, dan kebutuhan *multi-tenancy* yang sering muncul di lingkungan perusahaan.

Hal ini menunjukkan perlunya rancangan yang menggabungkan kekuatan *streaming* dengan *Feature Store*, katalog *metadata*, dan komponen *governance* untuk mencapai kesiapan tingkat perusahaan. Penelitian ini memanfaatkan Kafka, Flink, dan Feast dalam kerangka Kubernetes untuk membentuk jalur *near real-time* yang tetap selaras dengan persyaratan validitas data dan kepatuhan. Dengan pendekatan ini, pemrosesan dan pemodelan *real-time* dapat dilekatkan ke platform MLOps yang lebih utuh dan siap dioperasikan.

II.9.6 *Kubernetes untuk ML Workloads*

Lakka (2025) menguraikan strategi praktis *deploying workload* ML di Kubernetes, termasuk pemanfaatan GPU, pengaturan *resource requests/limits*, dan *auto-scaling*. Ahmad dkk. (June 2024) menyoroti aspek keamanan dalam skenario *multi-tenant*, sementara Adusumilli (2025) menjelaskan arah perkembangan menuju *serverless Kubernetes*. Ketiga sumber tersebut memperkuat pandangan bahwa Kubernetes relevan sebagai fondasi MLOps, terutama untuk mengelola *workload* yang beragam dan berubah-ubah.

Namun, literatur tersebut umumnya lebih menekankan aspek orkestrasi dan keamanan, dan belum secara detail menguraikan integrasi ujung-ke-ujung dengan *Feature Store*, *model registry*, dan *observability stack*. Penelitian ini berupaya mengisi celah tersebut dengan mengaitkan praktik terbaik Kubernetes ke dalam rancangan arsitektur DataOps dan MLOps yang lengkap untuk perdagangan finansial. Dengan demikian, Kubernetes tidak hanya berperan sebagai platform eksekusi, tetapi sebagai komponen yang terintegrasi dalam keseluruhan ekosistem MLOps.

II.9.7 Operasionalisasi dan *Deployment* ML

Shankar dkk. (2022) mengidentifikasi sejumlah tantangan praktis dalam operasionalisasi ML, seperti integrasi alat, pengelolaan *dependency*, dan efektivitas *monitoring*. Mereka menekankan bahwa strategi *canary release* dan *A/B testing* sangat penting untuk menekan risiko ketika mempromosikan model baru, terutama di lingkungan dengan SLA ketat. Di sisi lain, Hsu dkk. (2024) menunjukkan bahwa otomatisasi siklus hidup model *end-to-end* dapat mencapai tingkat kesetaraan dengan MLOps Level 2.

Pendekatan Hsu dkk. (2024) menekankan *closed loop* dari deteksi perubahan data hingga *deployment* otomatis, namun implementasinya masih sangat terkait dengan platform tertentu. Agar lebih luas diadopsi, konsep tersebut perlu diterjemahkan ke dalam *stack open-source* yang lebih generik. Penelitian ini mengambil pelajaran dari pendekatan tersebut dan mengimplementasikannya di atas Kubernetes dengan komponen *open-source*, sehingga hasilnya lebih mudah direplikasi dan diadaptasi oleh berbagai organisasi dengan infrastruktur yang berbeda.

II.9.8 *Governance* dan Reprodusibilitas

Stoyanovich, Howe, dan Jagadish (2020) menekankan pentingnya *responsible data management* melalui *lineage tracking*, kontrol akses, dan *audit logging*. Ketiga aspek ini memastikan data dan model dapat ditelusuri, diaudit, dan digunakan secara patuh terhadap regulasi. Pimentel dkk. (2019) menunjukkan bahwa banyak *Jupyter notebooks* sulit direproduksi karena *dependency* dan konfigurasi tidak terdokumentasi dengan baik, sehingga rekayasa eksperimen menjadi rapuh dan sulit diulang.

Temuan tersebut memperkuat perlunya *experiment tracking* dan pengelolaan lingkungan eksekusi dalam platform MLOps. Dalam penelitian ini, kebutuhan *governance* dan reprodusibilitas diakomodasi melalui kombinasi MLflow, DataHub, GitOps, dan *object storage* MinIO. Kombinasi ini memungkinkan setiap model dan dataset memiliki jejak asal-usul dan konteks penggunaan yang jelas sehingga organisasi dapat menjelaskan bagaimana suatu model dibangun, dari data apa, dan dalam kondisi seperti apa model tersebut digunakan di produksi.

II.9.9 Analisis Celah Riset dan Kontribusi Penelitian Ini

Literatur menunjukkan bahwa komponen DataOps dan MLOps banyak diteliti secara terpisah, namun arsitektur yang komprehensif dan terintegrasi dari hulu ke hilir masih terbatas. Akibatnya, organisasi sering kali harus merangkai sendiri *stack*

open-source yang heterogen dengan biaya rekayasa tinggi dan risiko integrasi yang tidak konsisten. Kesenjangan ini semakin terasa ketika memasukkan kebutuhan domain finansial, seperti data non-stasioner, SLA latensi yang ketat, serta kombinasi sinyal terstruktur dan tidak terstruktur.

Pada domain perdagangan finansial, arsitektur yang tersedia belum sepenuhnya dioptimalkan untuk *point-in-time correctness* dengan *throughput* tinggi, *vector similarity search* dengan latensi stabil, dan deteksi *drift* otomatis yang sesuai dengan karakter data finansial. Solusi yang ada umumnya bersifat parsial sehingga memerlukan rekayasa tambahan untuk membentuk platform yang stabil. Penelitian ini merespons celah tersebut dengan mengusulkan dan mengevaluasi sebuah *blueprint* arsitektur DataOps dan MLOps terintegrasi berbasis Kubernetes dengan menggunakan *open-source tools*. *Blueprint* ini merangkai komponen seperti *Feature Store*, orkestrasi *pipeline*, *model registry*, *stream processing*, *vector store*, *observability stack*, dan ekosistem komponen DataOps yang lebih luas (termasuk alat *ingest* seperti Raccoon (Raystack 2025h), transformasi seperti Optimus (Raystack 2025g) dan Dagger (Raystack 2025b), *sink* seperti Firehose (Raystack 2025c), dan katalog *metadata* seperti DataHub atau Compass (Raystack 2025a)). Rancangan ini dioptimalkan untuk perdagangan finansial, tetapi tetap dapat diadaptasi ke domain lain yang memiliki karakteristik dan kebutuhan serupa.

BAB III

ANALISIS MASALAH

III.1 Analisis Kondisi Saat Ini

Analisis kondisi saat ini dilakukan untuk memetakan adopsi DataOps dan MLOps di institusi finansial yang sudah lebih maju. Mengacu pada kerangka sistem, ekosistem DataOps–MLOps dipandang sebagai himpunan subsistem yang saling berinteraksi, meliputi jalur *data ingestion*, pemrosesan *batch* dan *streaming*, *feature store* dan penyajian fitur, *training pipeline*, *model serving*, serta lapisan *monitoring* dan *governance*. Interaksi antarsubsistem tersebut membentuk model konseptual sistem produksi ML yang menjadi dasar untuk mengidentifikasi titik lemah arsitektural maupun operasional. Hasil analisis ini digunakan untuk mengenali pola praktik industri, kesenjangan yang masih muncul, serta area yang memerlukan penguatan agar rancangan solusi tidak berhenti pada perbaikan parsial. Selain itu, gambaran situasi ini membantu memastikan solusi yang diusulkan tetap realistis untuk diintegrasikan ke ekosistem yang sudah berjalan di organisasi nyata.

Berbagai studi kasus menunjukkan bahwa persoalan data, integrasi, dan operasi merupakan hambatan utama ketika organisasi memindahkan model ML ke produksi. Paleyes, Urma, dan Lawrence (2022) menyoroti bahwa banyak kendala tersebut bersumber dari arsitektur yang terfragmentasi dan ketiadaan pola integrasi yang baku. Di sisi lain, Vangala, Mehta, dan Singh (2022) menegaskan bahwa MLOps perlu dipandang sebagai kerangka operasional menyeluruh yang mencakup siklus hidup model AI dari awal hingga pensiun. Integrasi *Continuous Integration* (CI), *Continuous Deployment* (CD), dan *Continuous Training* (CT) menjadi pembeda utama dibanding DevOps tradisional yang umumnya hanya berfokus pada perangkat lunak deterministik. Kondisi ini menunjukkan pentingnya pemahaman situasi saat ini sebagai pijakan sebelum merancang arsitektur yang lebih terintegrasi.

III.1.1 Implementasi MLOps di Institusi Finansial

Penerapan DataOps dan MLOps pada aplikasi perdagangan finansial mutakhir menunjukkan tingkat kematangan yang beragam antar organisasi. Dalam praktik, tingkat kematangan tersebut tersebar di berbagai spektrum dan dihadapkan pada hambatan adopsi di level organisasi, di tingkat sistem ML, maupun pada lapisan operasional harian. Akibatnya, banyak *pipeline* yang belum sepenuhnya otomatis dan ma-

sih membutuhkan intervensi manual dalam berbagai tahap. Kajian terhadap publikasi industri dan studi kasus memperlihatkan pola umum implementasi yang dapat dijadikan cerminan situasi di lapangan, di mana perbedaan skala organisasi, kemampuan teknis, dan strategi bisnis sangat memengaruhi bentuk akhir platform MLOps (Paleyes, Urma, dan Lawrence 2022; Kartakis dkk. June 2022; Vangala, Mehta, dan Singh 2022).

Institusi finansial besar secara konsisten berinvestasi dalam pengembangan platform ML internal yang dibangun *in-house*. Lembaga-lembaga ini mengembangkan platform untuk *algorithmic trading* yang memanfaatkan *reinforcement learning* dan analitik perdagangan canggih sebagai sumber keunggulan kompetitif. Pembangunan platform biasanya dilakukan melalui investasi multi-tahun dengan dukungan tim rekayasa berskala besar, sehingga dapat mencapai tingkat otomasi dan integrasi yang tinggi. Konsekuensinya, biaya pengembangan dan pemeliharaan menjadi sangat besar, tetapi organisasi memperoleh fleksibilitas dan kontrol hampir penuh atas seluruh rantai teknis. Pendekatan ini hanya realistis bagi institusi dengan kapasitas pendanaan dan sumber daya teknis yang kuat.

Berbeda dengan itu, **bank menengah dan perusahaan *fintech*** cenderung memilih pendekatan *hybrid* yang menggabungkan layanan *managed cloud* dengan komponen *open-source* untuk kebutuhan yang lebih spesifik. Banyak institusi memanfaatkan Amazon SageMaker atau Google Vertex AI sebagai tulang punggung, lalu menambahkan alat tambahan untuk *feature engineering* atau *monitoring* sesuai kebutuhan strategi perdagangan masing-masing. Pendekatan *hybrid* ini mencoba menyeimbangkan kemudahan operasional yang disediakan layanan terkelola dengan kebutuhan kendali yang lebih besar di area tertentu. Namun, pola ini juga memunculkan tantangan integrasi antara lingkungan yang sepenuhnya *managed* dan komponen yang dikelola sendiri, sejalan dengan hambatan integrasi *open-source stack* yang disorot Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025).

Sementara itu, ***hedge fund*** dan firma perdagangan kuantitatif umumnya memilih pendekatan yang sangat kustom dengan infrastruktur yang hampir seluruhnya dibangun *in-house*. Platform yang digunakan dirancang untuk mencapai latensi serendah mungkin dan *throughput* setinggi mungkin, sehingga menuntut investasi besar pada infrastruktur serta keahlian teknis yang sangat khusus. Karena sifat industrinya yang kompetitif dan bertumpu pada keunggulan algoritmik, rincian implementasi teknis di lingkungan ini jarang dibuka ke publik. Praktik terbaik di sektor tersebut lebih sering disimpulkan secara tidak langsung dari publikasi teknis yang terbatas atau

referensi tingkat tinggi (Paleyes, Urma, dan Lawrence 2022; Munappy dkk. 2022). Kondisi ini memperlebar jarak antara praktik faktual di lapangan dengan dokumentasi akademik yang tersedia dan memperkuat kebutuhan *blueprint* arsitektur yang dapat diakses lebih luas.

III.1.2 Arsitektur Umum yang Diterapkan

Analisis terhadap pola implementasi memperlihatkan sejumlah karakteristik arsitektur yang berulang di berbagai institusi. Pada **lapisan infrastruktur**, sistem umumnya dijalankan di atas *cloud provider* seperti AWS, GCP, atau Azure, atau di pusat data *on-premise*, dengan tingkat otomasi yang bervariasi. Kubernetes sering dipilih sebagai fondasi platform MLOps dan dipadukan dengan kombinasi alat *open-source* dan solusi *enterprise*. Cara organisasi memanfaatkan Kubernetes sangat beragam, mulai dari sekadar orkestrasi kontainer sampai menjadi landasan platform MLOps terintegrasi. Perbedaan tingkat pemanfaatan ini berpengaruh langsung pada cara tim membangun dan mengoperasikan *pipeline* produksi.

Pada **lapisan pemrosesan data**, institusi umumnya mengandalkan Apache Spark untuk *batch processing* (Zaharia dkk. 2016), serta Apache Kafka (Apache Software Foundation 2024c) dan Apache Flink (Carbone dkk. 2015) untuk *stream processing*. Saat merancang arsitektur, organisasi sering dihadapkan pada pilihan antara *Lambda Architecture* dan *Kappa Architecture*. *Lambda Architecture* Marz dan Warren (2015) mengusulkan pendekatan dua jalur dengan *batch layer* untuk akurasi dan *speed layer* untuk latensi, tetapi menambah beban operasional karena logika bisnis harus digandakan di dua jalur (Kreps 2014). Sebaliknya, *Kappa Architecture* Kreps (2014) menyederhanakan arsitektur dengan hanya mengandalkan *stream processing*, tetapi menuntut *stream processing framework* yang matang dan mampu menangani *reprocessing* historis dalam skala besar.

Rodrigues dkk. (2025) mencatat bahwa integrasi antara *batch pipeline* dan *stream pipeline* kerap tidak mulus karena adanya duplikasi logika dan kesulitan menjaga konsistensi. Di lapangan, *data engineer* merancang *pipeline ingestion* dari berbagai sumber seperti *market data feed*, *news API*, media sosial, dan *alternative data provider*. Namun, *pipeline* tersebut sering belum ditopang oleh pola *deployment* dan *monitoring* yang konsisten, sehingga gangguan operasional baru diketahui setelah dampaknya muncul di hilir. Situasi ini meningkatkan risiko inkonsistensi dan kegagalan yang sulit ditelusuri akar masalahnya, terutama ketika volume dan kecepatan data terus meningkat.

III.1.3 Tantangan Fragmentasi dan Kesenjangan Fungsional

Meskipun banyak institusi telah mengalokasikan sumber daya besar untuk infrastruktur ML, masalah fragmentasi dan kesenjangan fungsional masih sering muncul. Polyzotis dkk. (2018) menunjukkan bahwa ketiadaan repositori fitur terpusat membuat proses rekayasa fitur dan *reuse* fitur cenderung berulang dan tidak efisien. Di sisi lain, keberadaan *model registry* yang berperan sebagai penghubung antara *training pipeline* dan *deployment pipeline* memungkinkan penerapan *continuous training* dan pengelolaan *lifecycle* model yang lebih sistematis. Registry menyediakan satu sumber kebenaran atas versi model sehingga setiap perubahan dapat direkam dan dilacak dengan jelas.

Dari sudut pandang arsitektur, *model registry* membantu memisahkan (*decouple*) *pipeline* pelatihan dari *pipeline* inferensi dan menurunkan *technical debt* tersembunyi yang diidentifikasi Sculley dkk. (2015). Pendekatan ini mengelola *lineage* model dan dependensi sebagai metadata kelas satu, bukan informasi sampingan. Tanpa komponen terpusat seperti ini, perubahan kecil pada data atau kode dapat memengaruhi model produksi tanpa tercatat dengan baik, sehingga risiko kesalahan dan hambatan audit meningkat. Secara ringkas, bentuk fragmentasi dan kesenjangan yang lazim ditemukan dapat dirangkum sebagai berikut.

1. Fragmentasi *Feature Engineering* dan *Training–Serving Skew*

Di banyak sistem, proses rekayasa fitur masih dijalankan secara *ad hoc* oleh tim yang berbeda, dengan logika komputasi tersebar di Jupyter *Notebook*, skrip Python, atau aplikasi kustom. Tidak adanya *feature store* terpusat menyulitkan *reuse* fitur yang telah dibangun sebelumnya dan mendorong munculnya definisi fitur ganda dengan makna serupa. *Training–serving skew* muncul ketika logika perhitungan fitur pada fase pelatihan tidak identik dengan logika yang digunakan saat *inference* di produksi. Kondisi ini menurunkan kinerja model ketika dijalankan di lingkungan nyata, menyulitkan diagnosis masalah, dan menambah beban pemeliharaan jangka panjang.

2. Keterbatasan *Experiment Tracking* dan Reprodusibilitas

Pada banyak organisasi, *experiment tracking* belum menjadi praktik standar dan sering kali tidak dilakukan secara disiplin. Parameter, konfigurasi, dan hasil eksperimen kerap disimpan pada *spreadsheet*, wiki internal, atau bahkan hanya di *notebook* pribadi pengembang. Pimentel dkk. (2019) menemukan bahwa hanya 24,11% *notebook* publik yang dapat dijalankan ulang tanpa galat dan hanya 4,03% yang benar-benar dapat mereplikasi hasil, yang meng-

gambarkan lemahnya reproduksibilitas ketika eksperimen dan artefak tidak tercatat secara sistematis. Situasi serupa mudah terjadi di lingkungan internal jika *experiment tracking* tidak diperlakukan sebagai bagian inti MLOps.

3. **Proses *Deployment* Manual dan Tantangan Skalabilitas**

Di banyak arsitektur, proses *deployment* dan *model serving* masih bergantung pada infrastruktur kustom atau kerangka API generik yang dirakit bertahun-tahun sebelumnya. Shankar dkk. (2022) mencatat bahwa proses *deployment* sering kali masih manual atau hanya semiotomatis dan memerlukan koordinasi intensif lintas peran. Dukungan terhadap pola *canary release*, *A/B testing*, atau *blue-green deployment* umumnya belum tersedia secara *built-in* dan jika pun ada, sering hanya diimplementasikan secara *ad hoc*. Penskalaan layanan juga kerap diatur secara statis, sehingga sistem sulit beradaptasi terhadap perubahan beban dan pemanfaatan sumber daya menjadi kurang efisien.

4. ***Monitoring* Reaktif dan Degradasi Model Senyap**

Pada banyak sistem, *monitoring* masih berfokus pada metrik infrastruktur seperti CPU, memori, dan latensi, sedangkan visibilitas metrik spesifik ML masih terbatas. Kerangka tiga pilar *Cloud Native Computing Foundation* (CNCF) mencakup **metrics**, **logs**, dan **traces**, tetapi instrumentasi yang secara eksplisit mendukung model ML belum banyak diadopsi. Tanpa dukungan ini, penurunan kualitas prediksi, *data drift*, dan *concept drift* sering terlewat sampai dampaknya dirasakan pada indikator bisnis.

Lingkungan finansial yang nonstasioner menuntut *monitoring drift* yang proaktif agar penurunan kinerja dapat terdeteksi sebelum memengaruhi hasil perdagangan secara signifikan. Dalam praktik, degradasi model masih sering diketahui lewat indikator bisnis seperti penurunan *P&L*, bukan melalui *monitoring* otomatis yang terstruktur. Pola ini membuat respons terhadap masalah menjadi terlambat dan menambah risiko bagi strategi perdagangan.

5. **Kesenjangan *Governance* dan Risiko *Compliance***

Pada banyak sistem, praktik tata kelola dan *compliance* masih bergantung pada proses manual dan dokumentasi yang tersebar. Stoyanovich, Howe, dan Jagadish (2020) menekankan pentingnya *lineage* otomatis dan *governance* menyeluruh sepanjang *lifecycle* sistem keputusan otomatis. Tanpa kemampuan ini, organisasi kesulitan menelusuri asal-usul data, transformasi yang dilakukan, dan bagan tanggung jawab atas keputusan yang dihasilkan model. Kontrol akses cenderung *coarse-grained*, *audit trail* tidak lengkap, dan analisis forensik menjadi lambat, sehingga risiko kepatuhan meningkat di ling-

kungan yang diatur ketat oleh regulator.

III.1.4 Tantangan Spesifik Domain Perdagangan

Pada domain perdagangan saham dan *cryptocurrency*, terdapat tantangan tambahan yang bersumber dari karakteristik pasar yang sangat dinamis. Data pasar mengalir dengan kecepatan tinggi, dengan *tick data* yang dapat mencapai ratusan ribu hingga jutaan *event* per detik. Dalam konteks ini, arsitektur *event-driven* menjadi pendekatan kunci untuk menangani *real-time data streaming*. Arsitektur ini menggunakan pola komunikasi asinkron berbasis *event* untuk membangun *loose coupling*, skalabilitas horizontal, dan responsivitas tinggi (Amazon Web Services 2024; Stopford 2018). Amazon Web Services (AWS) mendefinisikan arsitektur *event-driven* sebagai arsitektur yang menggunakan *event* untuk memicu dan menghubungkan layanan yang terdekopel, sehingga produksi, deteksi, konsumsi, dan reaksi terhadap *event* dapat dikelola secara terpisah.

Komponen inti arsitektur *event-driven* meliputi *event producer*, *event broker/router* (misalnya Apache Kafka), dan *event consumer* yang memproses *event* secara asinkron untuk mencapai *fault tolerance* dan skalabilitas (Stopford 2018). Persyaratan latensi pada domain ini sangat ketat karena proses *inference* sering harus tuntas dalam satu digit milidetik. Isu kualitas data juga sering muncul, misalnya data salah dari bursa, celah data akibat gangguan jaringan, atau *corporate action* yang mengharuskan penyesuaian ulang historis Chien (February 2022). Di samping itu, regulasi mengharuskan adanya *audit trail* rinci, perlindungan privasi, dan kemampuan menjelaskan *fairness* serta *transparency* model, sehingga tuntutan tata kelola menjadi jauh lebih ketat.

1. Kompleksitas Integrasi *Alternative Data*

Alternative data semakin sering dijadikan sumber *alpha* di pasar yang kompetitif. Analisis sentimen dari artikel berita, unggahan media sosial, *earnings call transcript*, dan sumber tak terstruktur lainnya memberikan sinyal tambahan di luar harga dan volume. Namun, memadukan fitur berbasis *embedding* ke dalam sistem yang sudah beroperasi bukanlah tugas yang sederhana. Karakteristik *embedding* berbeda dari fitur tabular, sehingga memerlukan jalur pemrosesan dan *serving* yang berbeda. Dalam praktik, dibutuhkan *pipeline* khusus untuk ekstraksi, normalisasi, dan penyajian *embedding* agar model dapat memanfaatkannya secara konsisten dan terukur.

Vector similarity search menjadi komponen inti untuk memanfaatkan *alternative data* dalam sistem perdagangan modern. Algoritme HNSW (*Hierar-*

chical Navigable Small World) banyak digunakan untuk *Approximate Nearest Neighbor* (ANN) search karena mampu menyeimbangkan kecepatan dan akurasi. HNSW membentuk graf berlapis dengan koneksi jarak jauh di lapisan atas untuk navigasi cepat dan struktur rapat di lapisan bawah untuk presisi, sehingga pencarian semantik berlatensi rendah dalam skala besar dapat dicapai. Dengan demikian, dukungan terhadap *vector search* menjadi elemen penting dalam arsitektur target yang ingin memanfaatkan *embeddings* secara sistematis.

2. Tantangan Validitas Temporal

Proses *backtesting* di domain ini menjadi lebih rumit karena kuatnya dependensi temporal dan tingginya risiko *data leakage*. Polyzotis dkk. (2018) menegaskan bahwa validitas temporal, validasi data, dan *provenance* merupakan kunci integritas evaluasi, karena kebocoran temporal dapat merusak keandalan hasil uji. Wang dan Ruf (2021) menunjukkan bahwa penggunaan *random split* pada *time-series* finansial dapat menyebabkan *information leakage* dan menyarankan pemisahan data secara kronologis (*pseudo real-time*). Menjaga *point-in-time correctness* sambil tetap melayani *throughput* tinggi menjadi tantangan teknis besar yang belum sepenuhnya terjawab oleh mayoritas sistem yang ada saat ini.

III.1.5 Kesenjangan Kritis yang Teridentifikasi

Berdasarkan analisis sistematis terhadap *current state* implementasi di industri, terdapat beberapa celah kritis yang menjadi fokus utama proposal ini (Polyzotis dkk. 2018; Stoyanovich, Howe, dan Jagadish 2020; Shankar dkk. 2022). Celah-celah ini menggambarkan area yang belum tercakup secara menyeluruh oleh praktik dan platform yang ada saat ini dan menjadi sasaran perbaikan arsitektural. Dengan merumuskan kesenjangan secara eksplisit, rancangan arsitektur dapat diarahkan pada masalah yang paling berdampak terhadap keandalan sistem.

1. Belum tersedia arsitektur terintegrasi yang secara konsisten menghubungkan *data ingestion*, *feature engineering*, pelatihan model, *deployment*, dan *monitoring* ke dalam satu platform yang koheren dengan *workflow* otomatis dari ujung ke ujung.
2. Alat untuk menjamin validitas temporal dan mencegah kebocoran data pada konteks *time-series* masih terbatas, padahal aspek ini sangat menentukan validitas model dan sering diabaikan atau hanya diterapkan secara *ad hoc*.
3. Dukungan terhadap tipe data modern seperti *vector embeddings* pada infras-

struktur *feature serving* masih kurang, sehingga pemanfaatan *alternative data* yang semakin penting menjadi tidak optimal.

4. *Monitoring* proaktif untuk mendeteksi *drift* dan memicu remediasi otomatis belum tersedia secara menyeluruh, sehingga degradasi model kerap terjadi secara senyap dan baru terdeteksi melalui indikator bisnis.
5. Ketiadaan *governance* terpusat dan manajemen metadata yang mendukung *discoverability* serta *compliance* mengakibatkan inefisiensi operasional dan meningkatkan risiko pelanggaran regulasi.

Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mencatat bahwa meskipun telah tersedia banyak alat *open-source* berkualitas untuk tiap aspek DataOps dan MLOps, integrasi mulus dengan pengalaman pengguna terpadu, manajemen metadata konsisten, *workflow* otomatis dari data hingga model, serta *governance* komprehensif masih sulit dicapai. Menutup celah ini memerlukan pendekatan arsitektural yang holistik dengan mengombinasikan komponen *open-source* yang tepat dan menyusunnya menjadi satu platform terintegrasi. Pendekatan arsitektural tersebut menjadi ruang kontribusi utama proposal tugas akhir ini.

III.2 Analisis Kebutuhan

Analisis kebutuhan dilakukan untuk menerjemahkan masalah yang telah dipetakan menjadi spesifikasi yang konkret dan dapat diimplementasikan dalam bentuk sistem. Proses ini mengikuti kerangka *requirements engineering* yang meliputi identifikasi kebutuhan melalui analisis *user journey*, perumusan kebutuhan fungsional, dan penyusunan kebutuhan nonfungsional (Sommerville 2016). Nuseibeh dan Easterbrook (2000) menekankan bahwa kualitas suatu sistem sangat ditentukan oleh pemahaman yang tepat mengenai konteks operasional dan batasan domain. Dalam domain perdagangan finansial, konteks tersebut mencakup kompleksitas teknis, dinamika data, dan regulasi yang ketat.

Pendekatan *goal-oriented requirements engineering* (Lamsweerde 2009) digunakan agar setiap kebutuhan selalu terkait dengan tujuan bisnis dan teknis yang jelas. Dengan demikian, setiap kebutuhan yang dirumuskan memiliki alasan keberadaan yang dapat dijelaskan, bukan sekadar daftar fitur terpisah. Seluruh kebutuhan yang dihasilkan kemudian menjadi landasan bagi desain arsitektur yang akan disusun, sekaligus kriteria untuk mengevaluasi keberhasilan implementasi *proof-of-concept* yang direncanakan setelah proposal ini.

III.2.1 Identifikasi Masalah Pengguna

Untuk memperoleh gambaran kebutuhan yang komprehensif, dilakukan analisis terhadap masalah yang dihadapi berbagai *persona* kunci dalam *workflow* ML. Tiga *persona* utama yang dikaji adalah *Data Engineer*, *Data Scientist*, dan *ML Engineer*. Masing-masing peran memiliki tanggung jawab, alur kerja, dan hambatan yang berbeda, sehingga sistem perlu dirancang agar mampu mengakomodasi kebutuhan semua pihak. Sejalan dengan Amershi dkk. (2019), proposal ini menggunakan pendekatan berbasis *persona* untuk menangkap kebutuhan lintas peran secara tajam dan sistematis.

1. *Data Engineer* – Dian

Dian adalah seorang *Senior Data Engineer* di tim *quantitative trading* yang bertanggung jawab membangun dan memelihara *pipeline* data untuk *ingestion*, pemrosesan, dan *serving* data ke proses pelatihan maupun *inference* model. Ia memiliki latar belakang *distributed systems* dan *data engineering*, serta berpengalaman menggunakan Spark, Kafka, dan berbagai basis data SQL. Tugas hariannya mencakup menjaga kualitas dan *freshness* data, mengoptimalkan kinerja *pipeline*, dan memastikan ketersediaan data bagi tim di hilir. Dengan peran tersebut, efisiensi dan keandalan *pipeline* sangat memengaruhi produktivitas keseluruhan tim.

a) Masalah 1: Kompleksitas Manajemen Sistem

Dian dihadapkan pada kompleksitas tinggi ketika harus mengelola berbagai kerangka pemrosesan data dan sistem penyimpanan secara bersamaan. Setiap tahap dalam *pipeline* mengandalkan alat yang berbeda, misalnya Flink untuk fitur *real-time*, Spark untuk agregasi historis, dan beberapa basis data yang saling terhubung. Upaya menjaga konsistensi dan keseragaman antarsistem menyita banyak waktu dan energi. Polyzotis dkk. (2018) menekankan bahwa tantangan pada *data lifecycle* menjadi salah satu sumber utama kompleksitas dan memerlukan penanganan yang lebih terstruktur. Dalam praktik, Dian sering menghabiskan waktu menelusuri isu kualitas data yang sulit dilacak karena *lineage* dan *distributed tracing* belum lengkap.

b) Masalah 2: Sulitnya Penegakan Validitas Temporal

Karena data finansial memiliki ketergantungan temporal yang kuat, kebocoran informasi masa depan ke data pelatihan dapat terjadi jika tidak dikendalikan dengan hati-hati. Dian harus menulis logika *join* yang kompleks untuk memastikan hanya data yang benar-benar tersedia pada

waktu tertentu yang ikut digunakan. Implementasi manual semacam ini memakan waktu dan rentan terhadap kesalahan halus yang sulit terdeteksi. Polyzotis dkk. (2018) menegaskan pentingnya validitas temporal bagi integritas evaluasi, dan Wang dan Ruf (2021) menunjukkan bahwa *random splitting* menyebabkan evaluasi tidak valid pada data finansial yang bersifat *time-series*.

c) **Masalah 3: Beban Operasional Pemeliharaan Pipeline**

Perubahan skema data dari penyedia pasar secara berkala memicu kegagalan *pipeline* dan menuntut perbaikan segera. Ketika ditemukan isu kualitas data, Dian harus menelusuri rangkaian panjang logika transformasi untuk mengidentifikasi sumber masalah. *Monitoring* yang tersedia cenderung reaktif, sehingga *alert* biasanya muncul setelah terjadi kegagalan. Shankar dkk. (2022) menyoroti bahwa banyak aktivitas pemeliharaan ML di produksi masih dilakukan secara manual atau *ad hoc*, sehingga memperberat beban operasional dan mengurangi ruang untuk perbaikan strategis.

d) **Masalah 4: Friksi Kolaborasi**

Data scientist kerap mengajukan permintaan fitur baru atau transformasi kompleks yang harus direalisasikan oleh Dian di tingkat *pipeline*. Ketiadaan fasilitas *self-service* data membuat *data scientist* sangat bergantung pada tim data engineering untuk mengakses dataset atau fitur tertentu. Ketergantungan ini menciptakan *bottleneck* dan memperlambat iterasi pengembangan model. Berbagai studi mengenai integrasi MLOps–DataOps menyoroti bahwa friksi kolaborasi antara tim data dan tim model merupakan salah satu penghambat utama produktivitas.

2. **Data Scientist – Sarah**

Sarah adalah seorang *Data Scientist* dengan latar belakang *quantitative finance* yang bertugas mengembangkan model prediktif untuk strategi perdagangan. Ia menguasai algoritme ML, pemodelan statistik, dan memahami dinamika pasar, serta sehari-hari bekerja dengan Python dan berbagai *framework* ML. Tanggung jawabnya mencakup *exploratory data analysis*, *feature engineering*, pengembangan model, *hyperparameter tuning*, dan validasi kinerja model. Produktivitas Sarah sangat ditentukan oleh kemudahan mengakses data, mengelola eksperimen, dan berkolaborasi dengan tim lain.

a) **Masalah 1: Sulitnya Penemuan dan Pemanfaatan Fitur**

Tim telah membangun ratusan fitur dari berbagai sumber dan periode

waktu, tetapi belum tersedia katalog terpusat yang menjelaskan makna fitur, rumus perhitungan, dan detail penggunaannya. Sarah sering kali tidak mengetahui bahwa fitur yang ia butuhkan sebenarnya sudah ada, sehingga banyak pekerjaan rekayasa fitur yang terduplikasi. Ketidadaan *feature store* dan katalog fitur terpusat membuat proses *reuse* sulit dilakukan dan mendorong fragmentasi definisi fitur di seluruh organisasi.

b) Masalah 2: Waktu Berlebih pada Persiapan Data

Sarah menghabiskan porsi besar waktunya untuk *data wrangling* seperti menulis *join*, menangani *missing value*, dan memastikan validitas temporal. Tugas-tugas ini berulang di banyak proyek dan menunda aktivitas bernilai tinggi seperti eksplorasi ide model baru atau analisis performa yang mendalam. Polyzotis dkk. (2018) menyoroti bahwa kompleksitas *data lifecycle* sering menjadi penghalang utama dalam pemanfaatan ML, dan situasi Sarah mencerminkan temuan tersebut secara langsung di konteks perdagangan finansial.

c) Masalah 3: Tantangan *Experiment Tracking* dan Reprodusibilitas

Sarah menjalankan banyak eksperimen secara paralel untuk menguji kombinasi fitur dan konfigurasi model. Tanpa *experiment tracking* yang memadai, ia kesulitan menelusuri eksperimen mana yang menghasilkan model terbaik dan alasan di balik pemilihan model tersebut. Pimentel dkk. (2019) menunjukkan rendahnya kemampuan re-eksekusi *notebook* publik akibat kurangnya pelacakan sistematis terhadap *dependency* dan konfigurasi. Kondisi ini berpotensi terulang dalam lingkungan internal jika *experiment tracking* tidak ditangani sebagai bagian penting dari praktik MLOps.

d) Masalah 4: Sulitnya Integrasi *Alternative Data*

Data tak terstruktur seperti sentimen berita dan media sosial memerlukan *pipeline* tersendiri untuk menghasilkan *embedding* sebagai fitur. Infrastruktur *feature serving* yang ada biasanya dirancang untuk data tabular dan belum menyediakan dukungan bawaan untuk *similarity search*. *Vector similarity search* skala besar menuntut indeks seperti PQ dan sering kali membutuhkan akselerasi GPU (Jegou, Douze, dan Schmid 2011; Johnson, Douze, dan Jégou 2021). Keterbatasan ini menyebabkan Sarah kesulitan memasukkan *alternative data* secara konsisten ke dalam model produksi, meskipun secara teoritis sinyal tersebut memiliki potensi besar.

e) **Masalah 5: Visibilitas Kinerja Model Terbatas**

Setelah model di-*deploy*, Sarah sering baru menyadari penurunan kinerja melalui indikator bisnis, bukan dari *dashboard* teknis. Fasilitas *monitoring real-time* yang mampu mendeteksi *drift* secara dini masih belum memadai. Idealnya, sistem *real-time analytics* menggabungkan *stream processing*, *real-time inference*, dan arsitektur *event-driven*. Dalam pasar yang nonstasioner, kebutuhan *monitoring drift* proaktif menjadi krusial karena distribusi data berubah dari waktu ke waktu dan dapat berdampak langsung pada hasil perdagangan.

3. **ML Engineer – Michael**

Michael adalah seorang *ML Engineer* yang bertugas mengoperasionalkan model ke sistem perdagangan produksi. Ia memiliki latar belakang rekayasa perangkat lunak dan *distributed systems*, serta terbiasa bekerja dengan kontainerisasi, orkestrasi, dan CI/CD. Tanggung jawab utamanya adalah membangun *deployment pipeline*, memastikan model memenuhi persyaratan kinerja, serta menerapkan *monitoring* dan *alerting* yang memadai. Dengan posisi tersebut, Michael menjadi penghubung utama antara dunia eksperimen dan sistem produksi yang melayani permintaan nyata.

a) **Masalah 1: Besarnya Upaya *Productionization***

Model yang diserahkan oleh *data scientist* biasanya masih berupa Jupyter *Notebook* atau skrip eksperimen yang belum siap produksi. Michael harus melakukan *refactor* menjadi layanan *backend* dengan kemampuan *error handling*, *logging*, instrumentasi *monitoring*, dan optimasi kinerja yang memadai. Proses ini dapat memakan waktu signifikan untuk setiap model baru, sehingga kapasitas tim tersita untuk pekerjaan berulang. Paley, Urma, dan Lawrence (2022) mencatat bahwa usaha operasionalisasi model di produksi sering jauh lebih besar dibanding upaya pengembangan awal.

b) **Masalah 2: Kompleksitas *Model Deployment Lifecycle Management***

Lingkungan perdagangan menuntut kemampuan menjalankan beberapa versi model secara bersamaan untuk *A/B testing*, *canary deployment*, dan *rollback* cepat jika terjadi masalah. Implementasi manual pola-pola ini di atas infrastruktur generik membutuhkan orkestrasi yang rumit dan rentan kesalahan konfigurasi. Platform MLOps dapat membantu mengotomatisasi sebagian proses, tetapi implementasinya di setiap organisasi masih sangat bervariasi dan tidak selalu terhubung secara rapi dengan

sistem yang sudah ada.

c) **Masalah 3: Tantangan Persyaratan Kinerja Ketat**

Aplikasi perdagangan sering menuntut *prediction latency* pada orde satu digit milidetik di p99, sehingga ruang toleransi untuk overhead sangat sempit. Michael harus melakukan *profiling*, mengoptimalkan serialisasi, mengonfigurasi *thread*, dan menyetel parameter infrastruktur secara detail. Munappy dkk. (2022) menegaskan bahwa sistem *deep learning* produksi memerlukan manajemen *data lifecycle* yang kompleks dan infrastruktur khusus, sehingga upaya untuk mencapai kinerja tinggi menjadi tantangan struktural lintas lapisan.

d) **Masalah 4: Sulitnya Implementasi *Monitoring* Komprehensif**

Sistem ML membutuhkan *monitoring* yang mencakup metrik umum infrastruktur dan metrik khusus ML seperti distribusi prediksi, distribusi fitur, dan indikator *drift*. Tanpa platform *observability* terintegrasi, Michael harus menggabungkan banyak alat berbeda secara manual, yang menambah kompleksitas operasional dan berpotensi menimbulkan kesenjangan observabilitas. *Distributed tracing* berbasis OpenTelemetry banyak dimanfaatkan untuk memberikan visibilitas *end-to-end*, dan Shankar dkk. (2022) menekankan pentingnya *observability* yang melampaui metrik infrastruktur semata.

e) **Masalah 5: Tantangan *Model Governance* dan Kepastian *Compliance***

Firma perdagangan berada di bawah pengawasan regulator yang menuntut *fairness*, *transparency*, dan *explainability* terhadap model yang digunakan. Michael harus memastikan bahwa setiap prediksi memiliki *audit log* yang memadai dan *lineage* model dapat ditelusuri dari data hingga *deployment*. Stoyanovich, Howe, dan Jagadish (2020) menegaskan perlunya *governance* yang komprehensif, namun banyak sistem produksi yang masih belum memenuhi standar tersebut. Keterbatasan dalam tata kelola berpotensi menimbulkan risiko *compliance* yang signifikan bagi organisasi.

III.2.2 Sintesis Kebutuhan dari Analisis *Persona*

Dari analisis terhadap ketiga *persona* di atas, muncul sejumlah tema umum yang dapat digunakan untuk menyarikan kebutuhan sistem secara lintas peran. Tema-tema ini merangkum masalah yang paling sering muncul dan memiliki dampak besar ter-

hadap produktivitas serta keandalan sistem secara keseluruhan. Dengan menyatukan pola-pola ini, perancangan kebutuhan menjadi lebih terarah dan tidak terjebak pada perbaikan yang hanya menguntungkan satu peran saja.

1. **Ketiadaan platform terintegrasi** memaksa pengguna bekerja dengan banyak alat dan layanan yang tidak selalu terhubung secara baik, sehingga menimbulkan friksi dalam *workflow* dan meningkatkan risiko kesalahan integrasi.
2. **Tingkat otomasi yang belum memadai** menyebabkan pengguna menghabiskan waktu signifikan untuk tugas rutin berulang, seperti penyiapan data dan pengelolaan *deployment*, yang seharusnya dapat diotomatisasi.
3. **Keterbatasan *observability*** membuat proses *troubleshooting* cenderung reaktif dan memakan waktu karena akar masalah sulit ditelusuri secara *end-to-end*.
4. **Ketiadaan kapabilitas *governance* yang kuat** meningkatkan risiko *compliance* dan menghambat kolaborasi, karena informasi mengenai fitur, model, dan *pipeline* tidak terdokumentasi dengan rapi dan sulit dicari kembali.

Menjawab tema-tema ini memerlukan solusi yang tidak hanya menambal titik masalah tertentu, tetapi menghadirkan platform dengan kapabilitas terintegrasi yang selaras dengan *workflow* ML di konteks perdagangan finansial. Sasaran tersebut menjadi pijakan utama dalam perancangan sistem yang dikembangkan pada proposal tugas akhir ini.

III.2.3 Kebutuhan Fungsional

Berdasarkan analisis masalah pengguna, kebutuhan fungsional dirumuskan untuk menggambarkan perilaku dan kemampuan spesifik yang harus disediakan oleh sistem. Setiap kebutuhan diberi kode dengan *prefix* KF untuk memudahkan *traceability* selama proses perancangan dan implementasi (Sommerville 2016; Lamsweerde 2009). Dengan penandaan ini, setiap keputusan desain dapat ditelusuri kembali ke kebutuhan yang ingin dipenuhi.

Kebutuhan fungsional ini juga dipetakan secara langsung terhadap *gap* kritis pada *current state*, khususnya terkait integrasi *end-to-end*, validitas temporal, dukungan *embedding*, otomatisasi *deployment*, dan *governance* terpusat. Pemetaan tersebut membantu memastikan bahwa rancangan arsitektur yang disusun tetap fokus pada permasalahan utama yang telah diidentifikasi dan tidak bergeser ke fitur-fitur yang

tidak esensial.

Tabel III.1 Kebutuhan Fungsional Sistem

ID	Kebutuhan Fungsional	Deskripsi	Prioritas
KF-01	Manajemen Fitur Terpusat	Sistem menyediakan <i>feature registry</i> terpusat untuk mendefinisikan, mengelola, dan melakukan <i>versioning</i> fitur. <i>Registry</i> menyimpan metadata seperti deskripsi, tipe data, <i>owner</i> , dan dependensi, serta dikelola di dalam repositori Git agar mendukung <i>workflow</i> GitOps dan penelusuran perubahan.	Tinggi
KF-02	Penyajian Fitur Ganda	Sistem mampu melakukan <i>serving</i> fitur dalam dua mode, yaitu <i>online serving</i> untuk <i>real-time inference</i> dengan target latensi < 10 ms di p99 dan <i>offline serving</i> untuk <i>historical retrieval</i> berkapasitas besar yang mendukung <i>point-in-time correct join</i> pada data historis.	Tinggi
KF-03	Jaminan Validitas Temporal	Sistem secara bawaan menjamin <i>point-in-time correctness</i> ketika menyusun <i>training dataset</i> untuk data <i>time-series</i> , sehingga mencegah kebocoran data temporal yang dapat membuat evaluasi dan model menjadi tidak valid.	Tinggi
KF-04	Dukungan <i>Vector Embeddings</i>	Sistem mendukung penyimpanan, pengindeksan, dan <i>serving vector embeddings</i> pada <i>online store</i> dengan <i>similarity search</i> yang efisien (latensi < 10 ms di p99), misalnya melalui algoritme HNSW atau metode sejenis.	Tinggi
KF-05	Otomatisasi <i>Pipeline</i> Pelatihan	Sistem menyediakan <i>pipeline</i> otomatis untuk mengorkestrasi proses pelatihan model dari ujung ke ujung, terintegrasi dengan <i>feature store</i> , <i>experiment tracking</i> , dan <i>model registry</i> , sehingga mendukung pencapaian otomasi sesuai MLOps Level 2.	Tinggi
KF-06	Otomatisasi Penerapan Model	Sistem mengotomatisasi <i>deployment</i> model yang telah lulus validasi ke <i>inference endpoint</i> produksi, serta mendukung strategi <i>rolling update</i> , <i>canary deployment</i> , <i>A/B testing</i> , dan mekanisme <i>rollback</i> yang aman ketika terjadi degradasi.	Tinggi
KF-07	Pemantauan <i>Drift</i> Otomatis	Sistem secara otomatis dan berkala memantau <i>data drift</i> dan <i>concept drift</i> dengan membandingkan distribusi fitur di produksi terhadap distribusi pada saat pelatihan menggunakan uji statistik (misalnya PSI, K-S) dengan ambang adaptif, serta terintegrasi dengan <i>event-driven monitoring</i> untuk memungkinkan respon otomatis terhadap anomali.	Tinggi

ID	Kebutuhan Fungsional	Deskripsi	Prioritas
KF-08	<i>Governance</i> & Penemuan Otomatis	Sistem menangkap dan menyajikan metadata serta <i>lineage</i> untuk fitur, <i>dataset</i> , model, dan <i>pipeline</i> dalam sebuah katalog yang dapat dicari, sehingga memudahkan <i>discovery</i> , analisis dampak, dan pemenuhan kebutuhan <i>compliance</i> .	Tinggi
KF-09	Pelacakan Eksperimen Terintegrasi	Sistem menyediakan <i>experiment tracking</i> yang terintegrasi untuk mencatat parameter, metrik, artefak, dan versi kode, sehingga proses eksperimen dapat direproduksi dan dibandingkan secara sistematis.	Tinggi
KF-10	<i>Versioning</i> dan Registri Model	Sistem menyediakan <i>model registry</i> terpusat untuk mengelola versi model dan status siklus hidupnya (<i>staging, production, archived</i>) disertai <i>audit trail</i> lengkap untuk mendukung <i>governance</i> .	Tinggi
KF-11	Pemrosesan <i>Batch</i> dan <i>Stream</i>	Sistem mendukung <i>batch processing</i> untuk analisis dan rekayasa fitur historis serta <i>stream processing</i> untuk komputasi fitur <i>real-time</i> , di mana keduanya terhubung dengan <i>feature store</i> sehingga definisi fitur tetap konsisten.	Tinggi
KF-12	Konfigurasi Deklaratif	Sistem memungkinkan konfigurasi komponen seperti fitur, <i>pipeline</i> , dan <i>deployment</i> dalam format deklaratif (misalnya YAML atau Python), yang dikontrol versinya di Git dan selaras dengan prinsip GitOps.	Tinggi
KF-13	API dan SDK Terpadu	Sistem menyediakan API dan SDK terpadu (misalnya untuk Python) sebagai antarmuka standar untuk berinteraksi dengan seluruh komponen, sehingga kompleksitas integrasi menurun dan pola pemakaian menjadi konsisten.	Sedang
KF-14	Manajemen Sumber Daya	Sistem menyediakan kemampuan pengelolaan sumber daya komputasi yang efisien, termasuk <i>resource quota</i> , prioritas, serta <i>dynamic provisioning</i> pada lingkungan <i>multi-tenant</i> untuk mencegah satu <i>workload</i> menghabiskan sumber daya klaster.	Sedang

III.2.4 Kebutuhan Nonfungsional

Kebutuhan nonfungsional menjabarkan atribut mutu dan batasan yang harus dipenuhi agar sistem beroperasi dengan baik di lingkungan produksi. Kebutuhan ini memastikan sistem tidak hanya berfungsi secara teknis, tetapi juga memenuhi tuntutan kinerja, keandalan, keamanan, dan kemudahan penggunaan yang relevan dengan operasi harian. Dengan demikian, sistem yang dibangun tidak berhenti sebagai prototipe, tetapi memiliki potensi untuk dioperasikan secara berkelanjutan.

Setiap kebutuhan nonfungsional diberi kode dengan *prefix* KNF untuk menjaga *traceability* selama siklus pengembangan (Sommerville 2016; Lamsweerde 2009). Tar-

get metrik pada tabel digunakan sebagai tolok ukur dalam mengevaluasi performa *proof-of-concept*. Hasil pengujian terhadap metrik ini berperan sebagai indikator apakah rancangan arsitektur mendekati tujuan yang diharapkan dan kapan perbaikan lanjutan diperlukan.

Tabel III.2 Kebutuhan Nonfungsional Sistem

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-01	Kinerja Latensi Rendah	<i>Online feature serving</i> dan <i>vector similarity search</i> harus memberikan latensi yang stabil dan rendah untuk mendukung pengambilan keputusan perdagangan. Layanan ini perlu mampu merespons permintaan dengan cepat, bahkan pada saat beban tinggi.	$p50 < 5 \text{ ms}$, $p95 < 8 \text{ ms}$, $p99 < 10 \text{ ms}$
KNF-02	<i>Throughput</i> Tinggi	Sistem harus mampu menangani <i>workload</i> dengan <i>throughput</i> tinggi yang lazim pada aplikasi perdagangan, dengan karakteristik skalabilitas mendekati linear ketika sumber daya ditambah.	<i>Feature serving</i> : $> 10,000 \text{ QPS/node}$; <i>stream processing</i> : $> 1 \text{ juta events/detik/pipeline}$
KNF-03	Skalabilitas Horizontal	Seluruh komponen sistem harus mendukung skala horizontal, sehingga kapasitas dapat ditingkatkan dengan menambah <i>node</i> dan menghasilkan peningkatan kinerja yang linear atau mendekati linear.	<i>Scaling</i> linear atau <i>near-linear</i> hingga $10\times$ kapasitas dasar
KNF-04	Keandalan & Toleransi Kesalahan	Sistem harus andal dan tidak memiliki <i>single point of failure</i> , serta mampu melakukan pemulihan otomatis dari kegagalan sementara tanpa intervensi manual berlebihan.	<i>Availability</i> 99,9%; $\text{RTO} < 5 \text{ menit}$; $\text{RPO} < 1 \text{ menit}$
KNF-05	Konsistensi & Kebenaran	Sistem harus menjaga konsistensi data dan menjamin validitas temporal sehingga tidak terjadi kebocoran data yang merusak kebenaran hasil analisis maupun kinerja model.	<i>Zero tolerance</i> untuk kebocoran data temporal yang terdeteksi
KNF-06	<i>Observability</i>	Sistem harus memiliki <i>observability</i> menyeluruh, mencakup metrik, <i>structured log</i> , dan <i>distributed tracing</i> dengan dukungan OpenTelemetry, sehingga perilaku sistem dapat dianalisis end-to-end.	Seluruh komponen terinstrumentasi dengan metrik standar dan <i>distributed tracing</i> berbasis OpenTelemetry

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-07	Keamanan	Sistem harus menerapkan kontrol keamanan (otentikasi, otorisasi, enkripsi, dan <i>audit logging</i>) untuk mencegah akses tidak sah dan menjaga kerahasiaan data sensitif.	TLS 1.3 <i>in transit</i> , AES-256 <i>at rest</i>
KNF-08	Ekstensibilitas & Modularitas	Arsitektur harus modular sehingga komponen dapat diperluas atau diganti melalui antarmuka terdefinisi tanpa mengganggu komponen lain.	Komponen dapat diganti terpisah tanpa perubahan signifikan pada modul lain
KNF-09	Kemudahan Penggunaan	Antarmuka sistem harus intuitif dan didukung dokumentasi baik, sehingga pengguna baru dapat mengadopsi sistem dengan kurva belajar wajar.	Skor kepuasan pengguna > 4/5 pada survei internal
KNF-10	Efisiensi Sumber Daya	Sistem harus memanfaatkan sumber daya komputasi dan penyimpanan efisien untuk mengendalikan biaya sambil mempertahankan kinerja.	<i>Compression ratio</i> > 5:1; utilisasi CPU rata-rata > 70% pada <i>workload</i> target
KNF-11	Portabilitas	Sistem harus dapat dijalankan di berbagai lingkungan <i>deployment</i> dengan perubahan terbatas pada konfigurasi dan tanpa modifikasi kode besar.	Dapat di- <i>deploy</i> di AWS, GCP, Azure, atau <i>on-premise</i> hanya dengan penyesuaian konfigurasi
KNF-12	Kemudahan Pemeliharaan	Sistem harus mudah dipelihara, dengan struktur kode jelas, cakupan pengujian memadai, dan proses <i>deployment</i> otomatis.	<i>Test coverage</i> > 80% pada komponen inti

III.3 Analisis Pemilihan Solusi

Setelah kebutuhan fungsional dan nonfungsional dirumuskan, langkah berikutnya adalah menganalisis alternatif solusi yang dapat digunakan untuk memenuhi kebutuhan tersebut. Tahap ini berfungsi sebagai penghubung antara rumusan kebutuhan dan desain arsitektur konkret yang akan dibangun sehingga keputusan yang diambil memiliki dasar yang terukur dan dapat dipertanggungjawabkan secara teknis maupun konseptual.

Proses analisis mengikuti metodologi *Multi-Criteria Decision Analysis* (MCDA) untuk mengevaluasi alternatif arsitektural berdasarkan kriteria yang telah didefinisikan (Saaty 2008). Triantaphyllou (2000) menegaskan bahwa keputusan arsitektural dengan banyak dimensi memerlukan pendekatan terstruktur yang mampu mengakomodasi berbagai tujuan dan batasan secara simultan. Dengan MCDA, setiap alternatif dinilai melalui kriteria yang sama sehingga hasil perbandingan menjadi

lebih transparan dan mudah dijelaskan.

III.3.1 Alternatif Solusi

Sejumlah alternatif pendekatan arsitektural diidentifikasi untuk membangun platform DataOps dan MLOps yang sesuai dengan kebutuhan sistem. Setiap alternatif mencerminkan filosofi yang berbeda mengenai tingkat integrasi, jenis teknologi, dan *trade-off* antara kontrol dan kemudahan pengelolaan. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) menunjukkan bahwa pemilihan pendekatan arsitektur berdampak langsung pada kompleksitas implementasi dan *maintainability* jangka panjang. Di sisi lain, Amou Najafabadi dkk. (2024) memetakan 35 komponen arsitektural dan varian desain MLOps, sehingga menyediakan referensi untuk membandingkan alternatif secara sistematis.

1. Alternatif 1: Solusi *Fully Managed Cloud Provider*

Pendekatan ini bertumpu pada platform ML terkelola yang disediakan oleh *cloud provider* seperti Amazon SageMaker, Google Vertex AI, atau Microsoft Azure ML. Platform-platform tersebut umumnya menawarkan rangkaian layanan terintegrasi untuk mendukung siklus hidup ML, mulai dari persiapan data, pelatihan, hingga *deployment* dan *monitoring*. Dengan memanfaatkan solusi *fully managed*, organisasi memindahkan sebagian besar tanggung jawab pengelolaan infrastruktur ke penyedia layanan. Hal ini memungkinkan pembangunan *pipeline* ML dasar dengan lebih cepat tanpa banyak pekerjaan infrastruktur internal.

a) Karakteristik Utama

Solusi *fully managed* relatif mudah diadopsi karena integrasi antarkomponen telah disediakan vendor, sehingga kebutuhan integrasi manual berkurang. Beban operasional infrastruktur turun karena tugas seperti *scaling*, *patching*, dan pemeliharaan dasar ditangani oleh penyedia layanan. *Time-to-value* biasanya lebih singkat karena banyak fitur dapat langsung digunakan sebagai *managed service*. Bagi organisasi yang baru mulai membangun kapabilitas ML, pendekatan ini sering menjadi jalur masuk yang praktis.

b) Keterbatasan

Risiko utama pendekatan ini adalah *vendor lock-in*, karena perpindahan ke platform lain akan membutuhkan upaya rekayasa ulang yang besar. *Total cost of ownership* pada skala besar dapat meningkat, sementara ruang kustomisasi dibatasi oleh fitur yang disediakan vendor. Dari per-

spektif proposal tugas akhir, pendekatan ini kurang ideal karena artefak, konfigurasi, dan *workflow* yang dihasilkan sulit direplikasi oleh pihak lain yang tidak menggunakan *cloud provider* yang sama. Keterbatasan tersebut mengurangi nilai hasil proposal sebagai *blueprint* umum yang dapat diadopsi lintas lingkungan.

2. Alternatif 2: Komponen *Open-Source Best-of-Breed*

Pendekatan ini membangun platform dengan mengintegrasikan komponen *open-source* yang dianggap terbaik di kelasnya untuk setiap fungsi utama. Contohnya, Feast untuk *Feature Store*, Kubeflow untuk orkestrasi dan manajemen *pipeline* ML, MLflow untuk *experiment tracking* dan *model registry*, KServe untuk *model serving*, DataHub untuk *governance*, Apache Spark/Flink/Kafka untuk pemrosesan data, ClickHouse sebagai basis data analitik, Redis Stack sebagai *online store*, serta Prometheus/Grafana untuk *monitoring*, dan ArgoCD untuk GitOps. Kombinasi komponen tersebut mampu membentuk platform DataOps dan MLOps yang kaya fitur sekaligus tetap sepenuhnya berbasis *open-source*.

a) Karakteristik Utama

Pendekatan ini memberikan fleksibilitas dan kontrol tinggi terhadap rancangan arsitektur, karena setiap komponen dapat dipilih dan dikonfigurasi sesuai kebutuhan. Komponen dapat diganti ketika tidak lagi sesuai, sehingga risiko *lock-in* berkurang dan transparansi meningkat karena kode sumber terbuka. Inovasi di ekosistem *open-source* berjalan cepat, sehingga kemampuan baru dapat diadopsi tanpa menunggu siklus rilis vendor. Dalam jangka panjang, struktur biaya cenderung lebih terkontrol karena tidak ada biaya lisensi produk tertutup yang besar.

b) Keterbatasan

Tantangan utama pendekatan ini adalah kompleksitas operasional, karena setiap komponen memerlukan pengetahuan khusus untuk instalasi, konfigurasi, dan pemeliharaan. Integrasi antarkomponen agar bekerja sebagai satu platform yang mulus menuntut upaya rekayasa yang signifikan. Namun, dari perspektif proposal tugas akhir, fokus pada pola integrasi, desain *workflow* otomatis, dan praktik operasional justru menjadi kontribusi utama. Dengan demikian, alternatif ini selaras dengan tujuan untuk menyusun *blueprint* arsitektural yang dapat direplikasi.

3. Alternatif 3: Pengembangan Kustom dari Nol

Pada pendekatan ini, komponen kunci dibangun secara kustom dari *low-level*

primitives, misalnya dengan merancang sendiri *feature store*, orkestrator, dan infrastruktur *serving*. Tim pengembang memikul tanggung jawab penuh atas desain dan implementasi setiap modul. Pendekatan ini kadang dipilih oleh organisasi dengan kebutuhan sangat spesifik yang tidak terlayani oleh solusi umum, dan secara teoretis memberikan kebebasan penuh untuk bereksperimen dengan desain sistem.

a) **Karakteristik Utama**

Pendekatan ini memberikan kendali dan kustomisasi maksimum, karena seluruh *stack* dirancang khusus untuk kebutuhan organisasi. Optimasi dapat dilakukan secara presisi pada jalur data atau pola *serving* berlatensi sangat rendah, dan organisasi dapat mengembangkan fitur unik yang belum tersedia di produk lain. Untuk tugas akhir yang berfokus pada algoritma atau mekanisme sistem baru di level rendah, pendekatan ini membuka ruang eksplorasi yang luas.

b) **Keterbatasan**

Biaya pengembangan pendekatan ini sangat tinggi, terutama untuk proyek dengan sumber daya terbatas. Risiko *bug* dan masalah keandalan juga meningkat karena tidak ada basis kode yang telah diuji luas seperti pada proyek *open-source* besar, sehingga *time-to-production* cenderung jauh lebih panjang. Paleyes, Urma, dan Lawrence (2022) menunjukkan bahwa tantangan *deployment* dan operasionalisasi model ML di produksi sudah kompleks, sehingga membebani tim dengan pembangunan seluruh komponen dari awal berpotensi mengalihkan fokus dari kontribusi arsitektural yang ingin dicapai.

4. **Alternatif 4: Pendekatan Hibrida dengan Kustomisasi Selektif**

Pendekatan hibrida menggabungkan layanan terkelola untuk komponen yang dianggap tidak membedakan (*non-differentiating*), misalnya kluster Kubernetes terkelola atau basis data terkelola, sementara komponen yang menjadi fokus riset tetap diimplementasikan dengan solusi *open-source*. Dengan pola ini, sebagian kompleksitas infrastruktur dapat dialihkan ke *cloud provider*, sedangkan area inovasi tetap berada di bawah kendali langsung tim. Pendekatan ini umum digunakan oleh organisasi yang ingin menyeimbangkan kecepatan implementasi dengan kebutuhan kustomisasi.

a) **Karakteristik Utama**

Pendekatan hibrida menawarkan kompromi antara kemudahan operasional dan tingkat kendali. Kompleksitas pengelolaan infrastruktur ber-

kurang karena layanan seperti kontrol plane Kubernetes, *managed database*, atau solusi jaringan dikelola vendor. Pada saat yang sama, tim tetap dapat mengendalikan dan mengembangkan komponen kunci yang menjadi fokus riset, misalnya *Feature Store*, *model serving*, atau sistem *monitoring*, sehingga pengembangan dapat dipercepat tanpa kehilangan fleksibilitas sepenuhnya.

b) Keterbatasan

Ketergantungan terhadap vendor masih muncul pada lapisan infrastruktur, sehingga risiko *lock-in* belum hilang sepenuhnya. Kompleksitas di perbatasan antara layanan *managed* dan *self-managed* dapat meningkat, terutama terkait *networking*, keamanan, dan konsistensi *observability* di seluruh lapisan. Dari sisi riset, nilai tambah pendekatan ini juga dapat berkurang karena konfigurasi sering kali sangat terkait dengan fitur spesifik vendor, sehingga reproduksibilitas oleh pihak lain menjadi lebih sulit. Hal ini perlu dipertimbangkan jika tujuan utama adalah menghasilkan *blueprint* yang dapat diterapkan di berbagai lingkungan.

III.3.2 Evaluasi Kuantitatif Berdasarkan Pemenuhan Kebutuhan

Untuk memberikan dasar yang lebih objektif dalam memilih alternatif, dilakukan evaluasi kuantitatif terhadap kemampuan setiap pendekatan dalam memenuhi kebutuhan fungsional (KF) dan nonfungsional (KNF). Setiap kebutuhan dinilai menggunakan skala tiga nilai, yaitu 2 (terpenuhi penuh), 1 (terpenuhi sebagian), dan 0 (tidak terpenuhi). Penilaian dilakukan secara konsisten terhadap seluruh alternatif sehingga hasilnya dapat disejajarkan dan dibandingkan. Dengan cara ini, argumentasi kualitatif didukung oleh skor numerik yang transparan dan mudah ditelusuri.

Tabel III.3 Evaluasi Pemenuhan Kebutuhan Fungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KF-01: Manajemen Fitur Terpusat	2	2	1	2
KF-02: Penyajian Fitur Ganda	2	2	1	2
KF-03: Jaminan Validitas Temporal	1	2	2	1
KF-04: Dukungan <i>Vector Embeddings</i>	1	2	2	1
KF-05: Otomatisasi <i>Pipeline</i> Pelatihan	2	2	1	2
KF-06: Otomatisasi Penerapan Model	2	2	1	2
KF-07: Pemantauan <i>Drift</i> Otomatis	1	2	1	1
KF-08: <i>Governance</i> & Penemuan	2	2	1	2
KF-09: Pelacakan Eksperimen	2	2	1	2
KF-10: <i>Versioning</i> Registri Model	2	2	1	2
KF-11: Pemrosesan <i>Batch/Stream</i>	2	2	1	2
KF-12: Konfigurasi Deklaratif	2	2	1	2
KF-13: API dan SDK Terpadu	2	1	1	1
KF-14: Manajemen Sumber Daya	2	2	1	2
Total Skor KF	25	27	16	24

Tabel III.4 Evaluasi Pemenuhan Kebutuhan Nonfungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KNF-01: Kinerja Latensi Rendah	2	2	2	2
KNF-02: <i>Throughput</i> Tinggi	2	2	2	2
KNF-03: Skalabilitas Horizontal	2	2	1	2
KNF-04: Keandalan & Toleransi Kesalahan	2	2	1	2
KNF-05: Konsistensi & Kebenaran	1	2	2	1
KNF-06: <i>Observability</i>	2	2	1	2
KNF-07: Keamanan	2	2	1	2
KNF-08: Ekstensibilitas & Modularitas	0	2	2	1
KNF-09: Kemudahan Penggunaan	2	1	0	2
KNF-10: Efisiensi Sumber Daya	1	2	1	1
KNF-11: Portabilitas	0	2	1	1
KNF-12: Kemudahan Pemeliharaan	2	1	0	1
Total Skor KNF	18	22	14	19
TOTAL SKOR (KF+KNF)	43	49	30	43

III.3.3 Analisis Penentuan Solusi

Berdasarkan evaluasi kuantitatif terhadap pemenuhan kebutuhan fungsional dan non-fungsional, pendekatan komponen *open-source best-of-breed* (Alternatif 2) memperoleh skor tertinggi dengan total 49 poin. Nilai ini melampaui pendekatan *cloud managed* dan hibrida yang masing-masing meraih 43 poin, serta pendekatan kustom yang hanya mencapai 30 poin. Perbedaan skor tersebut mengindikasikan bah-

wa pendekatan *open-source* memberikan keseimbangan terbaik dalam menjawab kebutuhan yang telah dirumuskan, sehingga layak dipilih sebagai dasar arsitektur.

Pada kebutuhan yang menjadi fokus utama proposal, pendekatan *open-source* meraih skor penuh (2) untuk jaminan validitas temporal (KF-03), dukungan *vector embeddings* (KF-04), pemantauan *drift* otomatis (KF-07), serta ekstensibilitas dan modularitas (KNF-08). Kebutuhan ini merupakan aspek pembeda yang ingin ditekankan dalam rancangan arsitektur, terutama terkait penanganan data *time-series* finansial, integrasi *alternative data*, dan kemampuan beradaptasi terhadap perubahan distribusi data. Keunggulan di area tersebut memperkuat alasan pemilihan Alternatif 2, karena arsitektur yang dibangun lebih mudah dieksplorasi dan dikembangkan untuk menambah fitur lanjutan.

Dari perspektif portabilitas dan reproduksibilitas, pendekatan *open-source* juga memperoleh skor penuh pada KNF-11, sementara pendekatan berbasis *cloud* dinilai 0. Hal ini menunjukkan bahwa artefak riset seperti definisi *pipeline*, konfigurasi *Feature Store*, dan skema *deployment* dapat dijalankan di berbagai lingkungan dengan perubahan minimal. Bagi akademik dan praktik industri, kemampuan ini penting karena memungkinkan hasil desain dicoba ulang di organisasi lain tanpa bergantung pada vendor tertentu, sehingga memperluas potensi adopsi dan kontribusi komunitas.

Fleksibilitas kustomisasi tercermin dari skor tinggi pada konsistensi dan kebenaran (KNF-05) serta ekstensibilitas (KNF-08), yang membuka ruang untuk mengimplementasikan mekanisme *point-in-time correctness*, dukungan *vector embedding* dengan *similarity search*, dan *automated drift detection* dengan ambang adaptif. Pendekatan *open-source* memungkinkan perancang sistem mengendalikan desain skema data, mekanisme *indexing*, dan pola *serving* hingga ke level detail, sehingga kebutuhan khusus domain perdagangan finansial dapat difasilitasi secara langsung. Sebaliknya, pendekatan *cloud* umumnya hanya menyediakan kemampuan ini dalam bentuk layanan tertutup yang sulit dikustomisasi di luar batas yang sudah ditentukan.

Dari sisi biaya dan keberlanjutan, berbagai praktik industri menunjukkan bahwa pertimbangan biaya jangka panjang dan fleksibilitas mendorong organisasi untuk mengadopsi solusi *open-source*, khususnya ketika skala sistem terus bertumbuh. Meskipun perbandingan biaya selalu bergantung konteks, pendekatan *open-source* memberikan ruang lebih luas untuk mengoptimalkan pemanfaatan infrastruktur dan menghindari biaya lisensi tinggi. Kreuzberger, Köhl, dan Hirschl (2023) menegaskan bahwa arsitektur MLOps berbasis kombinasi *open-source* dan *enterprise* dengan

Kubernetes sebagai fondasi telah menjadi pola adopsi yang umum, sehingga memberikan validasi empiris terhadap arah yang dipilih. Kehadiran berbagai proyek pendukung seperti *API gateway*, *secrets management*, *observability* Kubernetes-native, serta ekosistem Raystack juga menunjukkan bahwa *stack open-source* dapat diperluas untuk mencakup kebutuhan operasional yang biasanya ditangani produk komersial.

III.3.4 Mitigasi Keterbatasan

Walaupun secara keseluruhan paling unggul, pendekatan *open-source* menunjukkan skor relatif lebih rendah pada kemudahan penggunaan (KNF-09) dan kemudahan pemeliharaan (KNF-12). Hal ini mencerminkan bahwa solusi *open-source* sering menuntut usaha konfigurasi dan perawatan yang lebih besar dibanding layanan *managed*. Agar kelemahan ini tidak menghambat tujuan proposal, dirancang beberapa strategi mitigasi yang berfokus pada pembatasan ruang lingkup implementasi operasional, penyederhanaan lingkungan, dan penguatan dokumentasi.

1. Pembatasan Cakupan Implementasi

Proposal ini memposisikan implementasi sebagai *proof-of-concept* yang menitikberatkan pada demonstrasi pola arsitektural dan *workflow* utama, bukan sebagai *production deployment* berskala besar. Dengan membatasi cakupan pada skenario yang terkontrol, kompleksitas operasional dapat dikurangi tanpa mengurangi nilai ilmiah dari konsep yang diuji. Pendekatan ini memungkinkan fokus dialihkan pada aspek arsitektural yang paling relevan, sembari tetap membuka ruang pengembangan lanjutan untuk skala produksi.

2. Pemanfaatan Alat *Development* Lokal

Pengembangan dan pengujian dilakukan terutama di lingkungan lokal seperti Minikube atau *kind*, yang menyediakan kluster Kubernetes dengan kompleksitas lebih rendah. Strategi ini mengurangi kebutuhan berinteraksi langsung dengan infrastruktur *cloud* yang lebih rumit pada tahap awal dan memungkinkan iterasi yang cepat. Setelah pola arsitektural stabil, konfigurasi dapat dipindahkan ke lingkungan *cloud* dengan penyesuaian yang terbatas.

3. Adopsi *Best Practices* Komunitas

Proyek *open-source* yang dipilih umumnya memiliki dokumentasi dan *best practices* yang cukup matang. Proposal ini memanfaatkan panduan tersebut untuk merancang *deployment*, konfigurasi, dan integrasi antarkomponen, sehingga risiko kesalahan desain berkurang. Keselarasan dengan praktik yang digunakan di industri juga diperkuat, sehingga hasil rancangan menjadi lebih

relevan dan mudah diadopsi.

4. Dokumentasi Komprehensif

Sebagai bagian dari kontribusi, disusun dokumentasi teknis dan *runbook* operasional yang menjelaskan cara menginstal, mengonfigurasi, dan mengoperasikan platform. Dokumentasi ini mencakup pengaturan konfigurasi utama, alur *deployment*, dan panduan *troubleshooting* untuk skenario yang umum terjadi. Dengan adanya dokumentasi tersebut, hambatan adopsi bagi tim lain yang ingin menerapkan arsitektur serupa dapat dikurangi dan hasil rancangan dapat direplikasi dengan lebih konsisten.

III.3.5 Evaluasi dan Pemilihan Komponen Spesifik

Setelah pendekatan *open-source best-of-breed* dipilih sebagai dasar arsitektur, langkah selanjutnya adalah mengevaluasi alternatif komponen untuk setiap fungsi utama sistem. Evaluasi dilakukan dengan mempertimbangkan beberapa dimensi kritis, yaitu tingkat maturitas, ekstensibilitas, kekuatan komunitas, dan kemampuan integrasi (terutama dengan Kubernetes dan komponen lain di dalam platform). Setiap kandidat komponen diberi skor dengan skala 1–5 (5 = terbaik) pada tiap dimensi, kemudian nilai tersebut dijumlahkan untuk memperoleh skor total. Skor ini berfungsi sebagai indikator kuantitatif yang melengkapi pertimbangan kualitatif (Triantaphyllou 2000; Keeney dan Raiffa 1993).

Tabel III.5 Evaluasi Alternatif *Feature Store Open-Source*

<i>Feature Store</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi K8s	Total
Feast	5	5	5	5	20
Hopsworks <i>Feature Store</i>	4	4	3	4	15
Feathr (LinkedIn)	3	3	2	3	11
Tecton (basis <i>open-source</i>)	4	3	2	3	12

Tabel III.6 Evaluasi Alternatif Orkestrasi ML *Open-Source*

Orkestrasi ML	Maturitas	K8s-Native	Pipeline ML	Komunitas	Total
Kubeflow Pipelines	4	5	5	4	18
Apache Airflow	5	3	3	5	16
Argo Workflows	4	5	3	4	16
Prefect	3	2	3	3	11

Tabel III.7 Evaluasi Alternatif *Experiment Tracking* dan *Model Registry*

<i>Tracking & Registry</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi	Total
MLflow	5	5	5	5	20
DVC (<i>Data Version Control</i>)	4	4	4	3	15
TensorBoard	4	2	4	2	12
Neptune	3	3	3	3	12

Tabel III.8 Evaluasi Alternatif *Model Serving Open-Source*

<i>Model Serving</i>	Maturitas	Skalabilitas	Framework	Komunitas	Total
KServe	4	5	5	4	18
Seldon Core	4	4	4	4	16
BentoML	3	3	4	3	13
TorchServe	3	3	2	3	11

Tabel III.9 Evaluasi Alternatif Manajemen Metadata dan *Governance*

<i>Metadata & Governance</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi	Total
DataHub (LinkedIn)	5	5	5	5	20
Apache Atlas	4	4	3	4	15
OpenMetadata	3	4	4	4	15
Amundsen (Lyft)	3	3	3	3	12

Tabel III.10 Evaluasi Alternatif *Monitoring* dan *Observability*

<i>Monitoring & Observability</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi K8s	Total
Prometheus + Grafana	5	5	5	5	20
InfluxDB + Chronograf	4	4	3	4	15
Elastic Stack (ELK)	5	4	4	4	17
Jaeger + OpenTelemetry	4	4	4	5	17

Tabel III.11 Evaluasi Alternatif Penyimpanan *Offline* (OLAP)

<i>Penyimpanan Offline</i>	Maturitas	Performa Query	Dukungan Ingestion	Komunitas	Total
ClickHouse	5	5	4	4	18
Apache Pinot	4	4	5	3	16
Apache Druid	4	4	5	4	17
Apache Kudu	3	3	4	3	13

Tabel III.12 Evaluasi Alternatif Penyimpanan *Online* (Latensi Rendah)

Penyimpanan <i>Online</i>	Maturitas	Latensi	Throughput	Komunitas	Total
Redis Stack	5	5	5	5	20
Apache Cassandra	5	4	5	5	19
Apache HBase	4	4	4	4	16
DynamoDB (<i>cloud-native</i>)	4	4	5	3	16

Tabel III.13 Evaluasi Alternatif *Framework* Pemrosesan Data

<i>Framework</i> Pemrosesan	Maturitas	Kapabilitas <i>Batch</i>	Kapabilitas <i>Streaming</i>	Komunitas	Total
Apache Spark	5	5	4	5	19
Apache Flink	4	4	5	4	17
Apache Kafka + Streams	5	2	5	4	16
Apache Beam	3	4	4	3	14

Berdasarkan evaluasi kuantitatif tersebut, komponen dengan skor tertinggi dipilih sebagai kandidat utama untuk diimplementasikan, kemudian keputusan tersebut diperkuat dengan analisis kualitatif. Pemilihan ini tidak hanya mencerminkan besaran skor, tetapi juga mempertimbangkan kesesuaian dengan kebutuhan domain dan *use case* yang dikaji. Dengan demikian, setiap komponen yang digunakan memiliki peran jelas dalam pemenuhan kebutuhan fungsional dan nonfungsional yang telah dirumuskan sebelumnya.

Perlu dicatat bahwa evaluasi kuantitatif pada Tabel III.5–III.13 difokuskan pada komponen inti yang secara langsung merealisasikan kebutuhan fungsional dan non-fungsional (KF dan KNF), seperti *Feature Store*, orkestrasi ML, *model serving*, penyimpanan data, dan *monitoring* dasar. Komponen pendukung lintas fungsi, misalnya *API gateway*, *secrets management*, *alerting*, serta manajemen identitas dan akses tidak diberi skor dengan skema yang sama. Pemilihan komponen pendukung tersebut dilakukan secara kualitatif, berdasarkan kesesuaian dengan kebutuhan keamanan, *observability*, dan operasional, dan akan dijabarkan lebih lanjut pada bab desain arsitektur.

Secara ringkas, **Feast** dipilih sebagai *Feature Store* utama karena mendukung pemisahan *offline–online store* dan menyediakan mekanisme *point-in-time correctness* yang kuat (KF-02, KF-03). **Kubeflow Pipelines** dipilih sebagai orkestrator ML karena bersifat *Kubernetes-native* dan mendukung pengelolaan *workflow* ML secara

deklaratif. **MLflow** digunakan untuk *experiment tracking* dan *model registry* sehingga kebutuhan KF-09 dan KF-10 dapat dipenuhi secara terpadu. Untuk *model serving*, **KServe** dipilih karena menyediakan pola *deployment* lanjutan dan integrasi dengan Knative yang mendukung efisiensi sumber daya.

Pada sisi manajemen metadata dan *governance*, **DataHub** digunakan sebagai katalog pusat dengan dukungan *lineage* dan skema metadata yang ekstensibel. Pengumpulan metadata dari berbagai sistem dapat dibantu oleh alat ekstraksi metadata terotomatisasi, sedangkan antarmuka katalog yang lebih ringan dapat disajikan bagi pengguna akhir. Untuk pilar *observability*, kombinasi **Prometheus** dan **Grafana** digunakan untuk metrik, **Grafana Loki** untuk log, dan **Jaeger** untuk *distributed tracing*, sementara kerangka *alerting* dimanfaatkan untuk mengorkestrasi pengiriman notifikasi berdasarkan metrik yang dipantau.

Untuk penyimpanan dan pemrosesan data, **ClickHouse** dipilih sebagai *offline store* karena mendukung kueri analitik berperforma tinggi dan operasi yang relevan untuk menjaga *point-in-time correctness*. **Redis Stack** digunakan sebagai *online store* berlatensi rendah yang juga mendukung *vector similarity search* bagi *embedding*. **Apache Spark** digunakan sebagai mesin pemrosesan *batch* utama, sedangkan **Apache Flink** digunakan untuk *stream processing* yang membutuhkan latensi rendah dan pengelolaan state yang kompleks. **Apache Kafka** diposisikan sebagai *event backbone* yang menghubungkan berbagai sumber data dan komponen pemrosesan.

Di lapisan keamanan dan manajemen akses, komponen seperti **Frontier** dan **Guardian** dimanfaatkan untuk mengelola identitas dan kebijakan akses data, sedangkan **HashiCorp Vault** menangani rahasia operasional dan kunci enkripsi (HashiCorp 2025; Raystack 2025d, 2025e). **Kong Gateway** berperan sebagai API gateway yang menegakkan autentikasi dan kontrol trafik di tepi kluster (Kong Inc. 2025). Untuk memastikan konsistensi konfigurasi dan otomatisasi *deployment*, pola GitOps dengan **ArgoCD** serta *pipeline* CI/CD Kubernetes-native dengan **Tekton** digunakan untuk membangun, menguji, dan merilis artefak aplikasi serta komponen ML (Tekton Contributors 2024). Ekosistem Raystack menyediakan komponen tambahan seperti Raccoon, Firehose, Dagger, Stencil, Optimus, Meteor, Compass, dan Siren yang mendukung pengumpulan dan pemrosesan *event* data, katalog metadata, serta *alerting* terpusat (Raystack 2025h, 2025c, 2025b, 2025j, 2025g, 2025f, 2025a, 2025i).

Secara keseluruhan, komponen-komponen Raystack dan platform pendukung lain (Vault, Kong, Loki, Jaeger, Tekton) tidak dimaksudkan untuk menggantikan kom-

ponen inti seperti *Feature Store*, orkestrator ML, atau *model serving*, melainkan berperan sebagai lapisan platform pendukung yang memperkuat pemenuhan kebutuhan nonfungsional (KNF-06, KNF-07, KNF-12) dan meningkatkan *operability* serta *governance* platform secara *end-to-end*. Dengan mengadopsi pendekatan *open-source best-of-breed* yang diperkaya dengan komponen tersebut, proposal tugas akhir ini dapat berfokus pada tantangan arsitektural yang paling relevan, yaitu perancangan integrasi antarkomponen, implementasi *proof-of-concept* yang menunjukkan *workflow end-to-end*, serta evaluasi solusi terhadap kebutuhan yang telah dirumuskan. Bab berikutnya akan menguraikan desain arsitektur yang menerjemahkan hasil analisis kebutuhan dan pemilihan solusi ini ke dalam rancangan sistem yang siap diimplementasikan dan dapat berfungsi sebagai *blueprint* bagi platform DataOps dan MLOps modern yang dioptimalkan untuk aplikasi perdagangan finansial.

BAB IV

DESAIN KONSEP SOLUSI

IV.1 Perbandingan Arsitektur Sistem

Bab ini menjelaskan perbandingan terstruktur antara arsitektur yang umum digunakan saat ini dengan arsitektur terintegrasi yang diusulkan. Fokus perbandingan bukan pada penggantian teknologi satu per satu, tetapi pada **pergeseran paradigma dari kumpulan komponen yang terpisah menuju platform DataOps dan MLOps yang terintegrasi**. Pergeseran ini diarahkan untuk menjawab tantangan operasionalisasi ML pada aplikasi perdagangan saham dan *cryptocurrency* yang menuntut kecepatan dan keandalan tinggi. Jain dan Das (2025) menegaskan bahwa integrasi DataOps–MLOps menjadi prasyarat *pipeline* yang skalabel dan andal, sehingga prinsip tersebut dijadikan rujukan utama dalam perancangan pola integrasi pada proposal tugas akhir ini. Perbandingan dilakukan dengan memanfaatkan arsitektur dari literatur akademik dan praktik industri sebagai *baseline* sehingga titik friksi dan perbedaan struktural, fungsional, maupun operasional antara kedua pendekatan dapat ditunjukkan secara eksplisit.

Keterkaitan antara Bab III dan bab ini dijaga dengan cara memetakan langsung kesenjangan yang telah diidentifikasi sebelumnya terhadap perubahan arsitektur yang diusulkan. Dengan demikian, setiap elemen desain yang diperkenalkan di sini memiliki alasan keberadaan yang jelas dan berakar pada analisis masalah yang telah dilakukan. Bab ini sekaligus menjadi jembatan menuju Bab desain implementasi berikutnya, karena perbedaan antara arsitektur eksisting dan arsitektur usulan akan diterjemahkan menjadi alur kerja dan konfigurasi konkret. Pendekatan ini diharapkan menghasilkan rancangan yang konsisten dari awal hingga akhir proposal tugas akhir serta meminimalkan elemen yang tidak relevan atau sekadar konseptual.

IV.1.1 Arsitektur Sistem Saat Ini: Paradigma Terfragmentasi

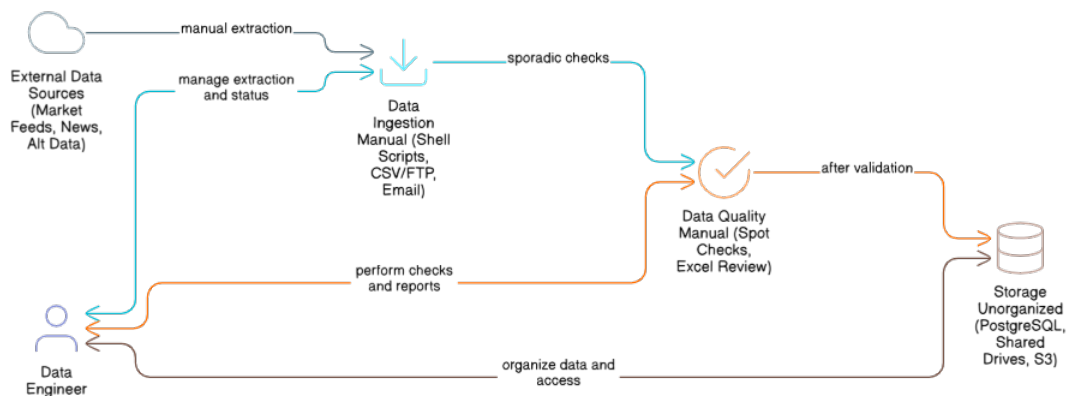
Arsitektur DataOps dan MLOps yang banyak ditemui di industri, sebagaimana dirangkum Kreuzberger, Kühl, dan Hirschl (2023) dan diperkuat studi empiris Munappy dkk. (2022), umumnya terbentuk dari kumpulan *tools* yang tumbuh secara terpisah lalu disambungkan secara *ad hoc*. Sculley dkk. (2015) menunjukkan bahwa pola seperti ini menimbulkan *hidden technical debt* besar karena komponen pendukung seperti *data collection*, *feature engineering*, *monitoring*, dan *serving* tidak per-

nah dirancang sebagai satu kesatuan sistemik. Amou Najafabadi dkk. (2024) juga mencatat bahwa organisasi sering mengombinasikan banyak alat MLOps tanpa pola integrasi yang konsisten, sehingga interaksi antarkomponen lebih bersifat oportunistik daripada direncanakan. Di sisi lain, Paleyes, Urma, dan Lawrence (2022) mendokumentasikan *deployment bottleneck*, kendala CI/CD, masalah reproduksibilitas, dan keterbatasan *monitoring* pascadeploy yang berulang di berbagai organisasi.

Friksi yang muncul pada arsitektur terfragmentasi diringkas pada Gambar IV.1. Alur kerja cenderung linear dan bergantung pada proses manual, misalnya tahap *Data Ingestion* (Manual) yang mengandalkan skrip atau FTP, *Data Quality* (Manual) yang dilakukan sebatas *spot check*, dan *Storage* (Unorganized) yang tersebar tanpa struktur yang jelas. Setelah data tersimpan, alur berpindah ke tahap berikutnya tanpa kontrol terpusat dan tanpa *feedback loop* yang tegas. Setiap perpindahan antarsilo menambah peluang kesalahan dan memperkuat isolasi informasi di masing-masing tahap, sehingga perubahan di satu titik mudah merambat ke titik lain tanpa kendali memadai dan sulit ditelusuri kembali.

Arsitektur pada Gambar IV.1 mewakili pola Level 0/1 MLOps sebagaimana dibahas Kreuzberger, Kühl, dan Hirschl (2023). Untuk menautkan gejala arsitektural dengan pengalaman peran di lapangan, pola umum tersebut diurai menjadi alur berbasis *persona* pada Gambar IV.2 hingga Gambar IV.5. Penguraian ini memperlihatkan bahwa fragmentasi yang sama muncul dengan bentuk berbeda di tiap peran, tetapi menghasilkan konsekuensi operasional yang saling berkaitan. Dengan pendekatan ini, hubungan antara masalah di tingkat sistem dan hambatan sehari-hari *Data Engineer*, *Data Scientist*, *ML Engineer*, dan *Business Users* dapat diamati secara lebih konkret, dan hal tersebut menjadi dasar perancangan arsitektur pada subseksi berikutnya.

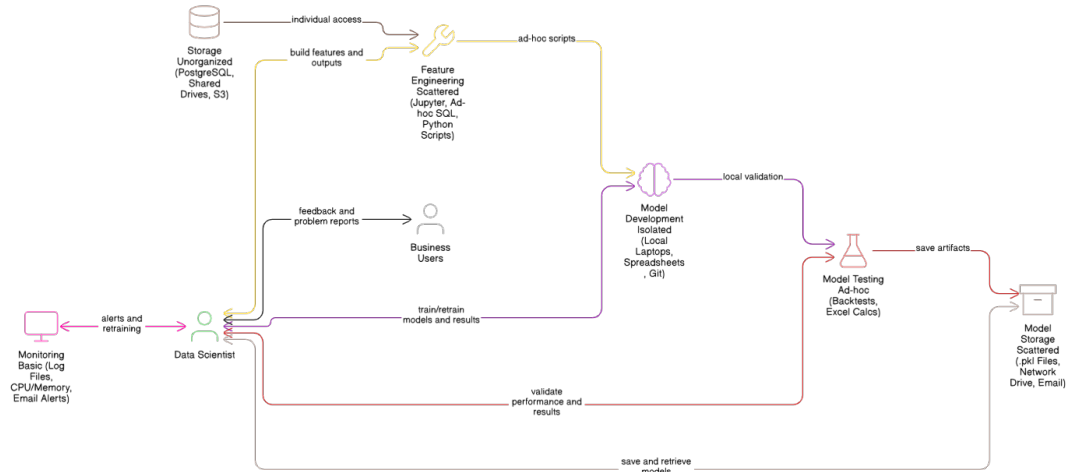
Gambar IV.2 memperlihatkan silo di bagian hulu. *Data Engineer* (DE) bertanggung jawab atas *data ingestion* dan pemeriksaan kualitas secara manual sebelum menempatkan data pada *Storage* (Unorganized). Setelah data tersimpan, DE memiliki visibilitas terbatas terhadap bagaimana data digunakan, model apa yang mengonsumsi data tersebut, dan siapa saja pemakai hilirnya. Ketidakjelasan ini menyulitkan penerapan tata kelola menyeluruh dari hulu ke hilir dan membuat upaya *data stewardship* berhenti pada aspek teknis semata. Dalam jangka panjang, kondisi tersebut mendorong munculnya *feature engineering* berulang dan tidak terkoordinasi pada lapisan berikutnya.



Gambar IV.2 Arsitektur Terfragmentasi Saat Ini (Data Engineer)

Gambar IV.3 menggambarkan silo pada tahap pengembangan model. *Data Scientist* (DS) mengambil data dari *storage* yang tersebar, kemudian membangun fitur dan model di lingkungan terpisah seperti *local laptop* atau *notebook* yang berdiri sendiri. Lingkungan ini biasanya berada di luar *pipeline* produksi, sehingga pelacakan eksperimen dan pengelolaan artefak sulit diintegrasikan dengan sistem operasional. Akibatnya, hasil eksperimen yang baik sering hanya terdokumentasi pada level in-

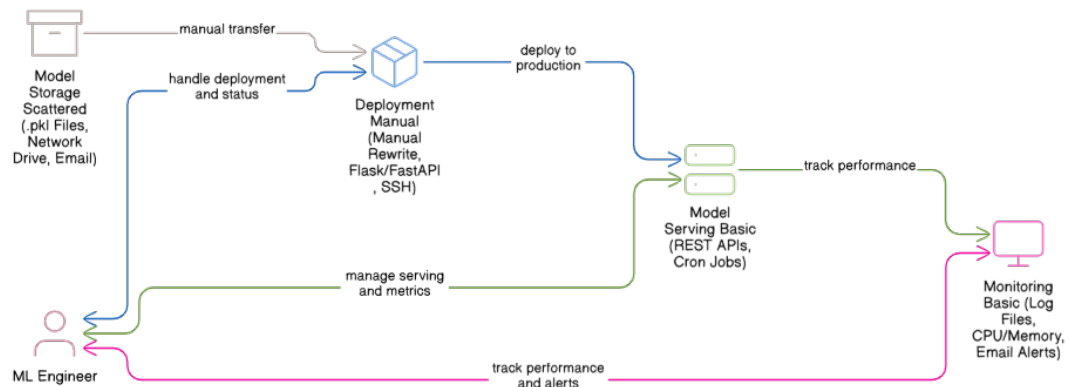
dividu, dan transisi dari eksperimen ke produksi kembali memerlukan kerja ulang dari nol. Pola ini membuat siklus iterasi menjadi lambat dan meningkatkan risiko hilangnya konteks teknis maupun bisnis yang melekat pada suatu model.



Gambar IV.3 Arsitektur Terfragmentasi Saat Ini (Data Scientist)

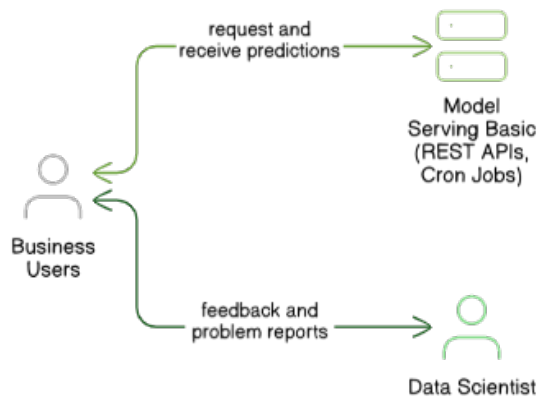
Dua konsekuensi utama dari alur tersebut adalah *training-serving skew* dan *monitoring* yang reaktif. *Training-serving skew* muncul karena logika fitur yang dibangun DS pada tahap pelatihan tidak identik dengan logika *inference* yang kemudian diimplementasikan *ML Engineer* (MLE), sehingga perilaku model di pelatihan dan produksi berbeda (Polyzotis dkk. 2018). *Monitoring* menjadi reaktif karena DS biasanya baru mengetahui kondisi model di produksi dari *problem reports* yang disampaikan *Business Users* (BU), bukan dari *dashboard* teknis. Pola ini menyebabkan degradasi model baru terdeteksi setelah dampaknya terhadap bisnis cukup besar, dan siklus perbaikan menjadi panjang karena penelusuran kembali ke data dan konfigurasi awal tidak didukung oleh sistem terintegrasi.

Gambar IV.4 memperlihatkan silo di sisi MLE. MLE mengambil artefak model dari *Model Storage* yang lokasinya tersebar, lalu melakukan *manual rewrite* dari kode eksperimen menjadi layanan produksi berbasis Flask/FastAPI atau kerangka serupa. Proses ini biasanya melibatkan *refactoring* besar dan penambahan lapisan *logging*, *monitoring*, dan keamanan. Pekerjaan tersebut menyita banyak waktu dan berisiko tinggi menimbulkan *bug*, sehingga *lead time* dari eksperimen ke produksi menjadi panjang. Dalam jangka panjang, pola ini justru menambah *technical debt* baru yang harus dikelola (Paleyes, Urma, dan Lawrence 2022) dan menghambat kemampuan organisasi untuk bereksperimen dengan model baru secara cepat.



Gambar IV.4 Arsitektur Terfragmentasi Saat Ini (ML Engineer)

Gambar IV.5 menempatkan *Business Users* (BU) sebagai pihak di tepi sistem yang perannya cenderung pasif. BU hanya dapat mengajukan permintaan prediksi, menerima hasil, lalu melaporkan masalah ketika keluaran dianggap tidak memuaskan atau berdampak negatif. BU umumnya tidak memiliki akses *self-service* ke data atau model serta tidak memiliki visibilitas terhadap status sistem. Situasi ini memperkuat paradigma reaktif dan menyulitkan organisasi membangun budaya pengambilan keputusan berbasis data yang konsisten. *Feedback loop* dari bisnis ke tim teknis akhirnya berjalan lambat dan tidak terstruktur.



Gambar IV.5 Arsitektur Terfragmentasi Saat Ini (Business Users)

Karakteristik masalah yang berulang pada pola terfragmentasi dapat dirangkum sebagai berikut:

1. *Heterogeneous Compute Infrastructure*

Infrastruktur masih banyak bergantung pada *provisioning* manual, waktu tunggu *scaling* yang panjang, dan tingkat standardisasi konfigurasi yang rendah. Perbedaan konfigurasi antartim dan antarlingkungan membuat portabilitas dan

reproduksibilitas sulit dicapai. Arsitektur usulan merespons kondisi ini dengan pendekatan *vertical scaling* untuk *batch*, *horizontal scaling* untuk *stream*, dan kombinasi keduanya untuk *real-time inference*, lalu mengevaluasi pendekatan tersebut menggunakan kerangka skalabilitas Jogalekar dan Woodside (2000).

2. Fragmentasi *Feature Engineering*

Logika fitur tersebar di berbagai *notebook* dan skrip, sehingga duplikasi dan inkonsistensi definisi fitur menjadi tinggi. Ketiadaan repositori terpusat yang mencatat definisi, dependensi, dan pemilik fitur membuat *troubleshooting* dan *impact analysis* sulit dilakukan. Fragmentasi ini juga menghambat *reuse* fitur antartim karena tidak ada titik referensi bersama yang disepakati.

3. Lingkungan Pengembangan Terisolasi

DS mengembangkan model di lingkungan lokal yang tidak terhubung dengan sistem produksi, sehingga praktik *experiment tracking* dan pengelolaan artefak sering bersifat *ad hoc*. Pengetahuan yang dihasilkan sulit ditransfer ke tim lain dan sulit diulang kembali. Perbedaan lingkungan yang lebar memaksa tim melakukan *refactoring* besar saat memindahkan model ke produksi.

4. Proses *Deployment Manual*

Deployment model masih mengandalkan *manual rewrite* dari kode eksperimen ke layanan produksi, sehingga risiko *bug* meningkat dan praktik *continuous delivery* sulit diterapkan. *Lead time* dari perubahan kecil ke *deployment* produksi menjadi tidak terprediksi (Paleyes, Urma, dan Lawrence 2022). Kondisi ini bertentangan dengan praktik modern pengiriman perangkat lunak yang mendorong rilis cepat dan sering.

5. Paradigma *Monitoring Reaktif*

Monitoring masih berfokus pada metrik infrastruktur, sementara metrik ML spesifik jarang diinstrumentasi. *Drift* jarang dipantau secara sistematis meskipun Jourand dkk. (2023) menunjukkan bahwa *retraining* yang dipicu deteksi *drift* dapat meningkatkan akurasi dari 0,5400 menjadi 0,7010. Tanpa *monitoring* proaktif, sistem cenderung merespons masalah setelah dampak bisnis terjadi.

6. Kesenjangan *Governance*

Lineage dan *audit trail* jarang dilacak otomatis, sehingga pemenuhan *compliance* menjadi sulit dan beban administratif saat audit meningkat. Dokumentasi aset data dan model tersebar serta tidak selalu sinkron dengan keadaan produksi. Dalam lingkungan yang diawasi regulator, kondisi ini meningkatk-

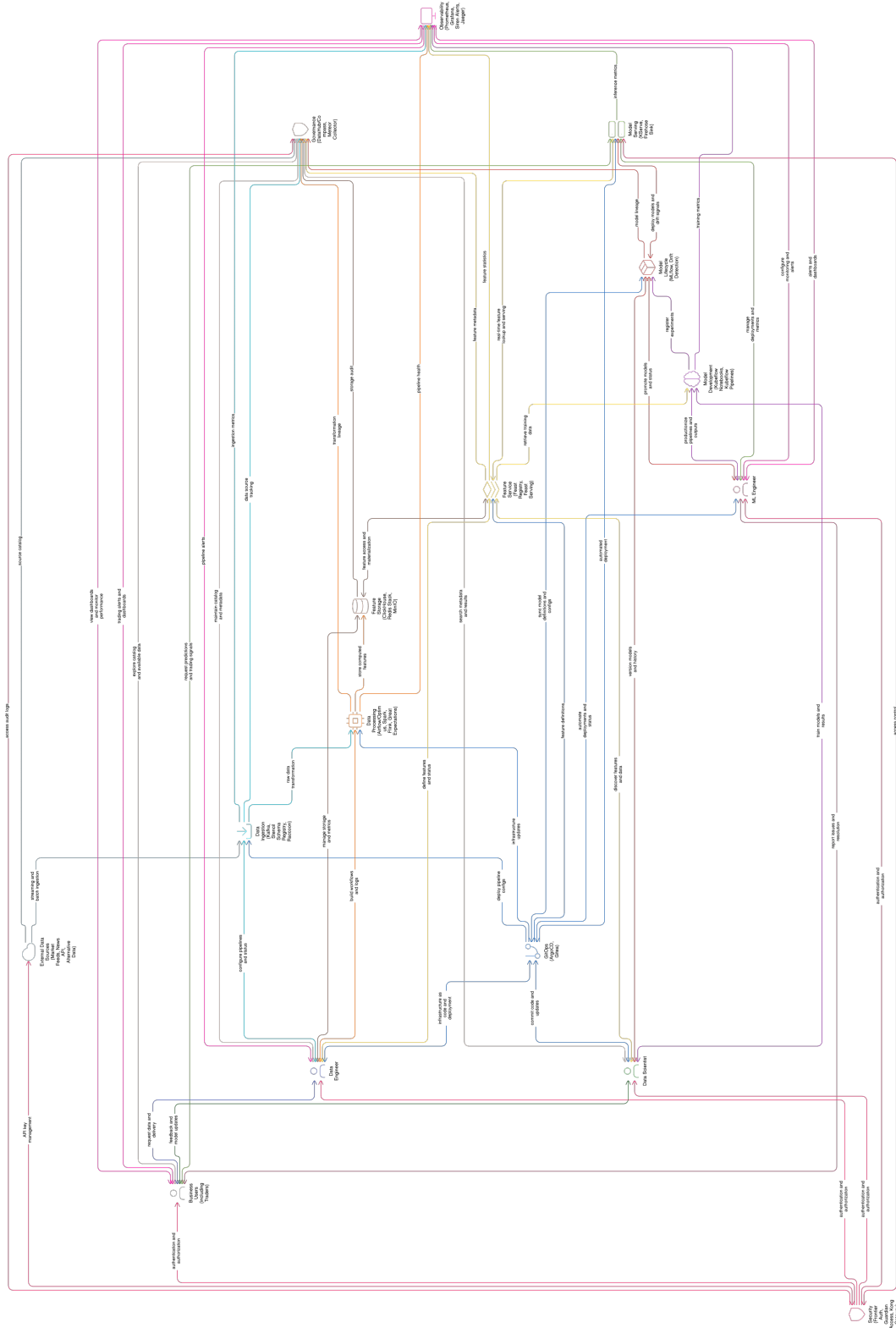
an risiko ketidaksesuaian dan sanksi.

Permasalahan lain yang penting adalah *training-serving skew* dan validitas temporal pada data *time-series*. Skew muncul ketika logika fitur pada tahap pelatihan tidak persis sama dengan logika *inference* di produksi, sehingga model gagal mempertahankan performa setelah di-*deploy* (Polyzotis dkk. 2018). Tanpa *feature store* terpusat, perbedaan ini menjadi masalah laten yang sulit dideteksi dan mudah terulang. Pada data perdagangan, kebocoran data temporal juga mengancam validitas model dan membuat hasil *backtesting* terlalu optimistis (Polyzotis dkk. 2018; Wang dan Ruf 2021). Implementasi *point-in-time correct join* secara manual di berbagai lingkungan terpisah sangat rentan kesalahan, sehingga risiko kebocoran mudah berulang di banyak *pipeline* tanpa terdeteksi.

IV.1.2 Arsitektur Sistem yang Diusulkan: Platform Terintegrasi

Arsitektur yang diusulkan mengubah pola *loosely coupled tools* menjadi platform DataOps–MLOps terintegrasi yang menopang siklus hidup ML secara menyeluruh. Platform ini menyatukan *workflow* utama, meningkatkan otomasi, dan menambahkan lapisan *governance* yang eksplisit sehingga hambatan pada arsitektur eksisting dapat diatasi secara sistematis. Desain mengadopsi prinsip *microservices* dengan *API Gateway* (Fowler dan Lewis 2014; Newman 2015), infrastruktur deklaratif, GitOps, dan pendekatan *observability-driven*. Lapisan keamanan dibangun berlapis dengan JWT, OAuth 2.0, dan mTLS yang dipadukan dengan *centralized identity* dan *access orchestration*. Dalam konfigurasi ini, *identity provider* seperti Frontier menangani otentikasi, sementara *access workflow engine* seperti Guardian mengelola permintaan akses terhadap aset data secara terdokumentasi dan dapat diaudit, sejalan dengan pola RBAC dan prinsip *least privilege*.

Gambar IV.6 memperlihatkan pergeseran alur dari pola linear yang sarat proses manual menjadi pola *hub-and-spoke* dengan tiga pilar utama. Tiga pilar tersebut adalah GitOps (ArgoCD) sebagai sumber kebenaran konfigurasi, *Governance* (DataHub) sebagai pusat metadata dan *lineage*, serta *Observability* (Prometheus, Grafana, dan komponen terkait) sebagai landasan pemantauan (Google Cloud 2024; Kreuzberger, Köhl, dan Hirschl 2023; Cloud Native Computing Foundation 2024). Lapisan keamanan dan akses yang melibatkan Frontier, Guardian, mekanisme *secrets management*, dan *API Gateway* berjalan melintang di atas ketiga pilar tersebut dan melindungi seluruh alur. Dengan struktur ini, setiap perubahan, insiden, dan keputusan operasional selalu dapat dirujuk kembali ke informasi yang konsisten dan tersimpan di satu tempat.

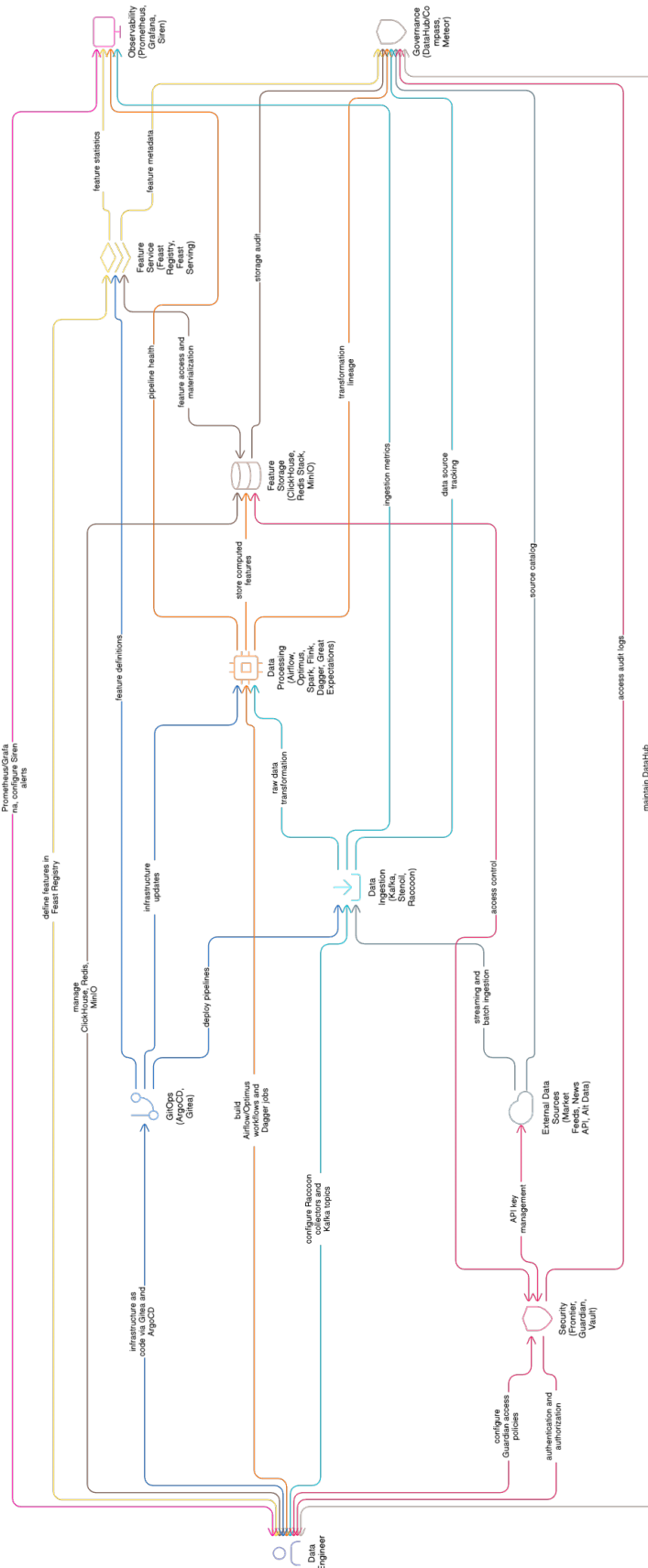


Gambar IV.6 Arsitektur Terintegrasi yang Diusulkan (General)

Arsitektur baru dirancang untuk mengurangi silo di setiap *persona*. Semua komponen fungsional dan peran dihubungkan melalui tiga pilar sentral, sejalan dengan

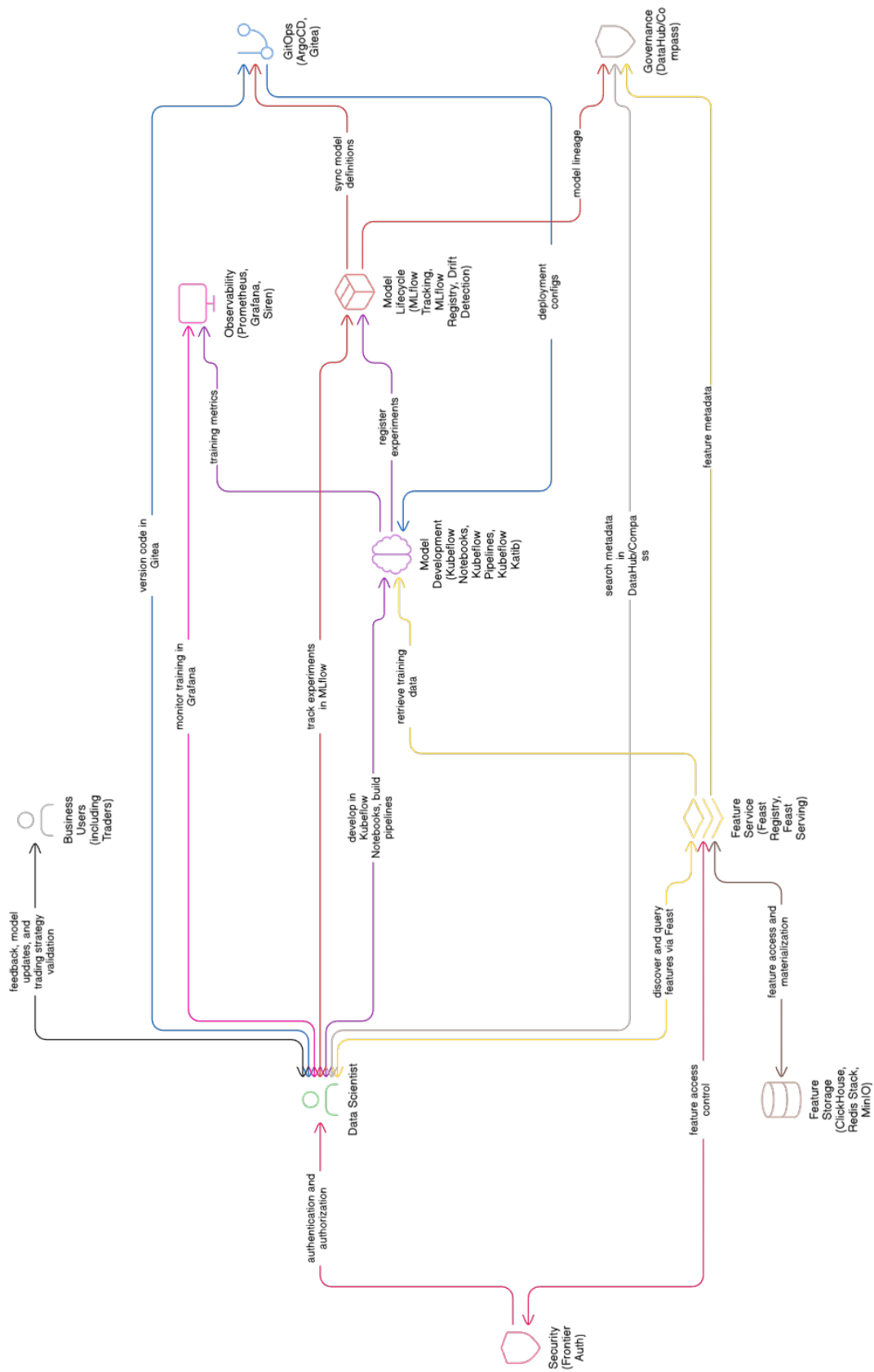
Level 2 pada *Google Cloud MLOps framework* (Google Cloud 2024; Kreuzberger, Kühn, dan Hirschl 2023). Dengan penyusunan seperti ini, perubahan di satu area dapat dikendalikan dan diaudit secara menyeluruh karena selalu terekam pada pilar GitOps, Governance, dan Observability. Konsekuensinya, reliabilitas sistem dan kualitas tata kelola meningkat, sementara friksi antartim menurun dan *feedback loop* bisnis–teknis menjadi lebih pendek.

Gambar IV.7 menunjukkan bahwa DE tidak lagi mengelola *pipeline* secara manual per komponen, tetapi mengoperasikan *pipeline* otomatis yang menghubungkan *Data Ingestion* (Kafka) dengan *Data Processing* (Airflow, Spark, Flink). Pada sisi tepi, *event collector* seperti Raccoon menerima *event* ber-throughput tinggi dari aplikasi perdagangan dan meneruskannya ke Kafka, sementara *schema registry* seperti Stencil menjaga konsistensi *schema* dan evolusinya. Definisi *job* dan jadwal disimpan di Git dan disinkronkan menggunakan GitOps, aset dan *lineage*-nya tercatat di DataHub, dan reliabilitas *pipeline* dipantau melalui lapisan *observability*. Dengan pola ini, keterkaitan hulu–hilir data menjadi transparan dan dapat dikelola secara terpusat.



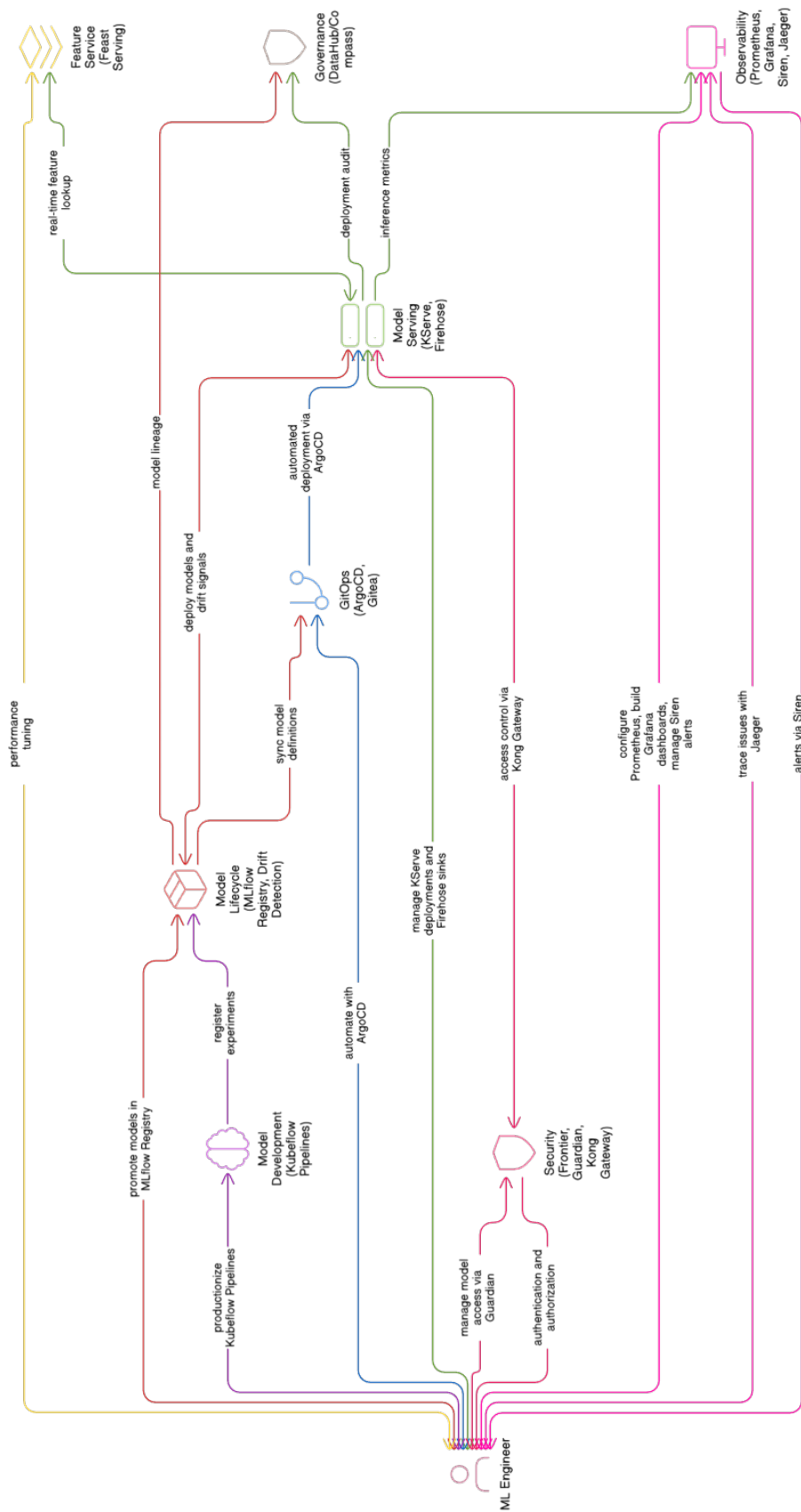
Gambar IV.7 Arsitektur Terintegrasi yang Diusulkan (Data Engineer)

Gambar IV.8 memperlihatkan bahwa DS mengakses fitur melalui *Feature Service* (Feast) yang menyediakan definisi konsisten untuk pelatihan dan *serving*. Eksperimen dijalankan pada Kubeflow Notebooks atau Kubeflow Pipelines yang berada di klaster yang sama dengan produksi sehingga perbedaan lingkungan dapat diminimalkan. Aset data, fitur, dan model dapat ditelusuri melalui DataHub, sehingga konteks teknis dan bisnis tidak terputus ketika model berpindah tahap. *Experiment tracking* dilakukan dengan MLflow, dan akses ke dataset serta fitur dikelola oleh Guardian agar tetap selaras dengan kebijakan akses organisasi.



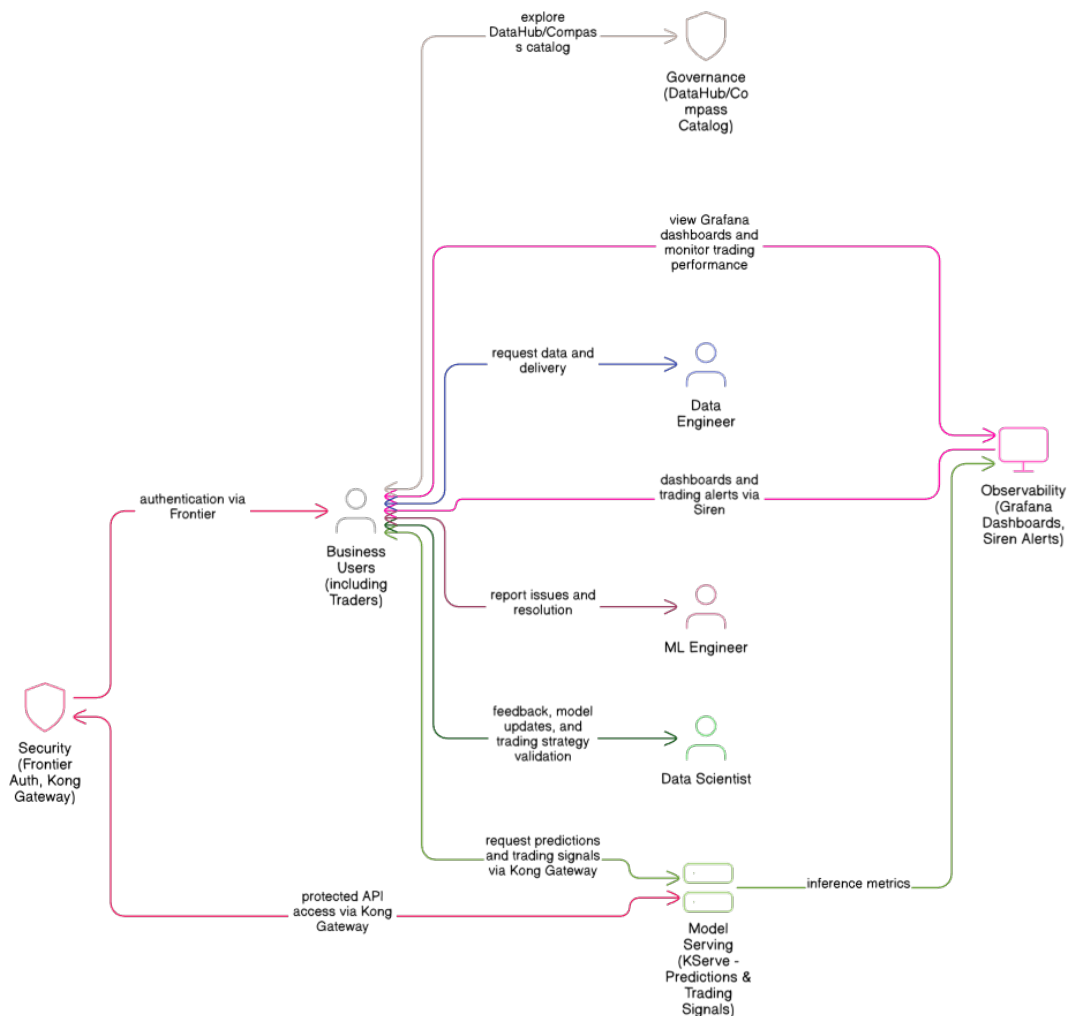
Gambar IV.8 Arsitektur Terintegrasi yang Diusulkan (Data Scientist)

Gambar IV.9 menunjukkan bahwa MLE tidak lagi melakukan *rewrite* manual dari model ke layanan produksi. MLE mempromosikan model melalui MLflow *registry*, yang kemudian memicu pembaruan konfigurasi di Git untuk sumber daya KServe. ArgoCD menyinkronkan konfigurasi tersebut ke kluster dan menyiapkan *deployment* secara deklaratif, sehingga kesalahan manusia berkurang dan proses *deployment* menjadi lebih dapat diprediksi. Metrik kinerja dan *drift* dimonitor lewat *observability stack*, sementara *sink service* seperti Firehose digunakan untuk menyalurkan log prediksi dan *feedback* ke data store analitik agar siklus *monitoring* dan *retraining* dapat diotomasi.



Gambar IV.9 Arsitektur Terintegrasi yang Diusulkan (ML Engineer)

Gambar IV.10 menunjukkan bahwa BU diposisikan sebagai bagian yang lebih aktif di dalam platform, bukan sekadar penerima hasil. BU dapat mengakses *dashboard* KPI dan metrik model secara *real-time* untuk memantau kinerja strategi perdagangan. Selain itu, BU dapat menelusuri data dan model di DataHub maupun katalog ringan seperti Compass untuk memahami konteks prediksi yang digunakan. Pola ini memperpendek *feedback loop* antara perspektif teknis dan bisnis dan memungkinkan diskusi yang lebih berbasis data, sehingga keputusan bisnis dapat dikaitkan langsung dengan kondisi aktual sistem.



Gambar IV.10 Arsitektur Terintegrasi yang Diusulkan (Business Users)

Kubernetes dipilih sebagai lapisan orkestrasi utama karena tingkat adopsi yang luas, dukungan yang baik terhadap *workload* ML, portabilitas lintas *cloud*, dan kemampuan mereproduksi lingkungan berbasis kontainer. Komponen fungsional disusun ke dalam beberapa lapisan arsitektur untuk memisahkan *concerns* dan memungkinkan setiap lapisan diskalakan secara independen (Kleppmann 2017; Marz dan Warren

2015; Stopford 2018). Penyusunan berlapis ini memudahkan pemetaan tanggung jawab dan memberikan fleksibilitas ketika organisasi ingin menambah atau mengganti komponen di masa mendatang. Rincian kontribusi setiap lapisan terhadap kebutuhan sistem dibahas lebih lanjut pada bagian *Desain Detail Lapisan Arsitektur* di bab ini sebagai dasar implementasi.

GitOps digunakan untuk memastikan konfigurasi *deployment* dan infrastruktur dikelola sebagai kode di Git, dengan ArgoCD sebagai komponen yang menyinkronkan keadaan kluster dengan isi repositori. Pola ini menyediakan *audit trail*, memudahkan *rollback*, dan meningkatkan reproduksibilitas ketika terjadi insiden atau percobaan konfigurasi baru. Dengan pendekatan ini, perubahan terhadap komponen Raystack maupun komponen lain dikelola melalui mekanisme kontrol versi yang sama di seluruh lapisan. Siklus perubahan konfigurasi dan *deployment* menjadi konsisten, sehingga risiko perubahan tak terpantau dapat dikurangi.

IV.1.3 Analisis Perbedaan Kunci

Perbandingan antara arsitektur eksisting dan arsitektur terintegrasi menunjukkan pergeseran dari Level 0/1 menuju Level 2 MLOps Kreuzberger, Kühl, dan Hirschl (2023). Pergeseran ini menyentuh sisi teknologi, pola kerja lintas tim, dan cara organisasi mengelola risiko operasional. Tabel IV.1 merangkum perbedaan utama dari sisi teknis, konfigurasi yang diusulkan, dan manfaat yang ditargetkan. Penyajian dalam bentuk tabel memudahkan pembaca melihat hubungan langsung antara keputusan desain dan dampak operasional tanpa perlu menelusuri seluruh penjelasan naratif.

Tabel IV.1 Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Orkestrasi	<i>Compute</i> heterogen dengan <i>provisioning</i> manual, waktu tunggu <i>scaling</i> panjang, dan tingkat standarisasi rendah.	Platform Kubernetes terpadu dengan manajemen deklaratif dan alokasi sumber daya otomatis, sejalan dengan komponen arsitektur MLOps yang diidentifikasi Kreuzberger, Kühl, dan Hirschl (2023). GitOps dengan ArgoCD memastikan perubahan infrastruktur dan aplikasi (termasuk komponen Raystack) mengikuti <i>single source of truth</i> di Git.	Studi yang dirangkum Lakka (2025) menunjukkan bahwa organisasi <i>high-performing</i> yang menerapkan praktik DevOps berbasis Kubernetes dapat meningkatkan <i>deployment frequency</i> hingga $973\times$ dan memperbaiki MTTR hingga $6570\times$; arsitektur yang diusulkan menargetkan karakteristik performa yang serupa.	(Kreuzberger, Kühl, dan Hirschl 2023; Lakka 2025)
Manajemen Fitur	Logika fitur tersebar di <i>notebook</i> dan skrip dengan duplikasi tinggi dan banyak fungsi yang saling tumpang tindih.	Feast dengan <i>feature registry</i> terpusat, <i>dual serving offline/online</i> , serta dukungan <i>point-in-time correctness</i> .	Duplikasi fitur dapat ditekan, <i>training-serving skew</i> berkurang melalui pemisahan <i>offline/online</i> dan <i>point-in-time retrieval</i> , dan <i>feature reuse</i> berpotensi meningkatkan signifikan.	(Munappy dkk. 2022; Polyzotis dkk. 2018)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
<i>Vector Embeddings</i>	Fitur <i>embedding</i> tidak didukung secara <i>native</i> atau diimplementasikan secara kustom dengan latensi tinggi dan <i>throughput</i> terbatas.	Redis Stack dengan dukungan HNSW <i>vector search</i> berlatensi rendah.	Johnson, Douze, dan Jégou (2021) menunjukkan bahwa penggunaan FAISS pada GPU dapat memberikan percepatan hingga 8,5 kali pada dataset YFCC100M; desain ini memanfaatkan prinsip serupa untuk <i>similarity search</i> berperforma tinggi.	(Johnson, Douze, dan Jégou 2021)
Validitas Temporal	<i>Point-in-time join</i> dilakukan manual dan rentan kesalahan, sehingga banyak tim mengalami <i>data leakage</i> .	<i>Point-in-time join</i> otomatis dari Feast dengan pola verifikasi yang lebih sistematis.	Pendekatan ini mencegah kebocoran data temporal yang dapat menghasilkan estimasi performa <i>backtest</i> yang terlalu optimistis dan tidak realistis di produksi.	(Polyzotis dkk. 2018)
Pemrosesan Data	<i>Batch</i> dan <i>stream</i> diproses oleh <i>pipeline</i> terpisah tanpa arsitektur yang koheren, sehingga hasil pemrosesan tidak konsisten.	Spark sebagai mesin <i>batch processing</i> dan Flink sebagai mesin <i>stream processing</i> yang hasilnya dimaterialisasikan langsung ke <i>Feature Store</i> . Optimus dan Dagger memfasilitasi definisi <i>pipeline</i> berbasis konfigurasi sehingga pengembangan dan pengelolaan jadwal transformasi menjadi lebih sistematis.	Waktu pengembangan <i>pipeline</i> dapat menurun dan <i>freshness</i> data meningkat berkat pemrosesan <i>streaming</i> yang lebih efisien dan terintegrasi.	(Rodrigues dkk. 2025)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Pelatihan Model	<i>Workflow</i> pelatihan berbasis <i>notebook</i> dengan <i>tracking</i> eksperimen informal dan <i>artifact</i> tidak terkelola.	Kubeflow <i>Pipelines</i> untuk orkestrasi pelatihan dan MLflow untuk <i>tracking</i> eksperimen.	Reprodusibilitas meningkat melalui kontainerisasi dan pengelolaan eksperimen terpusat, sementara siklus pengembangan <i>pipeline</i> menjadi lebih terstruktur.	(Amershi dkk. 2019; Pimentel dkk. 2019)
Deployment Model	Proses <i>deployment</i> manual dengan <i>refactoring</i> besar dan sering terjadi <i>bug</i> pasca- <i>deploy</i> .	KServe dengan <i>InferenceService</i> deklaratif dan <i>deployment</i> otomatis melalui GitOps. Firehose dapat digunakan untuk menyalurkan log <i>inference</i> ke data store secara konsisten sebagai dasar <i>monitoring</i> dan analitik lanjutan.	Waktu <i>deployment</i> berpotensi turun secara signifikan, dan insiden terkait kesalahan konfigurasi <i>deployment</i> dapat dikurangi lewat otomasi.	(Paleyes, Urma, dan Lawrence 2022)
Pola Deployment Lanjut	Pola seperti <i>canary release</i> jarang digunakan, organisasi umumnya mengandalkan <i>binary switchover</i> .	Dukungan <i>canary release</i> , A/B <i>testing</i> , dan <i>automatic rollback</i> prediktif.	<i>Rollout</i> dapat dilakukan secara bertahap dan aman, sehingga MTTR menurun karena kegagalan dapat dibatasi pada <i>canary traffic</i> .	(Shankar dkk. 2022)
Deteksi Drift	Deteksi <i>drift</i> bersifat reaktif dengan jeda deteksi yang panjang terhadap isu penting dan perubahan gradual.	<i>Monitoring</i> proaktif dengan uji statistik, <i>alert real-time</i> , dan penetapan ambang adaptif.	Jeda deteksi <i>drift</i> dapat dipersingkat sehingga <i>retraining</i> dapat dijadwalkan lebih tepat waktu untuk menjaga kinerja model.	(Céspedes Sisniega dkk. 2024; Bayram, Ahmed, dan Kassler 2022)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
<i>Governance & Lineage</i>	Dokumentasi dilakukan secara manual dan <i>audit trail</i> sering tidak lengkap atau tersebar.	DataHub dengan <i>metadata ingestion</i> otomatis (dibantu Meteor) dan <i>lineage</i> yang dapat ditelusuri dari <i>source</i> hingga model; Compass dapat melengkapi sebagai katalog ringan untuk aset tertentu.	Waktu persiapan audit dapat berkurang dan cakupan metadata meningkat melalui proses <i>ingestion</i> dan <i>lineage</i> otomatis.	(Stoyanovich, Howe, dan Jagadish 2020; Lamer dkk. 2024)
<i>Observability</i>	Fokus <i>monitoring</i> pada metrik infrastruktur, tanpa cakupan metrik ML spesifik di sebagian besar organisasi.	<i>Observability</i> mencakup metrik ML, <i>dashboard</i> kinerja model, dan <i>alert</i> berbasis indikator <i>drift</i> . Prometheus dan Grafana menjadi inti, sementara Siren mengonsolidasikan <i>alert</i> ke berbagai kanal notifikasi.	Proses <i>debugging</i> dan <i>root cause analysis</i> menjadi lebih efektif sehingga MTTR dapat diturunkan.	(Shankar dkk. 2022)
Integrasi CI/CD	CI/CD untuk <i>pipeline</i> dan model dilakukan secara manual atau semiotomatis, dengan <i>workflow</i> yang tidak seragam.	GitOps dengan ArgoCD sebagai mekanisme CI/CD deklaratif dengan Git sebagai sumber kebenaran tunggal, mencakup konfigurasi komponen MLOps dan komponen Raystack.	Frekuensi <i>deployment</i> dapat meningkat dan variasi kesalahan <i>deployment</i> dapat ditekan melalui otomasi dan <i>peer review</i> pada <i>pull request</i> .	(Kreuzberger, Kühl, dan Hirschl 2023)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Pengalaman <i>Developer</i>	<i>Developer</i> menghabiskan banyak waktu untuk tugas repetitif, <i>onboarding</i> lama, dan penggunaan alat yang terpisah.	Platform terintegrasi dengan <i>self-service</i> untuk data, fitur, <i>pipeline</i> , dan akses; Frontier dan Guardian membantu menyederhanakan proses permohonan hak akses.	<i>Onboarding</i> dapat dipercepat dan produktivitas <i>data scientist</i> meningkat karena hambatan akses infrastruktur berkurang.	(Paleyes, Urma, dan Lawrence 2022; Rella 2022)
Efisiensi Biaya	Utilisasi sumber daya tidak optimal dan tidak ada atribusi biaya yang jelas ke <i>workload</i> .	<i>Auto-scaling</i> , pemantauan biaya, dan alokasi sumber daya yang lebih terukur pada kluster Kubernetes. Seluruh komponen inti, termasuk ekosistem Raystack, berbasis <i>open source</i> sehingga biaya lisensi dapat ditekan.	Penghematan biaya signifikan dapat dicapai melalui optimasi pemakaian sumber daya dan <i>auto-scaling</i> , sebagaimana ditunjukkan dalam berbagai studi kasus adopsi Kubernetes (Lakka 2025); desain ini mengadopsi prinsip yang sama untuk mengoptimalkan pemakaian sumber daya.	(Lakka 2025)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Keamanan & Compliance	Kontrol akses beragam di tiap sistem, dan <i>audit</i> tersebar tanpa pandangan terpadu.	RBAC terintegrasi, <i>audit trail</i> komprehensif, dan pengecekan <i>compliance</i> yang terotomatisasi. Frontier memusatkan identitas, Guardian menangani <i>access workflow</i> ke sistem data, sementara kebijakan jaringan dan <i>secrets management</i> dikelola pada tingkat klaster.	Risiko insiden keamanan dapat berkurang melalui kontrol terpusat dan proses audit menjadi lebih ringkas.	(Ahmad dkk. June 2024)

Transformasi ke platform terintegrasi dirancang untuk menjawab akar masalah yang telah diidentifikasi pada Bab III. Feast diterapkan untuk menegakkan konsistensi fitur dan mengurangi *training-serving skew*, sementara GitOps mempercepat dan menertibkan *deployment*. Lapisan *observability* digunakan untuk mendukung deteksi masalah secara proaktif, dan DataHub memperkuat *governance* melalui metadata dan *lineage* komprehensif. Ekosistem Raystack (Frontier, Guardian, Raccoon, Stencil, Optimus, Dagger, Firehose, Siren, Meteor, Compass) berfungsi sebagai *building block* tambahan yang memperkaya lapisan akses, *ingestion*, transformasi, *observability*, dan metadata tanpa mengubah prinsip desain inti. Dengan demikian, platform tetap *extensible* dan adaptif terhadap dinamika aplikasi perdagangan finansial (Burgueno-Romero, Cánovas Izquierdo, dan García-García 2025).

IV.2 Desain Detail Lapisan Arsitektur

Bagian ini menguraikan desain tiap lapisan arsitektur agar kontribusi masing-masing komponen terhadap kemampuan sistem dapat ditelusuri dengan jelas. Penjelasan difokuskan pada alasan desain, dasar pemilihan teknologi, serta pola integrasi yang digunakan untuk menjaga operasi platform tetap konsisten. Dengan pendekatan ini, keterkaitan antara keputusan arsitektural dan kebutuhan fungsional maupun non-fungsional yang telah dirumuskan pada Bab III tetap terjaga. Penyusunan per la-

pisan juga memudahkan analisis risiko, identifikasi titik lemah, dan perencanaan pengembangan bertahap.

Rancangan berlapis pada bab ini sekaligus berfungsi sebagai landasan konseptual bagi tahap implementasi dan evaluasi pada bab berikutnya. Setiap lapisan dijelaskan sejauh relevansinya dengan *use case* perdagangan saham dan *cryptocurrency* yang menjadi fokus studi kasus. Dengan demikian, deskripsi tidak hanya bersifat generik, tetapi juga dikaitkan dengan kebutuhan domain yang telah dibahas sebelumnya. Penjelasan yang diberikan tetap diarahkan pada aspek yang berdampak langsung terhadap *workflow* end-to-end dan bukan pada detail konfigurasi operasional yang terlalu teknis untuk lingkup proposal tugas akhir.

IV.2.1 Lapisan Infrastruktur dan Orkestrasi

Lapisan infrastruktur dan orkestrasi menjadi fondasi bagi seluruh komponen di atasnya karena mengelola sumber daya komputasi, penjadwalan *workload*, jaringan, reliabilitas, keamanan, dan kebijakan akses. Kubernetes ditempatkan sebagai elemen inti karena tingkat kematangan tinggi, adopsi luas, dan dukungan ekosistem terhadap *workload* ML. Dengan fondasi ini, skalabilitas, ketersediaan, dan isolasi beban kerja dapat dicapai tanpa mengunci organisasi pada satu vendor. Sifat deklaratif Kubernetes juga memudahkan penerapan GitOps dan otomatisasi konfigurasi di berbagai lingkungan. Penempatan Kubernetes pada lapisan dasar terkait langsung dengan kebutuhan KNF yang menuntut keandalan serta kemudahan pemeliharaan jangka panjang.

Lapisan ini juga menjadi lokasi penerapan kebijakan keamanan lintas platform dan integrasi dengan komponen seperti Frontier, Guardian, dan Vault. Kebijakan akses terhadap kluster, *namespace*, dan *service* diturunkan dari identitas yang dikelola secara terpusat. Dengan pendekatan tersebut, tim dapat mengelola *workload* data dan ML dalam satu lingkungan yang konsisten tanpa harus membangun mekanisme kontrol akses terpisah untuk setiap komponen. Struktur ini memperkuat posisi lapisan infrastruktur dan orkestrasi sebagai tulang punggung platform yang menyeimbangkan fleksibilitas dengan kontrol.

1. Arsitektur Kluster Kubernetes

Kluster Kubernetes *production-grade* di-deploy dengan konfigurasi *multi-node* untuk mencapai *high availability* dan skalabilitas sebagaimana dibahas Lakka (2025). *Control plane* disebar ke beberapa *master node* untuk mengurangi risiko *single point of failure* yang krusial bagi target *availability* 99,9% (KNF-

04). Layanan *stateful* seperti *etcd* dijalankan dalam konfigurasi *highly available* sehingga status kluster tetap konsisten ketika sebagian *node* mengalami gangguan. *Worker node* dikelompokkan menjadi beberapa *node pool* sesuai jenis *workload*, misalnya *general-purpose pool* untuk validasi data dan orkestrasi, *GPU-enabled pool* untuk *training deep learning*, dan *high-memory pool* untuk *job* Spark atau kueri analitik berat di ClickHouse.

2. Kerangka Manajemen Sumber Daya

Namespace digunakan untuk memberikan isolasi logis antartim atau *tenant* pada skenario *multi-tenancy*. *Resource Quotas* dan *Limit Ranges* mengontrol konsumsi CPU dan memori sehingga satu tim tidak dapat menghabiskan seluruh sumber daya kluster. *Priority Classes* memprioritaskan *workload* produksi dibanding *job* eksperimen, sedangkan *Pod Disruption Budgets* memastikan jumlah minimum *pod* tetap berjalan selama pemeliharaan. Kombinasi mekanisme ini membantu menjaga stabilitas layanan penting tanpa menghambat fleksibilitas eksperimen.

3. Arsitektur Penyimpanan

Beberapa *storage class* disediakan untuk menangani pola akses yang berbeda. Media berlatensi rendah digunakan bagi komponen I/O intensif seperti ClickHouse dan Redis, sedangkan *object storage* kompatibel S3 (misalnya MinIO) menyimpan *training dataset*, *model binary*, dan log berukuran besar (Kreuzberger, Kühl, dan Hirschl 2023). Dengan pemisahan ini, biaya penyimpanan dapat dioptimalkan tanpa mengorbankan kinerja. Arsitektur penyimpanan tersebut juga memudahkan pemindahan data antar lingkungan jika dibutuhkan.

4. Jaringan dan Service Discovery

ClusterIP digunakan untuk komunikasi internal yang stabil, sementara *LoadBalancer* mengekspos layanan seperti endpoint *inference* atau antarmuka pengguna. *Ingress* mengatur *routing* permintaan dari luar kluster, termasuk *SSL termination* untuk keamanan koneksi. *Network Policy* membatasi lalu lintas berdasarkan prinsip hak akses minimal (Ahmad dkk. June 2024), sehingga hanya layanan yang berkepentingan yang dapat saling terhubung. *API Gateway* bertindak sebagai pintu masuk tunggal yang menerapkan *rate limiting*, *request validation*, serta integrasi dengan Frontier untuk otentikasi.

5. Manajemen Identitas dan Akses Lintas Platform

Frontier berperan sebagai pusat identitas dan otentikasi yang mengelola pengguna, organisasi, serta integrasi SSO dengan penyedia identitas eksternal.

Frontier mengeluarkan token yang kemudian divalidasi oleh *API Gateway* dan layanan hilir sebelum permintaan diproses. Untuk akses ke data dan aset sensitif, Guardian mengorkestrasi *access request workflow*, termasuk perekaman alasan, persetujuan berjenjang, dan *audit log*. Kombinasi Frontier dan Guardian melengkapi mekanisme RBAC Kubernetes sehingga kebijakan akses konsisten dari lapisan antarmuka pengguna hingga lapisan data.

6. **Continuous Deployment Berbasis GitOps**

ArgoCD menerapkan GitOps dengan menjadikan Git sebagai *single source of truth* bagi konfigurasi dan *deployment*. Perubahan dilakukan melalui *pull request* sebagaimana disarankan Limoncelli (2022), sehingga setiap perubahan tercatat dan dapat ditinjau sebelum diterapkan. Proses CI dipisahkan dari CD, sedangkan *controller* ArgoCD menyinkronkan konfigurasi dari Git ke kluster. Pola ini meningkatkan kontrol, memudahkan *auditability*, dan menjaga konsistensi konfigurasi (Limoncelli 2022; Kim dkk. 2016; Kreuzberger, Kühl, dan Hirschl 2023).

7. **Fondasi Monitoring dan Observability**

Prometheus mengumpulkan metrik melalui mekanisme *scraping*, AlertManager mengelola *alert* di tingkat infrastruktur, dan Grafana menyediakan *dashboard* untuk visualisasi dan analisis tren (Prometheus Authors 2024; Grafana Labs 2024a). Di atas itu, Siren dapat digunakan sebagai *alert router* terpusat yang mengatur penyaluran notifikasi ke Slack, email, atau kanal lain. Fondasi ini memastikan bahwa kondisi kluster dapat dipantau secara *real-time* dan insiden dapat ditangani lebih cepat.

8. **Pengerasan Keamanan**

Pod Security Standards, *Admission Controllers*, *Service Account*, dan *Network Policies* digunakan untuk menegakkan keamanan berlapis. *Secrets* disimpan dalam bentuk terenkripsi dan pengelolaannya diperkuat dengan solusi seperti Vault untuk menghasilkan kredensial dinamis dan melakukan rotasi kunci secara berkala. *Container image* dipindai kerentanannya sebelum *deploy*, dan akses ke *registry* dibatasi agar hanya sumber tepercaya yang dapat digunakan (Ahmad dkk. June 2024). Pendekatan ini membantu mengurangi permukaan serangan di seluruh platform.

Secara keseluruhan, lapisan infrastruktur dan orkestrasi menyediakan landasan bersama bagi komponen data dan ML yang berjalan di atasnya. Setiap keputusan di lapisan ini dihubungkan langsung dengan kebutuhan KNF terkait ketersediaan, keamanan, dan kemudahan pemeliharaan. Dengan menstandarisasi lingkungan ekse-

kusi, tim dapat berfokus pada logika bisnis dan rekayasa data, bukan pada pengelolaan server manual. Lapisan ini juga mempermudah penerapan praktik *DevSecOps* karena kontrol dan *observability* terpusat, sehingga perubahan di lapisan atas dapat dilakukan dengan risiko yang lebih terkendali dan proses audit lebih sederhana.

IV.2.2 Lapisan Ingesti dan *Streaming*

Lapisan ingest dan *streaming* membentuk jalur masuk bagi data dari berbagai sistem sumber. Lapisan ini bertugas mengonsumsi data eksternal, menyediakan *buffering* tahan lama, dan mendukung pola *real-time* maupun *batch*. Kafka dipilih sebagai *streaming platform* utama karena kombinasi *throughput* tinggi, latensi rendah, dan sifat *log* terdistribusi yang tahan lama (Apache Software Foundation 2024c). Di samping itu, Kafka memiliki ekosistem *connector* yang kaya sehingga integrasi dengan sistem lain dapat dilakukan tanpa penulisan adaptor kustom yang berat. Carbone dkk. (2015) menekankan pentingnya mesin pemrosesan terpadu untuk menjaga konsistensi arsitektur *stream* dan *batch*, dan prinsip tersebut tercermin dalam desain lapisan ini.

Lapisan ini juga menjadi titik awal penerapan *event-driven architecture* untuk domain perdagangan yang sangat dinamis. Produser dan konsumen dihubungkan melalui *event* sehingga dapat berkembang relatif independen tanpa harus mengetahui implementasi satu sama lain (Amazon Web Services 2024; Stopford 2018). Validasi *schema* dan kualitas data dilakukan sedekat mungkin dengan titik masuk untuk mengurangi risiko kesalahan yang merembet ke hilir. Dengan demikian, lapisan ingest dan *streaming* berperan penting dalam menjaga integritas data dan menyediakan dasar yang stabil bagi lapisan pemrosesan dan fitur.

1. Arsitektur Klaster Kafka

Klaster Kafka dikonfigurasi dengan beberapa *broker* dan replikasi agar tetap berfungsi ketika satu *broker* gagal. Parameter `min.insync.replicas` diatur sehingga *acknowledgement* penulisan hanya diberikan jika jumlah replika yang aktif sudah mencukupi. Dengan cara ini, pesan yang dinyatakan berhasil ditulis tetap aman meskipun terjadi gangguan pada sebagian *broker*. Pembagian *topic* menjadi beberapa *partition* memungkinkan konsumen memproses data secara paralel dan meningkatkan *throughput*.

2. *Schema Registry* dengan *Stencil*

Schema Registry menyimpan *schema* pesan dan menegakkan aturan kompatibilitas antarversi. *Stencil* digunakan sebagai *schema registry* ringan untuk format seperti Protobuf dan menyediakan mekanisme validasi *schema* di si-

si produser maupun konsumen. Validasi ini mencegah data yang tidak sesuai *schema* masuk ke sistem hilir dan memudahkan evolusi *schema* yang tetap kompatibel. Dengan demikian, perubahan format data dapat dilakukan secara terkendali tanpa mengorbankan kestabilan *pipeline*.

3. Kolektor *Event Raccoon*

Raccoon berperan sebagai *event collector* ber-throughput tinggi yang menerima *event* dari aplikasi perdagangan melalui HTTP, gRPC, atau protokol lain yang sesuai. Raccoon melakukan *batching* dan kompresi pesan sebelum menuliskannya ke Kafka sehingga beban koneksi ke Kafka berkurang di sisi klien. Dengan pola ini, aplikasi klien tidak perlu mengetahui detail topologi dan konfigurasi Kafka dan cukup berinteraksi dengan Raccoon. Pendekatan ini juga memudahkan penerapan *rate limiting* dan autentikasi di satu titik.

4. Kerangka Kerja Kafka Connect dan Firehose

Kafka Connect menyediakan mekanisme standar untuk menghubungkan sistem eksternal melalui *source connector* dan *sink connector*, sehingga integrasi dapat dilakukan tanpa penulisan kode kustom dari awal. Firehose melengkapi Kafka Connect sebagai layanan *sink* generik yang membaca *event* dari Kafka dan menuliskannya ke berbagai data store seperti ClickHouse, PostgreSQL, atau *data warehouse* melalui konfigurasi. Kombinasi keduanya mempercepat pengembangan *pipeline* dan mengurangi variasi implementasi integrasi, sehingga konsistensi dan *operability* lapisan ini dapat dijaga.

5. Ingesti Data Pasar dan *Alternative Data*

Source connector khusus atau komponen kolektor memanggil API pasar, menangani autentikasi dan *rate limiting*, lalu menormalkan data sebelum dikirim ke *topic* Kafka. Data *tick* dialirkan secara konsisten agar dapat diproses Flink dan disimpan di ClickHouse. Untuk *alternative data* seperti berita dan media sosial, *pipeline* terpisah mengekstrak teks, melakukan normalisasi, dan bila diperlukan mengubahnya menjadi *embedding* sentimen. *Embedding* tersebut kemudian dikirim ke *topic* lain sebagai sinyal pelengkap bagi model.

6. Pemrosesan *Stream* dengan Flink dan Dagger

Flink mengonsumsi pesan dari Kafka dan menjalankan transformasi *stateful* dengan jaminan *exactly-once semantics* (Apache Software Foundation 2024b). Penggunaan *event time* dan *watermark* menjaga kebenaran perhitungan meskipun pesan datang tidak berurutan (Carbone dkk. 2015). Dagger digunakan sebagai kerangka konfiguratif di atas Flink yang memungkinkan *job* didefinisikan dalam bentuk konfigurasi atau DSL, sehingga banyak transformasi

dapat dibangun tanpa penulisan kode Java/Scala secara manual. Pola ini menyederhanakan operasional sambil mempertahankan fleksibilitas.

7. Validasi Data dengan Great Expectations

Great Expectations digunakan untuk memeriksa kualitas data baik pada *batch pipeline* maupun *streaming pipeline* (Great Expectations 2024). *Expectation suite* mendefinisikan aturan seperti rentang nilai, kelengkapan, dan konsistensi antar kolom. Pelanggaran terhadap aturan kualitas dapat memicu *alert* dan ditindaklanjuti oleh tim data, sejalan dengan temuan Chien (February 2022) tentang pentingnya pemantauan kualitas. Hasil validasi ini juga direkam sebagai metadata kualitas di DataHub sehingga status kualitas data dapat dipertimbangkan ketika memilih dataset untuk pelatihan atau analisis.

Melalui lapisan ingesti dan *streaming*, data mengalir dari sumber ke berbagai komponen hilir secara andal dan dapat diawasi. Desain yang berfokus pada Kafka dan *event-driven architecture* mendukung kebutuhan aplikasi perdagangan yang menuntut pemrosesan *real-time*. Validasi *schema* dan kualitas data di titik masuk mengurangi risiko kesalahan di tahap berikutnya. Dengan memisahkan tanggung jawab koleksi, *buffering*, dan distribusi data, sistem menjadi lebih mudah dikembangkan dan dipelihara, serta menyediakan dasar yang kuat untuk menambahkan sumber data baru tanpa mengganggu *pipeline* yang sudah berjalan.

IV.2.3 Lapisan Pemrosesan Data

Lapisan pemrosesan data mengubah data mentah menjadi data yang siap digunakan dan mengorkestrasi transformasi kompleks yang melibatkan banyak langkah. Lapisan ini menjadi penghubung antara data yang dikumpulkan melalui Kafka dan fitur yang akhirnya disajikan kepada model. Arsitektur memadukan Airflow sebagai orkestrator *workflow*, Spark untuk *batch processing*, dan Flink untuk *stream processing*. Kombinasi ini selaras dengan pendekatan sistem terdistribusi yang diuraikan Rodrigues dkk. (2025) dan memberikan fleksibilitas dalam merancang *pipeline*. Dengan adanya lapisan ini, proses transformasi dapat didefinisikan, dijadwalkan, dan dimonitor secara konsisten.

Lapisan pemrosesan data juga memegang peran penting dalam penegakan validitas temporal dan kualitas data. Transformasi historis dan *window-based* yang berhubungan dengan *time-series* finansial dikelola secara eksplisit sehingga *point-in-time correctness* dapat dijaga. Validasi kualitas diterapkan di titik-titik strategis dalam *pipeline* dan hasilnya dikirim sebagai metadata ke DataHub. Dengan cara ini, dataset untuk pelatihan dan evaluasi model selalu dapat ditelusuri dari sisi asal-usul, tran-

sformasi, dan status kualitasnya. Lapisan ini mengikat praktik DataOps ke MLOps melalui proses yang terorkestrasi dengan jelas.

1. **Apache Airflow untuk Orkestrasi Alur Kerja**

Airflow digunakan untuk mengelola *pipeline* dengan dependensi kompleks dan jadwal beragam (Apache Software Foundation 2024a). DAG mendeskripsikan urutan tugas secara deklaratif, sementara *executor* di atas Kubernetes memungkinkan *task* dijalankan dalam kontainer terisolasi. UI Airflow menyediakan pemantauan status, riwayat eksekusi, dan log untuk setiap *task*. Pada lingkungan yang banyak menggunakan transformasi SQL/ELT, Airflow dapat bertindak sebagai *runtime* bagi Optimus sehingga definisi *job* lebih terstruktur dan terdokumentasi.

2. **Optimus untuk Transformasi dan *Data Modeling***

Optimus menyediakan lapisan deklaratif untuk mendefinisikan *data transformation job* berbasis SQL di atas *data warehouse* atau ClickHouse. Optimus mengelola dependensi antartabel, penjadwalan, dan sejumlah aturan kualitas sehingga model data untuk fitur dapat dibangun secara konsisten. Integrasi dengan Airflow membuat *job* Optimus dapat dijalankan sebagai bagian dari DAG yang sama dengan langkah non-SQL lain. Dengan pola ini, banyak transformasi dapat dikelola sebagai konfigurasi, bukan sebagai skrip yang tersebar.

3. **Apache Spark untuk Pemrosesan *Batch***

Spark menangani transformasi *batch* yang memerlukan komputasi terdistribusi, seperti agregasi berskala besar, *join* kompleks, dan *windowing* historis (Apache Software Foundation 2024d). *Catalyst optimizer* pada Spark meningkatkan efisiensi rencana eksekusi dan memanfaatkan struktur data kolumnar jika tersedia (Zaharia dkk. 2016). Spark dapat membaca dan menulis ke ClickHouse maupun MinIO sehingga integrasi dengan *offline store* dan *object storage* berjalan mulus. Hasil pemrosesan ini kemudian dimaterialisasikan sebagai fitur historis melalui Feast.

4. **Kualitas dan Validasi Data**

Validasi kualitas data dilakukan di berbagai titik dalam *pipeline* untuk memastikan data layak digunakan. Pengecekan meliputi konsistensi *schema*, kelengkapan nilai, rentang numerik yang wajar, deteksi *outlier*, dan kesesuaian dengan logika bisnis (Chien February 2022). Hasil validasi dicatat sebagai metadata kualitas di DataHub, sehingga pengguna dapat melihat status kualitas sebelum memakai dataset. Integrasi dengan Great Expectations di ta-

hap ini memastikan bahwa dataset untuk pelatihan dan *backtesting* memenuhi standar kualitas yang disepakati.

Dengan adanya lapisan pemrosesan data, organisasi memiliki satu tempat terstruktur untuk mendefinisikan, menjadwalkan, dan memantau transformasi yang menyangkut data skala besar. Penggunaan Airflow, Optimus, Spark, dan Flink memungkinkan berbagai pola pemrosesan diakomodasi tanpa kehilangan konsistensi operasional. Dokumentasi transformasi dalam bentuk DAG dan konfigurasi mempermudah analisis *lineage* dan *impact*. Ketika terjadi perubahan pada data sumber, tim dapat dengan cepat melihat *pipeline* mana yang terdampak, sehingga lapisan ini berkontribusi langsung terhadap keandalan dan transparansi sistem.

IV.2.4 Lapisan Layanan Fitur (*Feature Service*)

Lapisan layanan fitur menyediakan pengelolaan fitur secara terpusat agar konsistensi, kemudahan penemuan, dan *reuse* dapat tercapai. Lapisan ini menghubungkan data yang telah diproses dengan model yang membutuhkan representasi fitur yang stabil. Feast berfungsi sebagai komponen inti dengan arsitektur *dual-store* untuk *offline* dan *online* (Feast Project 2024; Munappy dkk. 2022). Pendekatan ini membantu mengatasi *training-serving skew* yang umum terjadi pada arsitektur terfragmentasi. Selain itu, lapisan ini memungkinkan berbagai tim berbagi fitur dengan definisi yang sama tanpa bergantung pada implementasi teknis masing-masing.

Lapisan layanan fitur juga menjadi tempat penegakan *point-in-time correctness* untuk *training dataset* dan pengelolaan *vector embeddings* sebagai sinyal tambahan. Dengan memanfaatkan ClickHouse sebagai *offline store* dan Redis Stack sebagai *online store*, fitur tabular dan *embedding* dapat disajikan melalui antarmuka yang sama. Integrasi dengan DataHub memastikan bahwa setiap fitur memiliki dokumentasi yang jelas, pemilik yang diketahui, dan *lineage* yang dapat ditelusuri. Desain ini menjadikan layanan fitur sebagai jembatan utama antara DataOps dan MLOps dalam arsitektur usulan.

1. Arsitektur dan Konsep Inti Feast

Feast mengelola fitur melalui konsep *Entity*, *Feature View*, *Feature Service*, dan *Feature Registry*. *Entity* merepresentasikan kunci utama seperti simbol saham atau akun, sedangkan *Feature View* mendeskripsikan sekumpulan fitur yang dihitung dari satu atau beberapa sumber. *Feature Service* mengelompokkan fitur-fitur yang relevan untuk suatu kasus penggunaan dan mempermudah klien dalam melakukan permintaan. Semua definisi tersebut disimpan

sebagai kode di Git sehingga dapat ditinjau, diuji, dan diaudit secara seragam.

2. **Offline Store dengan ClickHouse**

ClickHouse dipilih sebagai *offline store* karena performa kueri analitiknya yang tinggi dan dukungan terhadap penyimpanan kolumnar yang terkompresi (ClickHouse Inc 2024). Feast memanfaatkan ClickHouse untuk mengeksekusi *point-in-time correct join* saat menyusun *training dataset*, sehingga validitas temporal dapat dijaga dan kebocoran data dapat dihindari (Polyzotis dkk. 2018; Wang dan Ruf 2021). Kemampuan ini sangat penting untuk domain perdagangan yang bergantung pada *time-series*. Dengan struktur ini, dataset pelatihan yang dihasilkan oleh Feast terstandar dan dapat direproduksi di masa mendatang.

3. **Online Store dengan Redis Stack**

Redis Stack digunakan sebagai *online store* karena sifatnya yang *in-memory* dan sanggup memberikan latensi sangat rendah untuk permintaan *feature serving* (Redis Ltd. 2024). RedisSearch dengan indeks *hierarchical navigable small world* (HNSW) mendukung *vector similarity search*, sehingga fitur dalam bentuk *vector embeddings* dapat dilayani berdampingan dengan fitur tabular (Malkov dan Yashunin 2020). Pendekatan ini sejalan dengan kebutuhan modern terhadap pencarian vektor skala besar yang diulas Johnson, Douze, dan Jégou (2021). Dengan satu *online store*, komponen *inference* dapat mengambil beragam jenis fitur secara konsisten.

4. **API Penyajian Fitur Feast**

Feast menyediakan Python SDK untuk *offline retrieval* dan gRPC untuk *online serving*. Layanan *inference* cukup memanggil *Feature Service* dengan *entity key* dan mendapatkan fitur yang relevan tanpa perlu mengetahui detail penyimpanan di ClickHouse atau Redis. Penerapan API seperti ini menjaga *decoupling* antara komponen pemrosesan data dan komponen model. Selain itu, pola ini mempermudah pengujian karena definisi data yang digunakan model menjadi eksplisit.

5. **Integrasi dengan DataHub untuk Tata Kelola**

Metadata ingestion otomatis dari Feast ke DataHub menjaga daftar fitur, *owner*, dan *lineage* tetap mutakhir (DataHub Project 2024; Lamer dkk. 2024). Meteor dapat digunakan untuk mengumpulkan metadata tambahan dari berbagai sistem sehingga pandangan lintas komponen tetap konsisten. Dengan adanya katalog ini, DS dan tim lain dapat menemukan fitur yang sudah ada sebelum membuat yang baru. Hal ini mengurangi duplikasi dan membantu penerapan

praktik *governance* yang baik.

6. MinIO untuk Penyimpanan Objek

MinIO yang kompatibel S3 digunakan untuk menyimpan *training dataset*, artefak pelatihan, dan *snapshot offline store* (MinIO, Inc. 2024). Integrasi dengan Spark, Kubeflow, dan MLflow menjadi lebih sederhana karena banyak alat sudah mendukung API S3. MinIO juga memungkinkan sistem dipindah ke lingkungan lain yang memiliki *object storage* serupa tanpa perubahan signifikan. Dengan demikian, penyimpanan objek berfungsi sebagai lapisan bersama untuk artefak data dan model.

Melalui lapisan layanan fitur, organisasi memperoleh satu sumber kebenaran untuk definisi dan penyajian fitur. Penggunaan Feast dan *dual-store architecture* membantu menjaga konsistensi logika antara pelatihan dan *serving*. Integrasi dengan DataHub memastikan bahwa aspek tata kelola seperti dokumentasi, *ownership*, dan kualitas juga tercakup. Dengan adanya lapisan ini, tim dapat berkolaborasi membangun fitur yang dapat digunakan ulang secara lintas proyek, sehingga waktu pengembangan model baru dapat dipersingkat dan standar rekayasa fitur meningkat.

IV.2.5 Lapisan Layanan Model (*Model Service*)

Lapisan layanan model bertanggung jawab atas pengelolaan siklus hidup model, mulai dari pengembangan hingga *deployment* dan operasi di produksi. Lapisan ini menyatukan komponen-komponen yang mendukung eksperimen, pelacakan, *deployment*, dan *serving* model. Kubeflow mengelola *workflow* ML yang kompleks (Kubeflow Contributors 2024), MLflow digunakan untuk *experiment tracking* dan *model registry* (MLflow Contributors 2024), sementara KServe menyediakan *model serving* di atas Kubernetes (KServe Contributors 2024; Shankar dkk. 2022). Kombinasi ketiganya membentuk jalur yang jelas dari eksperimen menuju produksi, sehingga proses operasionalisasi model menjadi lebih sistematis dan terdokumentasi.

Lapisan ini juga dirancang untuk mendukung skenario *A/B testing*, *canary deployment*, dan *rollback* cepat ketika hasil model baru tidak memenuhi ekspektasi. Integrasi dengan GitOps memastikan bahwa perubahan konfigurasi *serving* selalu terkam di repositori versi. Dengan adanya *registry* yang terhubung ke metadata DataHub, organisasi dapat menelusuri model produksi hingga ke *training dataset* dan fitur yang digunakan. Lapisan layanan model dengan demikian menjadi pusat kendali atas kualitas dan evolusi model di lingkungan yang dinamis.

1. Kubeflow *Pipelines* untuk Orkestrasi ML

Kubeflow *Pipelines* digunakan untuk mengemas setiap langkah pelatihan ke dalam kontainer dan menjalankannya sebagai *pipeline* deklaratif. Langkah-langkah seperti ekstraksi data, pra-pemrosesan, pelatihan, evaluasi, dan pendaftaran model didefinisikan sebagai komponen terpisah. Kubeflow menyediakan UI untuk memantau *run*, melihat log, dan membandingkan hasil eksperimen. Integrasi dengan Feast dan MLflow memastikan bahwa data, konfigurasi, dan artefak model tercatat konsisten dan dapat ditelusuri kembali apabila terjadi masalah.

2. Pelacakan Eksperimen dengan MLflow

MLflow mencatat parameter, metrik, dan artefak dari tiap eksperimen secara sistematis (Pimentel dkk. 2019). Fitur *autologging* mengurangi beban konfigurasi dan membantu memastikan bahwa informasi penting tidak terlewat. Riwayat eksperimen ini menjadi dasar ketika memilih model terbaik untuk dipromosikan ke produksi. Selain itu, rekaman lengkap memudahkan tim melakukan audit teknis jika diperlukan penelusuran ulang.

3. Registry Model untuk Manajemen Siklus Hidup

Model registry MLflow mengelola versi dan tahap model, misalnya *Staging*, *Production*, dan *Archived*. Proses promosi model dapat diintegrasikan dengan GitOps sehingga perubahan konfigurasi *serving* terdokumentasi di repositori. Integrasi dengan DataHub melalui Meteor memungkinkan *lineage* dari model ke *training dataset* dan fitur dilihat secara grafis. Dengan cara ini, setiap model produksi memiliki jejak asal-usul yang jelas dan dapat dipertanggungjawabkan.

4. KServe untuk Penyajian Model

KServe menyediakan *inference service* yang berjalan secara *serverless* di atas Kubernetes, dengan dukungan *auto-scaling* dan *canary deployment* (KServe Contributors 2024; Shankar dkk. 2022). Konfigurasi *InferenceService* mendeskripsikan bagaimana model di-*deploy*, termasuk versi kontainer, sumber model, dan pengaturan skala. Kemampuan menjalankan beberapa versi model secara bersamaan memungkinkan *A/B testing* dan *canary rollout* yang aman. Log permintaan dan respons dapat dikirim ke Kafka dan kemudian disalurkan ke ClickHouse melalui Firehose untuk analitik pascadeploy.

5. Kubeflow Notebooks untuk Pengembangan Interaktif

Kubeflow Notebooks memberikan DS lingkungan interaktif yang berjalan di kluster yang sama dengan *pipeline* dan layanan produksi. Notebook dapat dikonfigurasi untuk menggunakan GPU atau sumber daya lain yang diperlukan.

Dari lingkungan ini, DS dapat mengakses Feast, MLflow, dan DataHub tanpa harus menyiapkan koneksi manual yang rumit. Hak akses ke data dan fitur tetap dikontrol oleh Frontier dan Guardian, sehingga penggunaan lingkungan interaktif tetap sejalan dengan kebijakan keamanan.

Lapisan layanan model memastikan bahwa transisi dari eksperimen ke produksi tidak lagi bergantung pada *manual rewrite* dan proses informal. Integrasi Kubeflow, MLflow, dan KServe membentuk jalur yang konsisten dari pelatihan hingga *serving*. Praktik ini meningkatkan *reproducibility* dan mengurangi risiko kesalahan konfigurasi yang sulit dideteksi. Di saat yang sama, lapisan ini menyediakan titik-titik kontrol yang jelas untuk audit dan *governance*, sehingga organisasi dapat mengoperasionalkan model dalam jumlah lebih besar tanpa mengorbankan kualitas dan keamanan.

IV.2.6 Lapisan *Observability* dan Pemantauan

Lapisan *observability* dan pemantauan menjadi prasyarat bagi operasi ML produksi yang aman dan andal. Lapisan ini menghubungkan metrik teknis, metrik data, dan metrik model dengan indikator bisnis sehingga tim dapat mengambil keputusan berdasarkan informasi yang konsisten. Prometheus dan Grafana menjadi inti untuk pengumpulan dan visualisasi metrik (Prometheus Authors 2024; Grafana Labs 2024a), sementara Siren dan AlertManager mengatur mekanisme *alerting*. Rancangan ini mengikuti panduan *observability* yang dikemukakan Cloud Native Computing Foundation (2024) dan IBM (2024) dan menambahkan fokus khusus pada metrik ML. Dengan dukungan observabilitas yang memadai, degradasi model dan masalah data dapat dideteksi lebih awal.

Lapisan ini juga mengintegrasikan *metrics*, *logs*, dan *traces* sehingga analisis akar masalah dapat dilakukan secara end-to-end. Pengukuran *drift*, kualitas data, dan performa model disandingkan dengan metrik infrastruktur untuk memisahkan sumber gangguan. Ketika terjadi penurunan kinerja strategi perdagangan, tim dapat menelusuri apakah penyebabnya berasal dari perubahan distribusi data, kesalahan konfigurasi model, atau gangguan infrastruktur. Dengan demikian, lapisan *observability* tidak hanya menangani pemantauan teknis, tetapi juga mendukung pengambilan keputusan operasional yang langsung terkait dengan hasil bisnis.

1. Pengumpulan Metrik Spesifik ML

Metrik yang dikumpulkan mencakup latensi dan *throughput* layanan fitur dan *inference*, statistik *drift* seperti PSI atau K-S, serta metrik kualitas data dari

proses validasi. Metrik tersebut melengkapi metrik infrastruktur seperti CPU, memori, dan penggunaan disk. Dengan menggabungkan keduanya, tim dapat membedakan apakah masalah berasal dari infrastruktur, data, atau model. Informasi ini penting untuk mempercepat proses *root cause analysis* ketika terjadi insiden.

2. **Dashboard dan Visualisasi Kustom**

Grafana digunakan untuk membangun *dashboard* tingkat platform, *dashboard* per komponen (misalnya untuk Feast, KServe, Kafka), dan *dashboard* per model. Tim teknis dan bisnis dapat mengakses tampilan yang sama, namun dengan fokus metrik yang berbeda sesuai peran. Misalnya, *dashboard* khusus performa strategi perdagangan dapat menampilkan kombinasi metrik model dan metrik finansial. Dengan cara ini, diskusi lintas fungsi menjadi lebih berbasis data dan tidak hanya mengandalkan persepsi subjektif.

3. **Peringatan dan Respons Insiden**

Aturan Prometheus didefinisikan untuk mendeteksi anomali, misalnya lonjakan latensi atau perubahan drastis distribusi prediksi. AlertManager menangani agregasi *alert* di tingkat infrastruktur dan mengatur pengelompokan *alert* ketika banyak peringatan muncul sekaligus. Siren berfungsi sebagai *alert router* terpusat yang mengirim notifikasi ke kanal seperti Slack atau email dengan templat yang seragam. *Runbook* operasional disusun agar respons terhadap *alert* mengikuti prosedur yang konsisten, sehingga MTTR dapat ditekan (Shankar dkk. 2022).

4. **Pelacakan dan Pencatatan Terdistribusi**

OpenTelemetry digunakan untuk mengumpulkan *tracing* end-to-end, sementara log terstruktur dikumpulkan ke sistem terpusat seperti Fluentd dan Loki. Kombinasi *metrics–logs–traces* memberikan pandangan yang kaya terhadap perilaku sistem dan mempermudah *root cause analysis*. Ketika terjadi regresi kinerja model atau anomali data, tim dapat melacak permintaan dari lapisan API hingga *pipeline* data di balik layar. Pendekatan ini mengurangi ketergantungan pada *debugging* manual yang memakan waktu.

Lapisan *observability* dan pemantauan memberikan kemampuan untuk memantau kesehatan sistem secara berkelanjutan dan mengambil tindakan korektif sebelum masalah berkembang terlalu jauh. Dengan memasukkan metrik ML sebagai bagian dari observabilitas, operasi model tidak lagi dipandang sebagai kotak hitam. Integrasi *alerting* dengan *runbook* membantu memastikan bahwa setiap insiden ditangani dengan prosedur yang terdokumentasi. Lapisan ini juga menyediakan data histo-

ris yang berharga untuk evaluasi jangka panjang terhadap strategi perdagangan dan keputusan desain sistem.

IV.2.7 Lapisan Tata Kelola Data

Lapisan tata kelola data (*Data Governance Layer*) memastikan bahwa aspek *compliance*, *discovery*, *lineage*, dan kualitas data serta model tertangani secara sistematis. Lapisan ini melengkapi fungsi teknis dengan konteks organisasi dan regulasi. DataHub dipilih sebagai platform metadata terpadu karena dukungannya terhadap *lineage*, *search*, dan integrasi dengan berbagai sistem data (DataHub Project 2024; Lamer dkk. 2024). Meteor digunakan sebagai pengumpul metadata, sedangkan Compass menyediakan antarmuka katalog yang lebih ringan untuk kebutuhan tertentu. Melalui lapisan ini, organisasi dapat memahami aset apa yang dimiliki, bagaimana data mengalir, dan siapa yang bertanggung jawab atas setiap aset.

Lapisan tata kelola data juga berperan sebagai sumber informasi bagi pengguna teknis dan non-teknis ketika mengambil keputusan terkait penggunaan data atau model. Dokumentasi kualitas, *ownership*, dan riwayat perubahan membantu mengurangi salah persepsi terhadap data yang tersedia. Integrasi dengan mekanisme akses Guardian memastikan bahwa permintaan akses selalu tercatat dan bisa diaudit. Dengan demikian, lapisan ini tidak hanya mendukung kepatuhan terhadap regulasi, tetapi juga meningkatkan efisiensi kolaborasi antar tim yang bekerja di atas platform.

1. *Ingestion* dan Manajemen Metadata

DataHub mengumpulkan metadata dari Feast, MLflow, Kubeflow, ClickHouse, Redis, Kafka, dan sistem lain melalui *ingestion pipeline*. Meteor bertindak sebagai *collector* yang mengekstrak metadata dari berbagai sumber kemudian menuliskannya ke DataHub atau Compass. Metadata yang dikelola mencakup *schema*, deskripsi, *owner*, *tag*, kualitas, dan statistik penggunaan. Dengan adanya repositori metadata terpusat, pencarian dan pemahaman aset menjadi lebih mudah.

2. Pelacakan *Lineage* dan Analisis Dampak

Lineage graph di DataHub merekam aliran data dan transformasi dari sumber ke konsumsi. Ketika terjadi perubahan pada tabel sumber atau *event stream*, tim dapat melihat model dan fitur mana yang terdampak. Fasilitas ini mempermudah analisis dampak dan perencanaan perubahan. Pada saat investigasi insiden, *lineage* membantu menelusuri penyebab masalah dengan lebih terarah.

3. Pencarian, Penemuan, dan Rekomendasi

DS dan BU dapat mencari aset melalui *full-text search* dan *filter* seperti tipe, pemilik, atau *tag*. Dokumentasi *data owner* memperjelas tanggung jawab dan meminimalkan duplikasi pekerjaan. Compass dapat digunakan sebagai antarmuka katalog yang lebih ringan untuk skenario tertentu, namun tetap berbagi metadata dengan DataHub melalui Meteor. Dengan demikian, proses penemuan aset data dan model menjadi lebih cepat dan terarah.

4. Integrasi Kualitas Data

Hasil validasi kualitas data dari Great Expectations dapat di-*ingest* sebagai metadata kualitas di DataHub. Pengguna data kemudian dapat melihat status kualitas terkini sebelum menggunakan dataset tersebut. Informasi ini membantu pengambilan keputusan yang lebih sadar risiko dan mendorong budaya transparansi kualitas data. Selain itu, laporan kualitas yang konsisten mempermudah diskusi ketika terjadi perbedaan hasil analisis antar tim.

Lapisan tata kelola data melengkapi arsitektur teknis dengan struktur pengelolaan informasi yang dibutuhkan organisasi dalam jangka panjang. Dengan metadata dan *lineage* yang lengkap, organisasi lebih siap menghadapi kebutuhan audit dan permintaan transparansi dari regulator. Di saat yang sama, lapisan ini mempermudah kolaborasi antar tim karena mereka merujuk pada katalog yang sama. Integrasi kualitas data ke dalam metadata membantu memastikan bahwa keputusan teknis dan bisnis mempertimbangkan aspek kualitas secara eksplisit.

IV.3 Alur Kerja *End-to-End*

Alur kerja *end-to-end* menjelaskan bagaimana seluruh komponen berinteraksi secara terkoordinasi untuk mendukung siklus hidup ML pada aplikasi perdagangan. Alur ini memperlihatkan jalur data dari saat masuk ke sistem hingga menghasilkan prediksi yang digunakan oleh pengguna bisnis. Penjelasan dibagi menjadi beberapa tahap, yaitu ingesti data dan materialisasi fitur, pelatihan model, *deployment* dan *serving*, deteksi *drift* dan *continuous training*, serta *governance* dan *audit workflow*. Dengan pembagian ini, keterkaitan antar bagian arsitektur dapat dilihat secara berurutan tanpa mengulang uraian teknis pada bagian sebelumnya. Alur ini sekaligus menjadi dasar bagi skenario evaluasi yang akan dibahas pada Bab V.

Penjelasan alur juga digunakan untuk menunjukkan bahwa komponen yang dipilih di Bab IV bukan hanya berdiri sendiri, tetapi terhubung dalam rangkaian *workflow* yang menyatu. Setiap tahap mengacu pada kebutuhan fungsional dan nonfungsio-

nal yang telah diturunkan dari Bab III. Dengan demikian, pembaca dapat menelusuri bagaimana kebutuhan tersebut dipenuhi secara operasional melalui aliran data dan kontrol yang jelas. Pendekatan ini diharapkan membuat desain arsitektur lebih konkret dan siap diterjemahkan menjadi rencana implementasi.

IV.3.1 *Data Ingestion hingga Feature Materialization*

Alur dimulai ketika data pasar masuk ke *topic* Kafka melalui *source connector* atau kolektor seperti Raccoon, dengan *schema* yang dikelola oleh Stencil. Pola ini menerapkan *event-driven architecture* dengan komunikasi asinkron dan *loose coupling* antara produser dan konsumen (Amazon Web Services 2024; Stopford 2018). Pesan yang masuk divalidasi terhadap *schema* dan aturan kualitas untuk mencegah data tidak valid mengalir ke hilir. Flink dan Dagger kemudian menghitung fitur *real-time* berbasis jendela waktu, misalnya rata-rata bergerak atau volatilitas jangka pendek, yang hasilnya dikirim ke Redis *online store* melalui Feast SDK sehingga fitur siap digunakan untuk *inference* berlatensi rendah.

Secara paralel, Firehose dan/atau Kafka Connect menyalurkan data mentah ke ClickHouse atau *data lake* untuk kebutuhan *batch processing*. Airflow menjadwalkan Spark dan Optimus untuk menghitung fitur historis dari ClickHouse dan MinIO, misalnya agregasi harian atau metrik berbasis jangka panjang. Hasil *batch processing* ini dimaterialisasikan sebagai *offline feature* di ClickHouse melalui Feast dan digunakan untuk pelatihan model. Metadata *pipeline* dan aset yang terbentuk di setiap langkah di-*ingest* ke DataHub melalui Meteor sehingga *discovery* dan *lineage* dapat dipetakan secara otomatis. Kombinasi jalur *stream* dan *batch* tersebut memastikan fitur selalu mutakhir sekaligus memiliki riwayat historis yang cukup.

IV.3.2 *Model Training Workflow*

Pada tahap pelatihan, DS terlebih dahulu menelusuri fitur yang tersedia di DataHub atau Compass untuk memahami definisi, kualitas, dan *lineage*-nya. Berdasarkan informasi tersebut, DS memilih fitur yang relevan dan menyusun *training dataset* melalui Feast. Feast melakukan *point-in-time correct join* untuk memastikan bahwa data yang digunakan pada setiap titik waktu hanya mencerminkan informasi yang tersedia pada saat itu, sehingga validitas temporal terjaga (Polyzotis dkk. 2018; Wang dan Ruf 2021). Dataset yang sudah siap kemudian diekspor ke MinIO atau *object storage* lain sebagai masukan proses pelatihan yang diorkestrasi Kubeflow.

DS kemudian mengorkestrasi pelatihan menggunakan Kubeflow *Pipelines* yang men-

cakup tahapan seperti pemuatan data, pra-pemrosesan, pelatihan, evaluasi, dan pendaftaran model ke MLflow. Selama proses ini, MLflow mencatat parameter, metrik, dan artefak secara otomatis sehingga hasil eksperimen terdokumentasi dengan baik. Akses ke dataset dan fitur dikontrol oleh Frontier dan Guardian untuk memastikan hanya pihak yang berwenang yang dapat menjalankan *pipeline* sensitif. Model yang memenuhi kriteria kualitas didaftarkan ke MLflow *registry* sebagai kandidat *deployment* dan dihubungkan dengan metadata yang tersimpan di DataHub, sehingga jalur dari data ke model dapat ditelusuri secara utuh.

IV.3.3 *Model Deployment dan Serving*

Ketika model siap dipromosikan ke *Production*, perubahan status di MLflow *registry* dapat memicu pembaruan *manifest* InferenceService di repositori Git. Proses ini dapat dilakukan melalui *webhook* atau mekanisme otomatis lain yang menghubungkan *registry* dengan Git. ArgoCD kemudian mendeteksi perubahan di repositori dan menyinkronkan konfigurasi tersebut ke kluster Kubernetes. KServe membuat atau memperbarui *inference service* sesuai definisi baru, termasuk mengatur *canary deployment* untuk membagi trafik antara model lama dan model baru (Fowler dan Lewis 2014; Newman 2015). Kinerja model baru dipantau melalui *observability stack*, dan trafik dinaikkan secara bertahap jika hasilnya stabil.

Layanan *inference* mengambil fitur *real-time* dari Feast *online store* menggunakan *Feature Service*. *API Gateway* bertindak sebagai titik masuk tunggal yang menangani autentikasi, *rate limiting*, dan *routing* permintaan berdasarkan *service* dan versi model. Pola ini selaras dengan prinsip *microservices* yang memisahkan *business logic* dari kekhawatiran lintas potong seperti keamanan dan pengaturan trafik (Fowler dan Lewis 2014; Newman 2015). Log permintaan dan respons dikirim ke Kafka dan kemudian disalurkan ke ClickHouse melalui Firehose untuk analitik pascadeploy, *debugging*, dan bahan evaluasi untuk iterasi model berikutnya.

IV.3.4 *Deteksi Drift dan Continuous Training*

Tahap berikutnya adalah deteksi *drift* dan *continuous training* (CT) sebagai mekanisme adaptasi terhadap perubahan distribusi data. *Pipeline* deteksi *drift* yang dijadwalkan secara berkala membandingkan distribusi fitur dan distribusi prediksi di produksi dengan statistik pelatihan yang tersimpan di MLflow. Berbagai metrik jarak seperti *Wasserstein distance*, KL divergence, atau uji Kolmogorov–Smirnov (K-S) digunakan untuk mengukur tingkat perbedaan distribusi Reis dkk. (2016). Jika nilai *drift* melewati ambang batas yang ditentukan, sistem memicu *alert* me-

lalui AlertManager dan Siren kepada tim yang bertanggung jawab. Dengan pola ini, penurunan kinerja model dapat dideteksi lebih awal sebelum berpengaruh besar terhadap indikator bisnis.

Continuous training (CT) didefinisikan sebagai proses *retraining* yang dikelola sebagai *pipeline as code*, menggunakan validasi ketat, metadata terpusat, dan *trigger* yang berasal dari jadwal atau metrik produksi (Liang dkk. 2024). *Pipeline* CT mengambil dataset terbaru dari Feast, menjalankan pelatihan ulang di Kubeflow, mencatat hasil ke MLflow, dan mendaftarkan model baru ke *registry*. Model hasil CT tidak otomatis menggantikan model produksi, tetapi terlebih dahulu dievaluasi dan dapat dipromosikan menggunakan mekanisme yang sama dengan model lain. Pendekatan ini membantu menjaga kinerja model pada lingkungan pasar yang sangat dinamis (Céspedes Sisniega dkk. 2024; Bayram, Ahmed, dan Kassler 2022; Vela dkk. 2022) tanpa harus mengorbankan kontrol dan transparansi.

IV.3.5 Governance dan Audit Workflow

Tahap terakhir dalam alur *end-to-end* adalah *governance* dan *audit workflow*. Metadata dan *lineage* diperbarui secara berkelanjutan melalui *ingestion* otomatis ke DataHub menggunakan Meteor. DS, MLE, dan BU dapat menelusuri aset data, fitur, dan model yang relevan melalui antarmuka DataHub atau Compass. Informasi seperti *owner*, kualitas data, dan riwayat perubahan membantu memperjelas konteks penggunaan aset tersebut. Dengan adanya katalog ini, diskusi lintas fungsi dapat merujuk ke entitas yang sama tanpa menimbulkan ambiguitas.

Alur audit memanfaatkan *lineage* DataHub untuk menelusuri satu prediksi kembali ke versi model, *training dataset*, fitur yang digunakan, hingga data mentah yang datang dari Kafka. *Audit trail* ini mendukung investigasi insiden, misalnya ketika terjadi anomali hasil prediksi, dan menjadi bukti bagi pemenuhan *compliance* (Stoyanovich, Howe, dan Jagadish 2020; Lamer dkk. 2024). Informasi akses dari Guardian melengkapi jejak audit dengan menunjukkan siapa yang memiliki hak akses terhadap aset tertentu pada saat prediksi dihasilkan. Desain arsitektur yang dijabarkan pada bab ini memberikan fondasi teknis yang kuat bagi implementasi platform DataOps–MLOps terintegrasi dan menjadi landasan langsung bagi tahapan implementasi pada Bab V.

BAB V

RENCANA SELANJUTNYA

V.1 Tahapan Implementasi

Implementasi arsitektur DataOps dan MLOps terintegrasi pada proposal tugas akhir ini direncanakan berlangsung secara bertahap dengan mengacu pada prinsip *incremental development*. Setiap tahap diusahakan menghasilkan komponen yang cukup utuh sehingga dapat diuji dan divalidasi sebelum menjadi dasar bagi tahap berikutnya. Pendekatan bertahap ini membuat dampak perubahan lebih mudah dikendalikan karena pengaruhnya terlebih dahulu diamati pada lingkup terbatas. Jain dan Das (2025) menunjukkan bahwa integrasi *data engineering* dan MLOps yang dibangun secara gradual cenderung menghasilkan *pipeline* ML yang lebih skalabel dan tahan gangguan, sehingga pola tersebut relevan dengan konteks tugas akhir ini. Dengan cara ini, risiko kegagalan implementasi dapat ditekan dan keputusan desain yang diambil di setiap fase dapat dievaluasi secara sistematis.

Setiap tahap implementasi disertai *deliverables* dan kriteria keberhasilan yang dapat diukur (*measurable success criteria*). Keberadaan kriteria yang eksplisit membantu penilaian kemajuan secara objektif, bukan hanya berdasarkan persepsi peneliti. Metodologi yang digunakan mengadopsi praktik *agile development* dengan siklus iterasi pendek untuk memfasilitasi umpan balik cepat dan penyesuaian prioritas bila diperlukan. Pola ini membuka ruang penyesuaian ruang lingkup ketika ditemukan kendala teknis atau perubahan kebutuhan, tanpa harus mengubah tujuan utama tugas akhir. Dengan demikian, arsitektur yang dibangun diharapkan stabil secara teknis, dapat direproduksi, dan siap dikembangkan menuju skenario produksi yang lebih luas pada tugas akhir lanjutan (Paleyes, Urma, dan Lawrence 2022).

V.1.1 Tahap 1: Persiapan Infrastruktur dan Platform Dasar

Tahap pertama berfokus pada penyediaan infrastruktur dasar yang menjadi pijakan bagi seluruh komponen arsitektur yang dirancang pada Bab IV. Tahap ini diawali dengan penyiapan kluster Kubernetes sebagai platform orkestrasi terpadu untuk menjalankan *workload* sistem. Kajian alternatif dilakukan antara layanan Kubernetes terkelola seperti Google Kubernetes Engine, Amazon Elastic Kubernetes Service, dan Azure Kubernetes Service, dibandingkan dengan opsi kluster yang dikelola sendiri menggunakan *kubeadm*, K3s, atau Minikube. Layanan terkelola memberikan ke-

nyamanan operasional dan dukungan *managed*, sedangkan opsi *self-managed* lebih fleksibel untuk eksperimen dan berpotensi lebih ekonomis untuk skenario *proof-of-concept*. Kombinasi kebutuhan eksperimen, tujuan pembelajaran teknis, dan batasan biaya menjadi dasar penentuan konfigurasi akhir klaster yang dipilih.

Arsitektur keandalan tinggi dirancang dengan mengadopsi pola *multi-availability zone* (Multi-AZ) untuk meningkatkan toleransi kesalahan dan ketersediaan layanan. Lakka (2025) menekankan bahwa distribusi *workload* dan redundansi pada beberapa zona merupakan praktik penting untuk sistem Kubernetes berskala *enterprise*. Dalam konteks proposal tugas akhir ini, konfigurasi minimum yang direncanakan terdiri atas tiga *node* dengan spesifikasi memadai untuk mendukung distribusi *workload* dan redundansi dasar. Setiap *node* ditempatkan pada zona ketersediaan yang berbeda sehingga klaster tetap dapat beroperasi ketika salah satu zona mengalami gangguan. Pendekatan ini membuat hasil pengujian lebih dapat diproyeksikan ke skenario produksi, meskipun implementasi dilakukan pada skala *proof-of-concept*.

Desain keamanan infrastruktur mengikuti prinsip *defense-in-depth* sebagaimana disarankan Ahmad dkk. (June 2024). Arsitektur jaringan memanfaatkan Virtual Private Cloud yang dibagi ke dalam beberapa *subnet* lintas zona ketersediaan untuk memisahkan lapisan aplikasi, data, dan akses publik. *Security group* dikonfigurasi sebagai *stateful firewall* pada tingkat instans untuk mengatur trafik masuk dan keluar, sedangkan *network access control list* memberikan lapisan proteksi tambahan pada tingkat *subnet*. Integrasi pengelola rahasia seperti *Secrets Manager* atau *Vault* digunakan untuk mengelola kredensial tanpa *hardcoding* di *manifest*. Pada lapisan aplikasi dan data, *API gateway* seperti Kong mengatur eksposur layanan, sementara Frontier dan Guardian diposisikan sebagai kandidat pusat autentikasi dan *data access governance* ketika platform berkembang.

Platform *GitOps* diterapkan menggunakan ArgoCD agar pengelolaan sumber daya Kubernetes bersifat deklaratif, terdokumentasi, dan mudah direproduksi. ArgoCD dikonfigurasi untuk memantau *repository* Git yang berisi *manifest* komponen dan melakukan sinkronisasi otomatis ke klaster, termasuk dukungan *rollback* bila konfigurasi gagal. Pendekatan deklaratif ini mengurangi risiko kesalahan konfigurasi manual dan meningkatkan konsistensi antar lingkungan. Temuan Polyzotis dkk. (2018) dan Paleyes, Urma, dan Lawrence (2022) menegaskan bahwa mekanisme serupa penting untuk menjaga kualitas dan *reproducibility* sistem ML yang kompleks dan sering berubah. Dengan fondasi ini, tahap-tahap berikutnya dapat berfokus pada logika data dan model tanpa terbebani masalah dasar lingkungan eksekusi.

Penyiapan komponen infrastruktur fundamental meliputi *container registry* untuk menyimpan dan melakukan *versioning image*, konfigurasi penyimpanan persisten melalui *storage class*, serta rancangan jaringan seperti *ingress controller* dan *network policy*. Praktik keamanan yang diadopsi mencakup penggunaan *private subnet* untuk komponen internal, penerapan prinsip *least privilege* pada *security group*, serta integrasi pengelolaan rahasia terpusat. Langkah ini konsisten dengan penekanan Ahmad dkk. (June 2024) mengenai pentingnya penguatan infrastruktur sebagai fondasi keberhasilan MLOps. Dengan infrastruktur yang stabil, *stack* di atasnya dapat diuji tanpa dibayangi risiko kegagalan lingkungan.

Penyiapan *stack observability* dasar dilakukan dengan tiga pilar utama, yaitu *logging*, *tracing*, dan *metrics*. Prometheus digunakan sebagai pengumpul metrik, Grafana dikonfigurasi dengan *dashboard* awal untuk memantau kesehatan kluster, dan AlertManager menangani pengiriman notifikasi insiden. Untuk *logging*, sistem seperti Loki digunakan sebagai *log aggregation layer* yang terintegrasi dengan Grafana sehingga log dan metrik dapat dianalisis pada satu antarmuka. OpenTelemetry diintegrasikan untuk *distributed tracing*, dengan Jaeger sebagai backend visualisasi jejak *request* lintas layanan. Pada tahap berikutnya, kerangka manajemen notifikasi seperti Siren dapat ditempatkan di atas AlertManager untuk mengonsolidasikan notifikasi ke berbagai kanal. Shankar dkk. (2022) menegaskan bahwa *observability* sejak awal membantu mempercepat *troubleshooting* dan meningkatkan kesiapan operasional.

Validasi tahap pertama dilakukan melalui pengujian fungsi dan keandalan infrastruktur. Aplikasi contoh di-*deploy* untuk memverifikasi bahwa *workload* berbasis kontainer dapat berjalan stabil di atas kluster. Pengujian penyimpanan persisten memastikan data tetap konsisten ketika *pod* berpindah *node* atau saat salah satu *node* gagal. Pengujian jaringan memverifikasi *routing*, *ingress*, dan kebijakan akses berjalan sesuai rancangan. Selain itu, alur GitOps diuji untuk memastikan perubahan di Git terefleksi di kluster melalui ArgoCD. Tahap ini dinyatakan berhasil apabila kluster Kubernetes berfungsi penuh, alur *deployment* berbasis GitOps stabil, dan cakupan *observability* memadai untuk kebutuhan operasi dasar.

V.1.2 Tahap 2: Implementasi Lapisan Ingesti dan Penyimpanan Data

Tahap kedua berfokus pada pembangunan infrastruktur ingest dan penyimpanan data yang menjadi dasar *pipeline* DataOps. Implementasi dimulai dengan penyiapan kluster Apache Kafka sebagai platform *event streaming* utama (Apache Software Foundation 2024c). Kafka berperan sebagai jalur transportasi data dari berbagai

sumber menuju komponen pemrosesan di hilir secara terpusat dan terukur. Untuk mengumpulkan *event* dari aplikasi dan layanan pendukung dengan *throughput* tinggi dan latensi rendah, komponen pengumpul seperti Raccoon dimanfaatkan sebagai *HTTP/gRPC event collector* yang menulis langsung ke topik Kafka. *Schema registry* seperti Stencil digunakan untuk mengelola *schema* pesan sehingga kontrak data antar komponen tetap konsisten ketika terjadi perubahan.

Sumber data *proof-of-concept* disiapkan melalui Kafka Connect dengan mengonfigurasi *connector* untuk mengingest data pasar historis publik dan *alternative data* seperti sentimen berita atau media sosial. Di sisi hilir, komponen *sink* generik seperti Firehose digunakan untuk mengalirkan data dari Kafka menuju berbagai target penyimpanan, misalnya ClickHouse, PostgreSQL, atau MinIO, secara deklaratif sehingga kode konsumsi data tidak perlu ditulis ulang saat tujuan baru ditambahkan. Rodrigues dkk. (2025) mengusulkan arsitektur MLOps *near real-time* yang memadukan *inference*, pembaruan model, *versioning*, dan *monitoring*. Dengan mengacu pada pendekatan tersebut, aliran data pada tugas akhir ini dirancang untuk diproses kontinu dan siap mendukung pembaruan model yang cepat ketika kondisi pasar berubah.

Arsitektur *streaming* menerapkan *real-time ingestion pipeline* berbasis Kafka dan komponen pemrosesan seperti Apache Flink. Flink dimanfaatkan untuk *stream processing*, termasuk *windowed aggregations* dan transformasi yang diperlukan untuk konstruksi fitur *real-time*. Pada skenario tertentu, kerangka konfigurasi seperti Dagger dapat digunakan untuk menyusun *job* Flink berbasis konfigurasi sehingga perubahan logika pemrosesan tidak selalu memerlukan perubahan kode program. Di sisi *batch*, aliran data yang sama disalurkan ke penyimpanan untuk mendukung pemrosesan historis dan pelatihan model, sehingga jalur data *online* dan *offline* tetap konsisten.

Lapisan penyimpanan dibangun secara bertahap agar setiap komponen dapat diuji baik secara mandiri maupun terintegrasi. ClickHouse dipilih sebagai *offline store* untuk *feature store* karena kemampuannya menangani kueri analitik kolumnar. Redis Stack dimanfaatkan sebagai *online store* dengan latensi rendah dan dukungan *vector similarity search* yang relevan untuk fitur embedding. MinIO berperan sebagai penyimpanan objek kompatibel S3 untuk menyimpan artefak model, data pelatihan, dan *log*. Kombinasi ini membentuk fondasi penyimpanan yang fleksibel dan konsisten dengan desain arsitektur pada Bab IV.

Validasi lapisan ingesti dan penyimpanan meliputi pengujian Kafka *end-to-end* un-

tuk memastikan tidak terjadi kehilangan pesan dalam skenario normal maupun saat gangguan terbatas. Kinerja ClickHouse diuji menggunakan *dataset* sampel untuk menilai latensi dan kemampuan pemrosesan kueri analitik. Latensi Redis diuji untuk memastikan target internal pengambilan fitur *online* tercapai, sedangkan operasi unggah dan unduh ke MinIO diuji guna menjamin integritas serta ketersediaan artefak. Tahap ini dianggap berhasil bila alur ingesti berjalan stabil, target latensi *online* terpenuhi, dan kueri *offline* mencapai standar minimal yang ditetapkan.

V.1.3 Tahap 3: Implementasi Lapisan Layanan Fitur dengan Feast

Tahap ketiga diarahkan pada penerapan Feast sebagai platform manajemen fitur terpusat (Feast Project 2024). Registri Feast disusun berbasis Git sehingga definisi fitur dikelola sebagai kode dan memperoleh keuntungan *version control*. Definisi fitur disusun untuk skenario perdagangan sampel dan mencakup fitur tabular seperti harga, volume, dan indikator teknikal. Selain itu, dikembangkan fitur *vector embedding* yang merepresentasikan sentimen berita dengan mengacu pada pendekatan pemodelan bahasa modern (Devlin dkk. 2019; Munappy dkk. 2022). Dengan demikian, lapisan fitur dapat mengakomodasi kombinasi sinyal numerik dan tekstual dalam satu kerangka yang konsisten.

Proposal tugas akhir ini menekankan demonstrasi kapabilitas *point-in-time correctness* untuk mencegah kebocoran temporal pada proses pelatihan dan evaluasi. Mekanisme tersebut memastikan bahwa setiap contoh data pada *training dataset* hanya memanfaatkan informasi yang tersedia hingga waktu referensi tertentu, sesuai kondisi yang akan ditemui model saat produksi. Polyzotis dkk. (2018) menyoroti bahwa kebocoran temporal dapat menyebabkan evaluasi *offline* terlihat terlalu optimistis dan tidak mencerminkan kinerja aktual di produksi. Oleh karena itu, fitur *point-in-time correctness* diimplementasikan dan diuji secara sistematis agar manfaatnya terhadap reliabilitas evaluasi dapat diamati secara eksplisit dan dikaitkan dengan kebutuhan domain perdagangan.

Feast *serving* dikonfigurasi untuk menyediakan API gRPC yang melayani pengambilan fitur dari *online store* maupun *offline store*. Feast dihubungkan dengan ClickHouse sebagai *offline store* dan Redis sebagai *online store* sehingga satu definisi fitur dapat dimanfaatkan di dua konteks berbeda tanpa menduplikasi logika. *Pipeline* materialisasi dibangun untuk mendemonstrasikan *offline-to-online materialization* bagi fitur *batch* dan pola *stream-to-online* bagi fitur *real-time* yang dihitung oleh Flink. Pendekatan hibrid ini meniru praktik di lingkungan produksi yang menggabungkan data historis dan sinyal *real-time* dalam satu kerangka fitur yang konsisten

dengan arsitektur Bab IV.

Pengujian integrasi Feast mencakup pembuatan *training dataset* menggunakan *point-in-time join* untuk memverifikasi validitas temporal dan mencegah *data leakage*. Selain itu, dilakukan pengukuran latensi pengambilan fitur *online* untuk memastikan permintaan dari layanan *inference* dilayani dalam batas yang wajar. Proses materialisasi diperiksa untuk memastikan konsistensi antara sumber data, ClickHouse, dan Redis, baik dari sisi isi maupun waktu pembaruan. Tahap ini dinyatakan berhasil apabila *point-in-time correctness* terbukti berjalan dengan benar, latensi *online* memenuhi target internal, dan proses materialisasi berjalan stabil tanpa indikasi inkonsistensi data.

V.1.4 Tahap 4: Implementasi Lapisan Layanan Model dan Siklus Hidup

Tahap keempat berfokus pada penerapan komponen yang mendukung pengembangan, pelatihan, dan pengelolaan siklus hidup model. Instalasi Kubeflow dilakukan dengan mengaktifkan komponen utama seperti Kubeflow *Pipelines*, Kubeflow *Notebooks*, dan *training operators* (Kubeflow Contributors 2024). Kubeflow dipilih karena bersifat *Kubernetes-native* sehingga mendukung portabilitas, skalabilitas, dan integrasi erat dengan infrastruktur yang telah disiapkan pada tahap pertama. Konfigurasi ini mengurangi perbedaan antara lingkungan eksperimen dan lingkungan operasi, serta menjembatani praktik pengembangan dan produksi (Kreuzberger, Köhl, dan Hirschl 2023). Dengan demikian, alur kerja *data scientist* dan *ML engineer* menjadi lebih dekat dengan arsitektur operasional yang dirancang di Bab IV.

Kubeflow *Pipelines* memanfaatkan Argo Workflows sebagai *execution engine*, basis data relasional sebagai penyimpan metadata, dan MinIO sebagai penyimpan artefak. *Pipeline* contoh dibangun untuk menunjukkan alur ML *end-to-end*, dimulai dari pengambilan fitur melalui Feast, proses pelatihan, evaluasi, hingga pendaftaran model ke MLflow *Model Registry* (MLflow Contributors 2024). Dengan alur ini, setiap langkah proses terekam dan dapat diulang apabila diperlukan, baik untuk keperluan reproduksi eksperimen maupun penyesuaian konfigurasi model. Pendekatan ini sekaligus memperlihatkan penerapan konsep *pipeline as code* pada siklus hidup model yang akan dinilai pada tahap evaluasi.

MLflow diterapkan melalui *tracking server* untuk pencatatan eksperimen dan *model registry* untuk mengelola status model, seperti *None*, *Staging*, *Production*, dan *Archived*. Alur pelatihan sampel menggunakan *dataset* publik terkait perdagangan, misalnya harga saham atau data kripto dari sumber terbuka, sehingga contoh yang

digunakan memiliki konteks realistis. Setiap eksperimen mencatat parameter, metrik, dan artefak agar perbandingan performa antar model menjadi terstruktur dan transparan. Munappy dkk. (2022) menegaskan bahwa *experiment tracking* yang konsisten merupakan prasyarat *reproducibility* pada pengembangan model produksi, sehingga implementasi MLflow di tahap ini langsung dikaitkan dengan kebutuhan tersebut.

Validasi lapisan layanan model dilakukan melalui uji fungsional *pipeline end-to-end*, verifikasi pencatatan eksperimen di MLflow, dan pengujian perpindahan model antar status di *model registry* sesuai aturan yang ditetapkan. Pengalaman pengguna juga diuji melalui sesi pengembangan interaktif menggunakan Kubeflow *Notebooks* untuk menilai kemudahan penggunaan dan integrasi dengan komponen lain. Tahap ini dianggap berhasil apabila eksekusi *pipeline* berjalan stabil dalam waktu yang wajar, pencatatan eksperimen dan pengelolaan versi model lengkap, serta lingkungan terintegrasi dinilai praktis digunakan untuk iterasi pengembangan model pada domain perdagangan.

V.1.5 Tahap 5: Implementasi Penyajian Model dan Pola *Deployment*

Tahap kelima memusatkan kegiatan pada penyajian model di lingkungan produksi menggunakan KServe (KServe Contributors 2024). Instalasi KServe dilakukan pada kluster Kubernetes dan memanfaatkan Knative *Serving* sebagai dependensi untuk mendukung arsitektur *serverless* dan mekanisme penskalaan otomatis. Adusumilli (2025) membahas pergeseran orkestrasi kontainer menuju pola *serverless* di atas Kubernetes dengan dukungan *scale-to-zero* dan *auto-scaling*. Shankar dkk. (2022) menekankan pentingnya otomatisasi *serving*, *staging*, dan *rollout* bertahap untuk menurunkan risiko saat model dibawa ke produksi, sehingga kemampuan ini menjadi fokus pada tahap implementasi.

Model sampel dari tahap sebelumnya digunakan untuk mendemonstrasikan rangkaian *deployment*. Sumber daya *InferenceService* didefinisikan untuk mengacu pada artefak model di MLflow *Model Registry*, memilih *runtime* penyajian yang sesuai, serta menetapkan pengaturan *autoscaling* berbasis metrik seperti *request rate* atau latensi. Integrasi Feast ditunjukkan melalui *transformer* kustom yang mengambil fitur *online* dari Redis sebelum proses inferensi, sehingga logika fitur tidak digandakan di dalam layanan. Dengan pola tersebut, risiko *training-serving skew* dapat ditekan karena definisi fitur pada pelatihan dan produksi berasal dari sumber yang sama (Munappy dkk. 2022).

Pola *deployment* tingkat lanjut diimplementasikan untuk mendemonstrasikan kemampuan MLOps Level 2. *Canary deployment* digunakan dengan membagi trafik secara bertahap dari model lama ke model baru sehingga risiko terhadap pengguna dapat dikendalikan bila terjadi regresi performa. *A/B testing* dijalankan dengan mengeksekusi dua versi model secara paralel untuk membandingkan kinerja berdasarkan metrik yang relevan. Selain itu, *feature toggle* digunakan untuk mengaktifkan atau menonaktifkan fitur tertentu secara fleksibel dan mendukung *rollback* cepat bila diperlukan. Perilaku *autoscaling* divalidasi melalui uji beban yang mensimulasikan variasi trafik agar respons sistem terhadap lonjakan permintaan dapat diamati secara terukur.

Pengujian lapisan penyajian mencakup validasi fungsional API inferensi, uji integrasi Feast untuk memastikan konsistensi fitur *online*, serta pengukuran latensi dan *throughput*. Simulasi skenario *canary* dengan *rollback* otomatis dilakukan untuk menilai ketahanan alur rilis ketika model baru tidak memenuhi ekspektasi. Tahap ini dinyatakan berhasil apabila latensi prediksi berada dalam kisaran yang ditargetkan, integrasi fitur tidak mengakibatkan *skew* signifikan, dan pola *deployment* dapat dijalankan dengan pemantauan metrik yang memadai.

V.1.6 Tahap 6: Implementasi Pemrosesan Data dan Deteksi *Drift*

Tahap keenam memperluas infrastruktur dengan kemampuan pemrosesan data historis dan pemantauan *drift* secara otomatis. Apache Spark diterapkan untuk pemrosesan *batch* dengan memanfaatkan Spark *operator* di atas Kubernetes (Apache Software Foundation 2024d). *Job* Spark sampel dikembangkan untuk menghitung fitur kompleks dalam skala besar, misalnya agregasi harian, indikator teknis, atau *feature engineering* khusus *time-series* (Zaharia dkk. 2016). Orkestrasi *job* dilakukan oleh Apache Airflow, sementara Optimus digunakan untuk mengelola transformasi data terstruktur, dependensi, dan *data quality check* secara deklaratif. Kerangka validasi data seperti Great Expectations dimanfaatkan untuk mendefinisikan dan menegakkan *expectation* terhadap kualitas data sebelum data dipakai dalam pelatihan atau inferensi.

Pipeline deteksi *drift* dikembangkan untuk memantau *data drift* dan *concept drift* secara berkala. Lu dkk. (2019) menyajikan kerangka komprehensif mengenai deteksi dan adaptasi terhadap perubahan distribusi data. Dalam proposal tugas akhir ini, deteksi *drift* diimplementasikan sebagai Kubeflow *pipeline* terjadwal yang mengambil prediksi produksi beserta fitur terkait, kemudian membandingkannya dengan statistik pelatihan historis. Uji statistik seperti Population Stability Index (PSI) un-

tuk fitur kategorikal dan Kolmogorov–Smirnov untuk fitur kontinu digunakan untuk mengukur tingkat *drift* (Bayram, Ahmed, dan Kassler 2022; Céspedes Sisniega dkk. 2024). Dengan cara ini, degradasi performa model dapat diidentifikasi secara kuantitatif, bukan hanya berdasarkan keluhan pengguna.

Jika *drift* signifikan terdeteksi dan nilai uji melampaui ambang batas yang ditetapkan, AlertManager memicu peringatan dan panel Grafana menampilkan perubahan tersebut pada *dashboard*. Pada tahap lanjutan, lapisan orkestrasi peringatan seperti Siren dapat digunakan untuk mengalirkan notifikasi ke berbagai kanal komunikasi. Integrasi deteksi *drift* dengan *continuous training* ditunjukkan dengan konfigurasi *pipeline* yang secara opsional melakukan pelatihan ulang otomatis ketika pola *drift* tertentu teridentifikasi. Dengan cara ini, kinerja model dapat dipulihkan lebih cepat sehingga dampak degradasi terhadap indikator bisnis dapat diminimalkan dan hasilnya dapat dievaluasi secara sistematis pada tahap pengujian platform.

Validasi tahap ini mencakup pengujian eksekusi *job* Spark untuk memastikan pemrosesan *batch* berjalan sesuai jadwal, pengujian deteksi *drift* menggunakan skenario sintesis untuk menilai sensitivitas dan stabilitas, serta pengujian pemicu *continuous training*. Tahap ini dianggap berhasil apabila pemrosesan *batch* dan *stream* terintegrasi, deteksi *drift* dapat disetel sensitivitasnya, dan *proof-of-concept retraining* otomatis berjalan sesuai rancangan tanpa mengganggu layanan produksi.

V.1.7 Tahap 7: Implementasi Tata Kelola Data dan Integrasi Final

Tahap ketujuh melengkapi platform dengan lapisan tata kelola data serta integrasi menyeluruh seluruh komponen. DataHub diterapkan untuk manajemen metadata dan pelacakan *lineage* (DataHub Project 2024). Konfigurasi *metadata ingestion* disiapkan agar metadata dari komponen platform seperti Feast, MLflow, Kubeflow, ClickHouse, Redis, dan Kafka dapat dikumpulkan secara otomatis. Untuk menyederhanakan proses integrasi, alat kolektor metadata seperti Meteor dimanfaatkan untuk mengekstrak metadata dari berbagai sumber dan mengirimkannya ke DataHub. Pada skenario yang memerlukan katalog lebih ringan, Compass dapat dipertimbangkan sebagai lapisan *data discovery* tambahan yang memanfaatkan metadata yang sama. Lamer (2024) menekankan bahwa metadata memiliki peran sentral dalam sistem DataOps modern karena menjadi penghubung antar komponen.

Dengan integrasi DataHub dan alat kolektor metadata, pengguna dapat melakukan *discovery* fitur dan model melalui antarmuka pencarian, menelusuri *lineage* dari data mentah hingga prediksi produksi, dan mengakses dokumentasi yang memuat kon-

teks teknis dan bisnis. Fasilitas ini mendukung kebutuhan *compliance* dan *debugging* karena alur data dan model dapat dilacak kembali secara sistematis. Stoyanovich dkk. (2020) menegaskan bahwa *responsible data management* mensyaratkan akuntabilitas, transparansi, dan kepatuhan regulasi, sehingga lapisan tata kelola ini menjadi jembatan antara aspek teknis dan mandat organisasi. Dalam konteks proposal tugas akhir, lapisan ini juga memperjelas bagaimana artefak yang dibangun dapat diaudit dan disampaikan kepada pemangku kepentingan.

Pengujian integrasi menyeluruh dilakukan untuk memvalidasi alur kerja *end-to-end*, mulai dari ingesti data pasar, perhitungan fitur, pelatihan model, penyajian di produksi, inferensi dengan fitur *online*, deteksi *drift*, hingga pelatihan ulang otomatis. Pengujian dirancang untuk mensimulasikan skenario realistis di domain perdagangan sehingga konsistensi dan kolaborasi antar komponen dapat dinilai secara objektif. *User acceptance testing* melibatkan pengguna representatif, idealnya *data scientist* atau *ML engineer* yang memahami domain perdagangan, untuk menilai kemudahan penggunaan, kelengkapan fungsi, dan kualitas dokumentasi. Rella dkk. (2022) menunjukkan bahwa integrasi DataOps dan MLOps yang baik dapat meningkatkan efisiensi operasional melalui *continuous integration/continuous delivery* (CI/CD) yang cepat dan kolaborasi lintas tim yang lebih efektif.

Kriteria keberhasilan akhir mencakup demonstrasi alur kerja *end-to-end* yang berjalan konsisten, metadata dan *lineage* yang komprehensif di DataHub, terpenuhinya kebutuhan fungsional (KF-01 hingga KF-14), serta diperolehnya umpan balik pengguna yang positif terhadap platform. Luaran utama tahap ini adalah platform DataOps dan MLOps terintegrasi yang dapat didemonstrasikan, diuji, dan dievaluasi secara objektif berdasarkan indikator keberhasilan yang telah dirumuskan pada bab sebelumnya. Platform ini juga diharapkan menjadi dasar yang kuat bagi tugas akhir lanjutan maupun pengembangan di lingkungan produksi.

V.2 Jadwal Pengerjaan

Jadwal pengerjaan disusun untuk memastikan seluruh tahapan implementasi dapat dilaksanakan secara sistematis dan terukur. Total durasi tugas akhir direncanakan selama delapan bulan, dari Oktober 2025 hingga Mei 2026, dan dibagi ke dalam beberapa fase. Fase 1 berfokus pada persiapan dan analisis yang memperkuat landasan teoritis, pemahaman masalah, serta perancangan arsitektur pada bab-bab sebelumnya. Fase 2 hingga Fase 6 disejajarkan dengan tahapan implementasi teknis pada bagian sebelumnya, mulai dari penyiapan infrastruktur, layanan fitur, siklus hidup

model, penyajian dan fitur lanjutan, hingga integrasi dan pengujian menyeluruh. Fase 7 diarahkan khusus untuk dokumentasi, evaluasi, serta finalisasi tesis dan persiapan presentasi sehingga seluruh hasil implementasi terdokumentasi secara utuh. Dengan pembagian fase yang jelas, peneliti dapat memonitor kemajuan dan menyesuaikan rencana ketika terjadi perubahan kondisi. Jadwal tersebut juga membantu mencegah penumpukan beban kerja di akhir periode tugas akhir.

Paley, Urma, dan Lawrence (2022) menekankan bahwa perencanaan terstruktur dengan *milestone* yang jelas penting untuk menurunkan risiko keterlambatan pada proyek ML yang kompleks dan melibatkan banyak komponen. Dengan jadwal yang dirumuskan secara eksplisit, pemantauan kemajuan dapat dilakukan sejak dini dan memberikan sinyal ketika terjadi deviasi dari rencana. Penyesuaian prioritas dan ruang lingkup dapat dilakukan lebih cepat tanpa mengganggu tujuan akhir tugas akhir. Selain itu, jadwal yang terdokumentasi memudahkan pembimbing maupun pihak lain meninjau realisme rencana kerja dan memberikan masukan yang konstruktif.

V.2.1 Tinjauan Linimasa

Tabel V.1 menyajikan ringkasan linimasa pengerjaan beserta alokasi waktu untuk setiap fase utama. Pembagian waktu mempertimbangkan tingkat kompleksitas relatif tiap fase, dependensi antar fase yang memerlukan urutan tertentu, serta kebutuhan waktu untuk validasi dan dokumentasi. Fase 1 mengakomodasi aktivitas persiapan seperti kajian pustaka, identifikasi masalah, perancangan arsitektur, dan penyusunan proposal. Fase 2 hingga Fase 6 mengakomodasi implementasi dan pengujian tahapan teknis yang telah diuraikan, sedangkan Fase 7 memusatkan kegiatan pada dokumentasi tesis dan finalisasi. Fase yang berisi integrasi banyak komponen memperoleh alokasi waktu lebih panjang dibanding fase dengan lingkup lebih sempit. Jadwal juga disusun dengan sedikit kelonggaran sehingga masih dapat mengakomodasi dinamika implementasi tanpa harus mengubah target utama.

Pendekatan penjadwalan ini sejalan dengan prinsip *agile* yang umum digunakan dalam proyek rekayasa kompleks, yaitu menyediakan kerangka terstruktur sambil tetap membuka ruang penyesuaian (Kreuzberger, Köhl, dan Hirschl 2023). Dengan linimasa yang jelas, pengendalian ruang lingkup dan prioritas aktivitas dapat dilakukan secara lebih disiplin hingga akhir tugas akhir. Selain itu, linimasa yang terdokumentasi mempermudah komunikasi status proyek kepada pembimbing dan pemangku kepentingan lain.

Tabel V.1 Linimasa Pengerjaan Penelitian

Fase	Periode	Durasi	Luaran Utama
Fase 1: Persiapan dan Analisis	Oktober - November 2025	2 bulan	Dokumen kebutuhan, desain infrastruktur, dan studi literatur yang terstruktur
Fase 2: Implementasi Infrastruktur	Desember 2025	1 bulan	Klaster Kubernetes, lapisan penyimpanan, dan penyiapan <i>GitOps</i>
Fase 3: Implementasi Layanan Fitur	Januari 2026	1 bulan	Penerapan Feast, fitur sampel, dan <i>pipeline</i> materialisasi awal
Fase 4: Implementasi Siklus Hidup Model	Februari 2026	1 bulan	Penerapan Kubeflow, penyiapan MLflow, dan alur kerja pelatihan model
Fase 5: Penyajian & Fitur Lanjutan	Maret 2026	1 bulan	Penerapan KServe, deteksi <i>drift</i> , serta pemrosesan data lanjutan
Fase 6: Integrasi dan Pengujian	April 2026	1 bulan	Penerapan DataHub, integrasi <i>end-to-end</i> , dan pengujian dengan pengguna
Fase 7: Dokumentasi dan Finalisasi	Mei 2026	1 bulan	Penyusunan tesis, persiapan presentasi, dan penyelesaian revisi akhir

V.2.2 Rincian Jadwal Detil

Rincian jadwal per bulan memberikan gambaran lebih spesifik mengenai kegiatan yang akan dilakukan dan waktu pelaksanaannya. Pengaturan jadwal hingga tingkat mingguan memungkinkan pemantauan kemajuan yang lebih dekat dan memudahkan penyesuaian ketika terjadi perubahan prioritas. Struktur tersebut dijaga cukup fleksibel untuk mengakomodasi dinamika implementasi, namun tetap cukup ketat agar penundaan tidak dibiarkan meluas. Dengan adanya rincian ini, pekerjaan besar dapat dipecah menjadi langkah-langkah yang lebih kecil dan mudah dikelola.

Pendekatan tersebut membantu mengurangi risiko penumpukan tugas di akhir periode dan membuat beban kerja terdistribusi lebih merata sepanjang durasi tugas akhir. Peneliti juga dapat mengidentifikasi lebih awal bila ada minggu tertentu yang berpotensi terlalu padat dan melakukan penyesuaian sebelum hal tersebut mengganggu fase lain. Rincian pada Tabel V.2 hingga Tabel V.5 menyajikan aktivitas per minggu selama periode tugas akhir, sehingga hubungan antara tahapan implementasi dan jadwal pelaksanaannya terlihat secara eksplisit.

Tabel V.2 Jadwal Detil Oktober - November 2025

Kegiatan	Oktober 2025				November 2025			
	M1	M2	M3	M4	M1	M2	M3	M4
Kajian pustaka dan analisis makalah	✓	✓						
Identifikasi dan analisis masalah	✓	✓	✓					
Pengumpulan dan dokumentasi kebutuhan		✓	✓	✓				
Perancangan arsitektur dan pemilihan teknologi			✓	✓	✓			
Perencanaan infrastruktur dan pengadaan sumber daya				✓	✓	✓		
Penulisan dan penyempurnaan proposal					✓	✓	✓	✓
Pengajuan dan presentasi proposal								✓

Tabel V.3 Jadwal Detil Desember 2025 - Januari 2026

Kegiatan	Desember 2025				Januari 2026			
	M1	M2	M3	M4	M1	M2	M3	M4
Instalasi dan konfigurasi kluster Kubernetes	✓	✓						
Penerapan lapisan penyimpanan (MinIO, ClickHouse, Redis)		✓	✓					
Pengaturan infrastruktur <i>GitOps</i> (ArgoCD)		✓	✓					
Penerapan <i>stack</i> pemantauan (Prometheus, Grafana)			✓	✓				
Dokumentasi pengaturan infrastruktur			✓	✓				
Pengujian dan validasi dasar infrastruktur				✓				
Instalasi dan konfigurasi Feast					✓			
Pengaturan registri fitur dan definisi sampel					✓	✓		
Penerapan <i>serving</i> Feast						✓		
Implementasi <i>pipeline</i> materialisasi						✓	✓	
Pengujian integrasi dengan lapisan penyimpanan							✓	✓
Pengujian performa dan optimisasi							✓	✓
Dokumentasi dan contoh penggunaan								✓

Tabel V.4 Jadwal Detil Februari - Maret 2026

Kegiatan	Februari 2026				Maret 2026			
	M1	M2	M3	M4	M1	M2	M3	M4
Instalasi dan konfigurasi Kubeflow	✓							
Penerapan dan pengaturan MLflow	✓	✓						
Pengaturan lingkungan <i>notebook</i>		✓						
Pengembangan <i>pipeline</i> sampel		✓	✓					
Implementasi alur kerja pelatihan			✓	✓				
Dokumentasi dan contoh alur kerja			✓	✓				
Pengujian integrasi dengan Feast				✓				
Instalasi dan konfigurasi KServe					✓			
Contoh penerapan model					✓	✓		
Integrasi fitur dalam proses <i>serving</i>						✓		
Penerapan Spark untuk pemrosesan <i>batch</i>						✓	✓	
Pola penerapan lanjutan (<i>canary, A/B</i>)							✓	
Implementasi <i>pipeline</i> deteksi <i>drift</i>							✓	✓
Pengujian dan validasi								✓

Tabel V.5 Jadwal Detil April - Mei 2026

Kegiatan	April 2026				Mei 2026			
	M1	M2	M3	M4	M1	M2	M3	M4
Instalasi dan konfigurasi DataHub	✓							
Pengaturan <i>ingestion</i> metadata	✓	✓						
Pengujian dan validasi <i>lineage</i> data		✓						
Pengujian integrasi menyeluruh (<i>end-to-end</i>)		✓	✓					
<i>Benchmarking</i> performa			✓					
Uji penerimaan pengguna (UAT)			✓	✓				
Perbaikan <i>bug</i> dan penyempurnaan				✓				
Pengujian dan validasi akhir					✓			
Penulisan tesis - Bab Implementasi					✓	✓		
Penulisan tesis - Bab Evaluasi					✓	✓	✓	
Penulisan tesis - Bab Kesimpulan						✓	✓	
Persiapan materi presentasi							✓	✓
Peninjauan akhir dan revisi							✓	✓
Pengajuan dan presentasi tesis								✓

V.2.3 Analisis Jalur Kritis

Analisis jalur kritis dilakukan untuk mengidentifikasi rangkaian kegiatan yang harus diselesaikan tepat waktu agar tugas akhir selesai sesuai jadwal. Kegiatan-kegiatan pada jalur ini memiliki tingkat dependensi tinggi dan keterlambatan pada salah satunya dapat menggeser keseluruhan linimasa. Dengan memahami jalur kritis sejak

awal, peneliti dapat memfokuskan pengelolaan risiko dan alokasi sumber daya pada bagian yang paling sensitif terhadap keterlambatan. Pendekatan ini selaras dengan rekomendasi Paleyes, Urma, dan Lawrence (2022) mengenai pentingnya manajemen risiko disiplin pada proyek ML berskala besar.

Dependensi utama yang memengaruhi jalur kritis dapat diringkas sebagai berikut. Pertama, penyiapan kluster Kubernetes menjadi prasyarat penerapan komponen berikutnya karena seluruh layanan utama berjalan di atas kluster ini. Kedua, penerapan lapisan penyimpanan (ClickHouse, Redis, MinIO) harus selesai sebelum Feast dapat diuji secara menyeluruh. Ketiga, penerapan Feast menjadi prasyarat alur pelatihan model *end-to-end* karena model membutuhkan fitur yang dikelola melalui *feature store*. Keempat, penyajian model dengan KServe baru dapat dilakukan setelah model terlatih tersedia di MLflow *Model Registry*. Kelima, integrasi DataHub ditempatkan di tahap akhir karena memerlukan metadata dari seluruh komponen utama sebagai input. Komponen tambahan seperti Raccoon, Stencil, Firehose, Optimus, Meteor, dan Siren terhubung ke jalur kritis melalui ketergantungan pada kluster inti, namun cakupan fungsinya dapat dikurangi jika jadwal tertekan tanpa mengganggu inti *pipeline*.

Waktu penyangga eksplisit disisipkan pada jadwal untuk mengantisipasi masalah tak terduga dan memberi ruang cukup bagi pengujian serta validasi pada setiap fase. Penyangga ini mencegah efek domino jika terjadi keterlambatan kecil sehingga jadwal keseluruhan masih dapat dipertahankan. Selain itu, penyangga memberi fleksibilitas bagi peneliti untuk memperdalam bagian tertentu tanpa menabrak tenggat akhir. Pendekatan ini sejalan dengan praktik yang disarankan Paleyes, Urma, dan Lawrence (2022) untuk proyek ML yang melibatkan banyak komponen dan dependensi.

V.3 Kebutuhan Sumber Daya

Kebutuhan sumber daya untuk merealisasikan arsitektur DataOps dan MLOps terintegrasi mencakup infrastruktur komputasi, perangkat lunak, sumber daya manusia, serta kemungkinan kebutuhan anggaran. Setiap kategori sumber daya memberikan kontribusi langsung terhadap kelancaran implementasi dan kualitas hasil akhir sistem. Perencanaan kapasitas, ketersediaan, dan keterbatasan sumber daya sejak awal diperlukan agar proses implementasi dapat dikendalikan dan tidak terhambat di tengah jalan. Selain itu, pemetaan kebutuhan yang jelas membantu peneliti dan pembimbing menilai kelayakan rencana teknis terhadap kondisi nyata.

Seluruh komponen perangkat lunak utama yang digunakan dalam arsitektur ini ber-

asal dari ekosistem *open-source* atau memiliki *free tier*, sehingga biaya lisensi dapat diasumsikan sangat kecil atau mendekati nol. Konsekuensinya, struktur biaya tugas akhir terutama akan didominasi oleh konsumsi infrastruktur (baik lokal maupun *cloud*) dan alokasi waktu sumber daya manusia, bukan oleh biaya lisensi perangkat lunak. Dengan demikian, penekanan perencanaan anggaran bergeser ke pengelolaan kapasitas komputasi dan strategi pemakaian *cloud* secara hemat.

V.3.1 Infrastruktur dan Sumber Daya Komputasi

Lapisan infrastruktur dan sumber daya komputasi berfungsi sebagai fondasi fisik dan logis bagi seluruh komponen arsitektur. Penentuan kapasitas minimal dan bentuk orkestrasi mempertimbangkan tujuan tugas akhir sebagai *proof-of-concept*, namun tetap dijaga agar relevan dengan kebutuhan produksi. Rancangan ini juga menyediakan ruang bagi eksperimen lanjutan sehingga konfigurasi yang dipilih dapat beradaptasi ketika kompleksitas beban kerja meningkat. Dengan demikian, infrastruktur tidak hanya memadai untuk proposal tugas akhir, tetapi juga memiliki jalur perkembangan yang jelas.

1. Perangkat keras lokal (laptop pengembangan)

Perangkat keras lokal berupa laptop pribadi digunakan sebagai lingkungan pengembangan utama. Spesifikasi yang direncanakan mencakup RAM 32 GB (sekitar 26 GB *usable*), penyimpanan SSD 1–2 TB, GPU kelas menengah dengan memori sekitar 8 GB, dan sistem operasi Linux (native) atau Windows dengan WSL2. Dengan konfigurasi tersebut, sekitar 70–80% komponen arsitektur—seperti distribusi Kubernetes ringan, MinIO, Redis Stack, ClickHouse skala pengembangan, Kafka *single broker*, Kubeflow Notebooks, MLflow, Feast, dan *stack observability* dasar—dapat dijalankan secara lokal. Hal ini mengurangi ketergantungan pada *cloud* untuk aktivitas eksplorasi harian dan mempercepat siklus coba-gagal selama fase pengembangan awal.

2. Klaster Kubernetes

Untuk mendukung skenario yang lebih mendekati produksi, klaster Kubernetes direncanakan terdiri atas minimum tiga *node* agar mendukung *high availability* dan distribusi *workload*. Setiap *node* diperkirakan memiliki sedikitnya empat inti CPU dan 16 GB RAM. Konfigurasi ini dinilai memadai untuk kebutuhan *proof-of-concept* yang dijalankan secara berkala di *cloud* maupun lingkungan komputasi bersama, sekaligus menyediakan ruang peningkatan kapasitas bila beban kerja bertambah (Lakka 2025). Dalam praktik tugas akhir, klaster Kubernetes penuh hanya akan diaktifkan untuk pengujian terintegrasi

dan uji beban, sementara eksplorasi awal dan *debugging* rutin tetap dilakukan di klaster ringan pada laptop pengembangan.

3. Penyimpanan

Kapasitas penyimpanan minimal yang direncanakan adalah sekitar 200 GB untuk menampung *dataset* sampel, artefak model, serta data operasional seperti log dan metadata. Untuk skenario tugas akhir yang mempertimbangkan *dataset* historis yang lebih besar, kapasitas tersebut dapat ditingkatkan bertahap hingga 500 GB atau 1 TB. Penyimpanan berbasis SSD diprioritaskan untuk komponen berlatensi rendah seperti Redis dan ClickHouse agar waktu respons tetap konsisten pada beban yang lebih tinggi. Pemisahan media penyimpanan untuk data kritis dan nonkritis juga dapat dipertimbangkan jika sumber daya memungkinkan sehingga risiko gangguan dapat ditekan.

4. Sumber daya GPU

Ketersediaan GPU bersifat opsional tetapi bermanfaat untuk mendemonstrasikan pelatihan *deep learning*. Satu unit GPU pada lingkungan lokal atau *cloud* dinilai cukup untuk *proof-of-concept* model berukuran menengah. Apabila GPU tidak tersedia, tugas akhir tetap dapat berjalan dengan berfokus pada model yang lebih ringan sehingga pelatihan di CPU masih berada dalam rentang waktu yang wajar. Keputusan penggunaan GPU akan disesuaikan dengan kebutuhan skenario dan ketersediaan anggaran tanpa mengubah struktur utama arsitektur.

Secara keseluruhan, kebutuhan infrastruktur dirancang agar realistis untuk konteks mahasiswa namun tetap representatif terhadap pola operasional di produksi. Konfigurasi minimal dipastikan mampu menjalankan *pipeline* DataOps dan MLOps secara *end-to-end* tanpa mengharuskan penggunaan sumber daya berlebihan. Di sisi lain, rancangan ini masih menyisakan ruang untuk eksperimen teknis lanjutan jika infrastruktur tambahan tersedia, misalnya ketika proyek diadaptasi ke lingkungan perusahaan.

V.3.2 Perangkat Lunak dan Alat

Pemilihan perangkat lunak dalam tugas akhir ini berfokus pada pemanfaatan ekosistem *open-source* yang matang, terdokumentasi dengan baik, dan banyak digunakan di industri. Pendekatan tersebut menjaga relevansi hasil tugas akhir dan menghindari ketergantungan pada lisensi komersial tertentu. Setiap komponen dipilih untuk mengisi peran yang jelas dalam arsitektur sehingga implementasi dapat direplikasi dan diverifikasi oleh pihak lain. Penambahan komponen ekosistem Raystack (seper-

ti Frontier, Guardian, Raccoon, Stencil, Optimus, Dagger, Firehose, Siren, Meteor, dan Compass) tetap berada dalam ranah lisensi permisif sehingga tidak menambah biaya lisensi dan terutama berpengaruh pada kompleksitas operasional.

1. Platform orkestrasi

Platform orkestrasi utama terdiri atas Kubernetes, Docker, dan ArgoCD. Kubernetes berfungsi sebagai *control plane* yang mengelola *workload* berbasis kontainer, Docker menyediakan mekanisme pembuatan dan distribusi *container image*, sedangkan ArgoCD menambahkan kemampuan GitOps untuk menerapkan konfigurasi secara deklaratif dari *repository* Git ke kluster. Kombinasi ketiganya membentuk fondasi pengelolaan aplikasi yang konsisten dan dapat diaudit, serta mendukung pola *continuous delivery* untuk komponen arsitektur.

2. Ingesti dan pemrosesan data

Lapisan ingest dan pemrosesan data memanfaatkan Apache Kafka sebagai *event streaming backbone*, Kafka Connect untuk integrasi dengan sumber dan tujuan eksternal, serta Raccoon untuk menerima *event* dari aplikasi dan meneruskannya ke Kafka. Stencil digunakan sebagai *schema registry* untuk pengelolaan *schema* berbasis Protobuf. Apache Spark dimanfaatkan untuk pemrosesan *batch*, sedangkan Apache Flink digunakan untuk pemrosesan *stream* berstatus. Dagger dapat digunakan sebagai kerangka konfigurasi untuk membangun *job* Flink tanpa banyak kode tambahan. Orkestrasi *workflow* pemrosesan dilakukan menggunakan Apache Airflow, dan Optimus memperkaya eksekusi transformasi data dengan pengelolaan dependensi dan pemeriksaan kualitas. Kerangka *data validation* seperti Great Expectations dimanfaatkan untuk mendefinisikan dan mengevaluasi *expectation* terhadap kualitas data pada titik-titik kritis *pipeline*.

3. Layanan fitur

Layanan fitur dibangun menggunakan Feast, ClickHouse, dan Redis Stack. Feast menyediakan *feature store* terpusat dengan dukungan *offline* dan *online serving*. ClickHouse bertindak sebagai *offline store* berorientasi kolom untuk menyimpan fitur historis, sedangkan Redis Stack digunakan sebagai *online store* berlatensi rendah yang cocok untuk *real-time inference*. Kombinasi ini selaras dengan arsitektur yang dijelaskan pada Bab IV dan mendukung penerapan *point-in-time correctness* yang dibahas pada tahapan sebelumnya.

4. Penyimpanan

Lapisan penyimpanan memanfaatkan MinIO sebagai *object storage* kompa-

tibel S3 untuk menyimpan *dataset*, artefak model, dan log. ClickHouse dan Redis menggunakan *persistent volume* Kubernetes dengan kelas penyimpanan berbasis SSD untuk memenuhi kebutuhan latensi. PostgreSQL dimanfaatkan sebagai *relational database* untuk menampung metadata sistem, khususnya pada DataHub dan MLflow, sehingga data konfigurasi dan metadata tersimpan terstruktur dan mudah diakses.

5. Siklus hidup model

Pengelolaan siklus hidup model didukung oleh Kubeflow, MLflow, dan KServe. Kubeflow menyediakan lingkungan *pipeline* dan *notebook* di atas Kubernetes, MLflow menangani *experiment tracking* serta *model registry*, dan KServe berperan di sisi *serving*. Integrasi ketiganya membentuk *pipeline* eksperimen ke produksi yang konsisten dan mengurangi jurang antara lingkungan pengembangan dan lingkungan operasional.

6. Observability

Tumpukan *observability* terdiri atas Prometheus, Grafana, AlertManager, Loki, Jaeger, dan OpenTelemetry. Prometheus mengumpulkan metrik melalui mekanisme *scraping*, Grafana menyajikan *dashboard*, dan AlertManager mengelola notifikasi peringatan. Loki digunakan sebagai sistem agregasi log yang terintegrasi dengan Grafana, sedangkan Jaeger menjadi backend *tracing* yang menerima data dari OpenTelemetry. Siren dapat dimanfaatkan sebagai kerangka manajemen notifikasi yang mengatur aliran *alert* ke berbagai kanal tanpa mengubah konfigurasi dasar AlertManager.

7. Tata kelola

DataHub digunakan untuk mengelola katalog aset data, fitur, model, dan *pipeline* beserta *lineage*-nya. Meteor dimanfaatkan sebagai *metadata collector* untuk mengekstrak metadata dari berbagai sistem (misalnya ClickHouse, Kafka, Feast, MLflow) dan mengirimkannya ke DataHub atau katalog alternatif seperti Compass. Dengan cara ini, pandangan lintas komponen dapat disatukan pada satu platform metadata dan mempermudah aktivitas *data discovery* maupun audit.

8. Bahasa dan framework pemrograman

Bahasa utama yang digunakan adalah Python dengan pustaka ML seperti *scikit-learn*, TensorFlow, dan PyTorch. Pustaka data seperti *pandas* dan *numpy* dimanfaatkan untuk analisis dan manipulasi data dalam tahap eksplorasi maupun pelatihan model. Klien Feast dan MLflow digunakan untuk berinteraksi dengan *feature store* dan *tracking server* secara terprogram sehingga *pipeli-*

ne dapat ditulis sebagai kode yang dapat diulang. Integrasi dengan Kafka, ClickHouse, dan Redis menggunakan pustaka klien resmi maupun komunitas sehingga interaksi dengan sistem terdistribusi tetap konsisten dan terdokumentasi.

9. Alat pengembangan

Alat pengembangan mencakup Git (dengan layanan Git terpusat seperti Gitea), IDE seperti VS Code, PyCharm, atau Jupyter, serta alat CLI seperti *kubectl*, *helm*, dan *docker*. Alat pengujian beban seperti Locust atau k6 digunakan untuk menguji kinerja layanan secara sintetis pada berbagai skenario trafik. Kombinasi alat ini mendukung siklus pengembangan, pengujian, dan *deployment* harian serta mempermudah kolaborasi dan *code review*.

10. Lapisan keamanan dan akses

Untuk mendukung autentikasi dan otorisasi yang terstruktur, Frontier dan Guardian dari ekosistem Raystack dipertimbangkan sebagai lapisan di atas penyimpanan data dan antarmuka administrasi. API gateway seperti Kong digunakan untuk memusatkan kontrol akses, *rate limiting*, dan *request validation* pada *endpoint* layanan. Pengelolaan rahasia dan kunci layanan dapat diperkuat dengan Vault jika dibutuhkan kemampuan di luar *secrets manager* bawaan penyedia *cloud*, sehingga kredensial dan kunci enkripsi tidak tersebar di konfigurasi aplikasi.

11. CI/CD dan automasi

Lapisan *continuous integration/continuous delivery* (CI/CD) dan automasi dapat memanfaatkan Tekton sebagai *pipeline engine* berbasis Kubernetes untuk menjalankan *build*, *test*, dan *deployment pipeline*. Firehose digunakan untuk *streaming* data *real-time* dari Kafka ke berbagai *sink* seperti penyimpanan objek atau basis data analitik, sekaligus mendukung pola *batch* dan *streaming ingestion* dengan konfigurasi yang fleksibel. Dengan kombinasi ini, proses rilis dan pengelolaan aliran data dapat diotomasi secara konsisten dan terintegrasi dengan kluster Kubernetes yang sama.

Penggunaan perangkat lunak *open-source* membuat tugas akhir lebih mudah direplikasi dan diverifikasi oleh pihak lain. Dukungan dokumentasi komunitas dan forum diskusi juga membantu mempercepat *troubleshooting* ketika muncul kendala teknis. Dengan demikian, peneliti dapat memfokuskan usaha pada penyusunan arsitektur dan integrasi alih-alih membangun komponen dasar dari awal.

V.3.3 Layanan *Cloud* dan Pertimbangan Anggaran

Pemanfaatan layanan *cloud* menjadi opsi penting untuk menyediakan infrastruktur komputasi dan penyimpanan yang fleksibel selama tugas akhir berlangsung. Jika arsitektur dijalankan sepenuhnya di *cloud* dengan konfigurasi minimal tiga *node* dan penyimpanan persisten untuk ClickHouse, Redis, dan MinIO, biaya bulanan dapat berada pada kisaran ratusan dolar Amerika, bergantung pada penyedia, kelas instans, dan wilayah. Komponen biaya utama meliputi instans *compute* untuk *node* Kubernetes, penyimpanan persisten, dan biaya transfer data keluar. Penambahan komponen ekosistem Raystack tidak mengubah profil biaya lisensi karena semuanya berbasis *open-source*, sehingga struktur biaya tetap menitikberatkan konsumsi komputasi dan penyimpanan.

Dalam konteks proposal tugas akhir ini, digunakan strategi pemanfaatan *cloud* secara **hibrida** untuk menekan biaya aktual. Mayoritas aktivitas pengembangan, eksplorasi, dan *debugging* dijalankan pada klaster Kubernetes lokal di laptop pengembangan yang memiliki kapasitas memori, penyimpanan SSD, dan GPU yang memadai. Klaster *cloud* berukuran kecil hanya diaktifkan ketika diperlukan pengujian terintegrasi dan uji beban yang lebih mendekati skenario produksi, misalnya untuk menguji skala Kafka, ClickHouse, atau KServe pada beban yang lebih tinggi. Setelah pengujian selesai, klaster dapat di-*scale down* atau dimatikan agar tidak menimbulkan biaya berjalan. Pendekatan ini mengurangi biaya berkelanjutan sekaligus tetap memberikan kesempatan menguji arsitektur dalam lingkungan yang menyerupai produksi.

Strategi hibrida tersebut dikombinasikan dengan pemanfaatan **kredit gratis** dari penyedia *cloud*. Akun baru dari penyedia besar umumnya memperoleh kredit yang dapat dimanfaatkan untuk eksperimen dalam periode tertentu. Dengan disiplin dalam mengaktifkan dan mematikan klaster, tugas akhir menargetkan konsumsi biaya *cloud* kotor pada rentang yang masih dapat ditutup sepenuhnya oleh kredit gratis selama pemakaian dikendalikan. Selama konsumsi kumulatif berada di bawah plafon kredit yang tersedia, biaya *cloud* aktual yang perlu dibayar peneliti diharapkan mendekati nol. Ringkasan estimasi biaya layanan *cloud* untuk cakupan tugas akhir ditampilkan pada Tabel V.6. Tabel tersebut secara eksplisit hanya memuat komponen biaya *cloud*, karena seluruh perangkat lunak utama yang digunakan bersifat *open-source* atau memiliki *free tier*, dan biaya kelistrikan perangkat keras lokal tidak dihitung sebagai biaya langsung tugas akhir.

Tabel V.6 Ringkasan Biaya Utama untuk Skala Tugas Akhir

Komponen	Estimasi Minimum (USD)	Estimasi Maksimum (USD)
Konsumsi layanan <i>cloud</i> kotor (komputasi, penyimpanan, <i>network</i>)	90	260
Kredit gratis layanan <i>cloud</i> yang tersedia	300	300
Perkiraan biaya <i>cloud</i> aktual setelah pemanfaatan kredit	0	0

V.3.4 Sumber Daya Manusia dan Keahlian

Keberhasilan implementasi arsitektur DataOps dan MLOps terintegrasi tidak hanya ditentukan oleh infrastruktur dan alat, tetapi juga oleh kompetensi teknis peneliti dan pemangku kepentingan yang terlibat. Berbagai keterampilan dibutuhkan untuk mengonfigurasi, mengoperasikan, dan memelihara platform, mulai dari administrasi Kubernetes hingga pengembangan *pipeline* ML. Walaupun tidak semua kemampuan harus dikuasai secara mendalam sejak awal, pemahaman dasar di setiap area mempercepat proses belajar dan mengurangi risiko kebuntuan teknis. Dengan pemetaan keahlian yang jelas, peneliti dapat mengenali area yang memerlukan dukungan tambahan dan menyesuaikan fokus belajar.

Secara garis besar, peran dan alokasi waktu yang diasumsikan pada tugas akhir ini adalah sebagai berikut. Mahasiswa atau peneliti utama berperan sebagai perancang arsitektur, penanggung jawab *deployment stack* lokal dan *cloud*, pengembang *pipeline* DataOps dan MLOps, serta penyusun eksperimen, evaluasi, dan dokumentasi, dengan durasi kerja intensif sekitar beberapa bulan. Pembimbing akademik atau mentor industri berperan melakukan *review* desain, membantu mengelola ruang lingkup dan prioritas, serta mengkaji hasil eksperimen dan klaim ilmiah, dengan keterlibatan berkala yang biasanya diwujudkan dalam pertemuan bimbingan mingguan. Engineer atau DevOps internal bersifat opsional dan dapat dilibatkan jika *proof-of-concept* ini nantinya ingin diintegrasikan lebih dalam ke infrastruktur perusahaan dan diperkuat untuk kebutuhan keamanan serta *compliance* produksi.

Dalam konteks tugas akhir di lingkungan akademik, biaya sumber daya manusia biasanya tidak dihitung sebagai biaya langsung, namun pemetaan peran dan alokasi waktu tetap relevan untuk menunjukkan bahwa rancangan arsitektur realistik diimplementasikan oleh satu orang mahasiswa dengan dukungan terbatas. Selain alokasi peran, sejumlah keterampilan teknis menjadi prasyarat agar tugas akhir dapat ber-

jalan dan seluruh tahapan di Bab V dapat direalisasikan.

1. **Administrasi Kubernetes** dan orkestrasi kontainer diperlukan untuk mengelola klaster, men-*deploy* aplikasi, dan melakukan *troubleshooting*. Pemahaman tentang *manifest*, *kubectl*, *ingress*, *service*, dan *config map* menjadi prasyarat untuk pengelolaan klaster yang andal. Keterampilan ini juga membantu dalam menafsirkan metrik dan log ketika muncul kendala pada *workload*.
2. **Kemahiran pemrograman Python** dibutuhkan untuk mengembangkan *pipeline* data, menulis definisi fitur di Feast, menyusun alur pelatihan model, dan membuat skrip pengujian. Penguasaan pustaka *pandas*, *numpy*, dan *scikit-learn* akan mempercepat pengembangan dan eksplorasi. Keterampilan ini juga relevan untuk mengintegrasikan berbagai klien layanan seperti Kafka, ClickHouse, dan Redis dalam satu alur eksperimen.
3. **Konsep sistem terdistribusi** penting untuk memahami *messaging*, pemrosesan *stream*, konsistensi data, dan toleransi kesalahan. Bekal ini membantu merancang layanan yang tangguh dan dapat diskalakan seiring pertumbuhan beban kerja. Pemahaman tersebut juga berkontribusi pada pengambilan keputusan ketika terjadi insiden yang melibatkan beberapa komponen sekaligus.
4. **Dasar-dasar *machine learning*** diperlukan untuk memahami alur kerja ML, metrik evaluasi, dan isu seperti *overfitting* serta *data leakage*. Pengetahuan ini membantu menafsirkan hasil eksperimen dan menentukan langkah perbaikan model yang tepat. Dengan dasar yang kuat, peneliti dapat menghubungkan indikator teknis yang muncul di platform dengan implikasi pada kualitas prediksi.
5. **Praktik DevOps** yang mencakup GitOps, *continuous integration/continuous delivery* (CI/CD), *monitoring*, dan *observability* diperlukan untuk merancang proses perubahan yang terkontrol dan terdokumentasi. Keterampilan ini juga mendukung integrasi komponen yang mulus dan mengurangi risiko kesalahan konfigurasi. Penerapan praktik tersebut memperkuat klaim bahwa platform yang dibangun bukan hanya berfungsi, tetapi juga siap dioperasikan secara berkelanjutan.
6. **Bimbingan pembimbing dan dukungan komunitas** menjadi elemen penting untuk mengatasi kurva pembelajaran. Dokumentasi, tutorial, dan forum komunitas untuk proyek seperti Feast, Kubeflow, MLflow, KServe, DataHub, serta komponen Raystack menyediakan referensi praktis yang dapat dimanfaatkan ketika menemui hambatan teknis. Dukungan ini melengkapi upaya

mandiri peneliti dalam memahami dan mengintegrasikan berbagai komponen.

Dengan kombinasi keahlian teknis, bimbingan akademik, dan dukungan komunitas, peneliti diharapkan dapat membangun kapasitas yang memadai untuk menyelesaikan proposal tugas akhir ini. Fokus utama bukan pada penguasaan penuh semua teknologi yang terlibat, tetapi pada kemampuan memilih dan mengadaptasi alat yang relevan secara efektif. Pendekatan ini sejalan dengan tujuan tugas akhir yang menekankan pemahaman arsitektural dan integrasi, bukan sekadar penguasaan spesifik satu produk.

V.4 Analisis Risiko dan Mitigasi

Pendekatan sistematis terhadap identifikasi dan mitigasi risiko menjadi salah satu faktor kunci keberhasilan tugas akhir ini. Proyek yang menggabungkan banyak komponen *open-source* rentan terhadap berbagai kendala teknis, penundaan jadwal, dan masalah koordinasi. Paleyes, Urma, dan Lawrence (2022) mencatat bahwa proyek ML sering menghadapi tantangan pada aspek organisasi, teknis, dan operasional, termasuk kualitas data, integrasi sistem, dan kompleksitas *deployment*. Oleh karena itu, analisis risiko disusun sejak awal agar strategi mitigasi dapat diterapkan secara proaktif, bukan hanya reaktif ketika masalah sudah muncul. Pendekatan ini membantu menjaga stabilitas dan kualitas selama siklus tugas akhir.

V.4.1 Risiko Teknis

Risiko teknis berkaitan dengan tantangan yang muncul dari sisi teknologi dan integrasi antarkomponen. Kompleksitas arsitektur terdistribusi, perbedaan versi perangkat lunak, dan keterbatasan sumber daya perangkat keras dapat memunculkan masalah yang sulit diprediksi. Dengan mengidentifikasi risiko tersebut sejak awal, peneliti dapat menyiapkan waktu cadangan, alat bantu, dan strategi penyelesaian yang sesuai. Upaya ini diharapkan mengurangi kemungkinan hambatan teknis yang mengganggu jadwal utama.

1. Risiko 1: Kompleksitas integrasi antarkomponen.

a) Deskripsi

Integrasi beberapa komponen *open-source* dalam satu platform dapat lebih rumit daripada perkiraan awal. Perbedaan versi, perilaku yang kurang terdokumentasi, dan asumsi implisit terhadap lingkungan *deployment* sering memicu masalah yang sulit dilacak. Ekosistem yang akan diintegrasikan mencakup Feast, Kubeflow, MLflow, KServe, DataHub,

Kafka, Spark, Flink, serta berbagai komponen tambahan dari ekosistem Raystack. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mencatat bahwa tantangan integrasi merupakan salah satu hambatan utama dalam membangun arsitektur MLOps berbasis *open-source*.

b) Strategi mitigasi

Mitigasi dilakukan dengan menyusun *proof-of-concept* awal untuk integrasi yang dinilai paling kritis, seperti Feast–ClickHouse/Redis, MLflow–Kubeflow, KServe–Feast, dan alur Kafka–Flink–Firehose. Versi komponen dipilih dari rilis yang stabil dan, bila memungkinkan, mengacu pada kombinasi yang telah banyak diadopsi komunitas. Waktu cadangan dialokasikan secara eksplisit untuk *debugging* dan penyesuaian konfigurasi. Dokumentasi resmi, contoh integrasi, dan diskusi komunitas dimanfaatkan sebagai rujukan utama untuk mengurangi risiko mencoba konfigurasi yang belum teruji.

2. Risiko 2: Keterbatasan sumber daya dan kinerja.

a) Deskripsi

Menjalankan platform MLOps yang relatif lengkap pada sumber daya terbatas berpotensi memicu perebutan sumber daya sehingga kinerja menurun atau terjadi *out-of-memory*. Lakka (2025) menegaskan bahwa manajemen sumber daya dan desain skalabilitas yang tepat merupakan faktor penting pada lingkungan Kubernetes, terutama ketika banyak komponen ditempatkan dalam satu klaster.

b) Strategi mitigasi

Mitigasi dilakukan dengan menyesuaikan ukuran *deployment* dengan kapasitas nyata yang tersedia. Layanan nonkritis dijalankan menggunakan konfigurasi *lightweight*, dan *resource requests* serta *limits* diterapkan di setiap *pod* untuk mencegah satu komponen menghabiskan seluruh sumber daya. Pemantauan CPU, memori, dan I/O melalui Prometheus dan Grafana digunakan untuk mendeteksi *bottleneck* lebih awal. Ukuran *dataset* sampel disesuaikan agar tetap representatif tanpa terlalu membebani sistem, dan bila diperlukan dimanfaatkan kredit *cloud* gratis atau infrastruktur tambahan untuk uji beban tertentu.

3. Risiko 3: Kompatibilitas versi dan konflik dependensi.

a) Deskripsi

Proyek *open-source* berkembang secara independen dan cukup cepat.

Dalam praktik, versi tertentu dapat menghadirkan *breaking changes* atau konflik dependensi. Hal ini khususnya terasa di ekosistem Python yang memiliki ketergantungan antarpustaka yang kompleks. Kondisi tersebut dapat memicu kegagalan *deployment* atau perilaku tidak terduga jika tidak dikendalikan sejak awal.

b) Strategi mitigasi

Peneliti menelusuri kombinasi versi yang telah diketahui kompatibel melalui dokumentasi resmi, *compatibility matrix*, atau referensi dari komunitas. *Version pinning* diterapkan pada berkas konfigurasi dependensi dan *container image*. Pembaruan versi diuji terlebih dahulu di lingkungan terpisah sebelum diterapkan ke klaster utama, dan penggunaan *virtual environment* atau kontainer memastikan dependensi Python tetap terisolasi antar layanan. Dengan cara ini, risiko konflik versi dapat dikurangi tanpa menghambat kebutuhan pembaruan keamanan.

V.4.2 Risiko Operasional

Risiko operasional berkaitan dengan pengelolaan jadwal, ruang lingkup, dan dokumentasi selama tugas akhir berlangsung. Proyek tugas akhir rentan terhadap keterlambatan, perubahan cakupan, dan penumpukan dokumentasi pada tahap akhir. Tanpa mekanisme pengendalian linimasa dan ruang lingkup, kualitas hasil dapat menurun meskipun rancangan teknis sudah baik. Strategi pengelolaan operasional disusun untuk menjaga keseimbangan antara ambisi teknis dan realitas waktu.

1. Risiko 4: Keterlambatan linimasa dan *scope creep*.

a) Deskripsi

Setiap fase implementasi berpotensi memerlukan waktu lebih lama daripada yang direncanakan karena kendala teknis atau kurva pembelajaran yang lebih curam. *Scope creep* dapat muncul ketika fitur tambahan dimasukkan di luar kebutuhan inti yang telah didefinisikan. Paleyes, Urma, dan Lawrence (2022) menegaskan bahwa faktor-faktor tersebut sering menjadi sumber keterlambatan pada proyek ML yang kompleks. Jika tidak dikendalikan, penumpukan pekerjaan di akhir periode dapat mengurangi kualitas pengujian maupun dokumentasi.

b) Strategi mitigasi

Mitigasi dilakukan dengan menyediakan waktu cadangan pada tiap fase dan meninjau kemajuan secara berkala bersama pembimbing. Peneliti menyusun prioritas fitur menggunakan kerangka Must-have, Should-

have, Could-have, Won't-have (MoSCoW) sehingga kebutuhan inti didahulukan. Fitur tambahan yang bersifat *nice-to-have* dapat ditunda atau dikeluarkan apabila jadwal mulai tertekan. Dengan demikian, keterlambatan pada bagian nonesensial tidak mengganggu penyelesaian komponen utama dan tenggat akhir tetap realistis.

2. Risiko 5: Dokumentasi dan transfer pengetahuan yang tidak cukup.

a) Deskripsi

Kompleksitas sistem menimbulkan kebutuhan dokumentasi yang besar, baik dari sisi arsitektur maupun operasional. Jika dokumentasi kurang, *reproducibility* menjadi sulit, penanganan masalah lambat, dan nilai jangka panjang platform berkurang. Munappy dkk. (2022) menekankan bahwa dokumentasi dan *version control* yang sistematis merupakan prasyarat penting bagi sistem ML produksi. Dalam konteks tugas akhir, dokumentasi yang lemah juga dapat menyulitkan proses penilaian dan tindak lanjut tugas akhir.

b) Strategi mitigasi

Dokumentasi dikembangkan secara paralel dengan implementasi menggunakan pola *documentation-as-code*. Templat dokumentasi terstruktur digunakan untuk menutupi aspek arsitektur, konfigurasi, panduan penggunaan, dan *troubleshooting*. Sebagian waktu tiap fase dialokasikan khusus untuk dokumentasi agar tidak tertunda hingga akhir. README yang jelas, *runbook* operasional, contoh alur *end-to-end*, dan diagram arsitektur dijadikan keluaran standar untuk memudahkan transfer pengetahuan dan penilaian oleh pembimbing.

V.4.3 Risiko Personal dan Organisasi

Risiko personal dan organisasi berkaitan dengan kondisi individu peneliti dan lingkungan akademik, termasuk beban studi lain dan faktor motivasi. Proyek dengan durasi beberapa bulan membutuhkan konsistensi upaya dan manajemen waktu yang baik. Tanpa perhatian pada aspek nonteknis ini, kualitas dan ketercapaian hasil dapat terpengaruh walaupun rancangan teknis sudah tepat. Strategi mitigasi pada bagian ini bertujuan menjaga keberlanjutan tugas akhir hingga selesai.

1. Risiko 6: Tuntutan waktu yang bersaing dan manajemen beban kerja.

a) Deskripsi

Tugas akhir dijalankan bersamaan dengan mata kuliah, ujian, atau tang-

gung jawab pribadi lainnya. Akumulasi tugas dapat memicu *burnout*, menurunkan kualitas pekerjaan, atau menyebabkan keterlambatan signifikan jika tidak dikelola. Ketidakseimbangan ini dapat menghambat penyelesaian fase penting dan menurunkan kualitas pengujian. Dalam jangka panjang, tekanan berlebih juga dapat memengaruhi kesehatan dan motivasi peneliti.

b) Strategi mitigasi

Peneliti melakukan penilaian realistis terhadap ketersediaan waktu sejak awal dan menyesuaikan jadwal tugas akhir jika diperlukan. Teknik *time blocking* digunakan untuk mengalokasikan waktu khusus tugas akhir yang relatif bebas dari gangguan aktivitas lain. Komunikasi proaktif dengan pembimbing dilakukan ketika kendala waktu muncul sehingga ruang lingkup atau prioritas dapat disesuaikan. Dengan cara ini, beban kerja dapat diseimbangkan tanpa mengorbankan kualitas atau tenggat utama.

2. Risiko 7: Kehilangan motivasi atau minat.

a) Deskripsi

Linimasa yang panjang dan pekerjaan teknis yang rumit dapat menurunkan motivasi, terutama ketika peneliti menghadapi masalah berulang atau progres terasa lambat. Kondisi ini berpotensi menurunkan konsistensi dan kualitas hasil akhir jika tidak diantisipasi. Dalam konteks tugas akhir, hilangnya motivasi juga dapat menghambat komunikasi dengan pembimbing dan mengurangi frekuensi iterasi desain maupun eksperimen.

b) Strategi mitigasi

Peneliti menjaga fokus pada tujuan jangka panjang dan kontribusi tugas akhir terhadap praktik DataOps dan MLOps. Setiap *milestone* yang tercapai didokumentasikan sebagai bukti kemajuan dan sumber motivasi baru. Berbagi perkembangan dengan rekan dan pembimbing membantu memperoleh umpan balik serta dukungan ketika menghadapi hambatan. Dengan pendekatan tersebut, peneliti diharapkan dapat mempertahankan ritme kerja hingga akhir proyek dan menyelesaikan seluruh tahapan implementasi yang direncanakan.

Dengan mengenali berbagai risiko tersebut dan menyiapkan strategi mitigasi yang konkret, proposal tugas akhir ini memiliki landasan yang lebih kuat untuk mencapai tujuannya. Fleksibilitas dalam perencanaan, komunikasi terbuka dengan pem-

bimbing, dan fokus pada *deliverables* inti menjadi faktor utama dalam menghadapi tantangan selama implementasi. Pendekatan ini diharapkan menjaga kualitas hasil akhir sekaligus memastikan bahwa platform DataOps dan MLOps yang dibangun dapat dipertanggungjawabkan baik dari sisi teknis maupun metodologis.

DAFTAR PUSTAKA

- Adusumilli, Lakshmi Vara Prasad. 2025. "Serverless Kubernetes: The Evolution of Container Orchestration". *European Journal of Computer Science and Information Technology* 13 (30): 20–36. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>.
- Ahmad, Tazeem, Mohd Adnan, Saima Rafi, Muhammad Azeem Akbar, dan Ayesha Anwar. June 2024. "MLOps-Enabled Security Strategies for Next-Generation Operational Technologies". Dalam *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*, 662–670. <https://doi.org/10.1145/3661167.3661283>. <https://doi.org/10.1145/3661167.3661283>.
- Amazon Web Services. 2024. *What is an Event-Driven Architecture?* Diakses 15 Oktober 2025. <https://aws.amazon.com/event-driven-architecture/>.
- Amershi, Saleema, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, dan Thomas Zimmermann. 2019. "Software Engineering for Machine Learning: A Case Study". Dalam *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. IEEE. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>.
- Amou Najafabadi, Faezeh, Justus Bogner, Ilias Gerostathopoulos, dan Patricia Lago. 2024. "An Analysis of MLOps Architectures: A Systematic Mapping Study". Dalam *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, 3–6 September 2024, Proceedings*, 14889:69–85. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-031-70797-1_5. https://doi.org/10.1007/978-3-031-70797-1_5.
- Anaconda, Inc. July 2020. *2020 State of Data Science: Moving From Hype Toward Maturity*. Diakses 1 November 2025. <https://know.anaconda.com/rs/387-XNW-688/images/Anaconda-SODS-Report-2020-Final.pdf>.
- Apache Software Foundation. 2024a. *Apache Airflow Documentation*. Dokumentasi. Diakses 5 November 2025. <https://airflow.apache.org/docs/>.

- . 2024b. *Apache Flink Documentation*. Diakses 30 September 2025. <https://flink.apache.org/documentation/>.
- Apache Software Foundation. 2024c. *Apache Kafka Documentation*. Diakses 15 Oktober 2025. <https://kafka.apache.org/documentation/>.
- . 2024d. *Apache Spark Documentation*. Diakses 2 November 2025. <https://spark.apache.org/docs/latest/>.
- Argo Project. 2024. *Argo CD - Declarative GitOps CD for Kubernetes*. Dokumentasi. Diakses 8 November 2025. <https://argo-cd.readthedocs.io/>.
- Bayram, Firas, Bestoun S. Ahmed, dan Andreas Kassler. 2022. “From Concept Drift to Model Degradation: An Overview on Performance-Aware Drift Detectors”. *Knowledge-Based Systems* 245:108632. <https://doi.org/10.1016/j.knosys.2022.108632>. <https://doi.org/10.1016/j.knosys.2022.108632>.
- Bernardo, B. M. V., H. S. Mamede, J. M. P. Barroso, dan V. M. P. D. dos Santos. 2024. “Data Governance and Quality Management: Innovation and Breakthroughs across Different Fields”. *Journal of Innovation and Knowledge* 9 (4): 100598. <https://doi.org/10.1016/j.jik.2024.100598>. <https://doi.org/10.1016/j.jik.2024.100598>.
- Burgueno-Romero, Anabel M., José L. Cánovas Izquierdo, dan José García-García. 2025. “Toward an Open Source MLOps Architecture”. *IEEE Software* 42 (1): 28–35. <https://doi.org/10.1109/MS.2024.3421675>. <https://doi.org/10.1109/MS.2024.3421675>.
- Carbone, Paris, Stephan Ewen, Kostas Tzoumas, Volker Markl, dan Fabian Hueske. 2015. “Apache Flink: Stream and Batch Processing in a Single Engine”. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 36 (4): 28–38. <https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf>.
- Céspedes Sisniega, Jaime, Vicente Rodríguez, Germán Moltó, dan Álvaro López García. 2024. “Efficient and Scalable Covariate Drift Detection in Machine Learning Systems with Serverless Computing”. *Future Generation Computer Systems* 161:174–188. <https://doi.org/10.1016/j.future.2024.07.010>. <https://doi.org/10.1016/j.future.2024.07.010>.

- Chien, Melody. February 2022. *How to Improve Your Data Quality*. Diakses 29 September 2025. Gartner. <https://www.gartner.com/smarterwithgartner/how-to-improve-your-data-quality>.
- ClickHouse Inc. 2024. *ClickHouse Documentation*. Dokumentasi. Diakses 5 November 2025. <https://clickhouse.com/docs/>.
- Cloud Native Computing Foundation. 2024. *Observability Whitepaper*. Diakses 3 November 2025. <https://github.com/cncf/tag-observability/blob/main/whitepaper.md>.
- DataHub Project. 2024. *DataHub Documentation*. Dokumentasi. Diakses 5 November 2025. <https://datahubproject.io/docs/>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, dan Kristina Toutanova. 2019. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. Dalam *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>. <https://doi.org/10.18653/v1/N19-1423>.
- Feast Project. 2024. *Feast Documentation*. Dokumentasi. Diakses 5 November 2025. <https://docs.feast.dev/>.
- Fowler, Martin, dan James Lewis. 2014. *Microservices*. Diakses November 2025. <https://martinfowler.com/articles/microservices.html>.
- Gitea Contributors. 2025. *Gitea Documentation*. Dokumentasi. Diakses 10 November 2025. <https://docs.gitea.com/>.
- Google Cloud. 2024. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*. Cloud Architecture Center. Diakses 2 November 2025. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>.
- Grafana Labs. 2024a. *Grafana Documentation*. Dokumentasi. Diakses 5 November 2025. <https://grafana.com/docs/grafana/latest/>.
- . 2024b. *Grafana Loki Documentation*. Dokumentasi. Diakses 10 November 2025. <https://grafana.com/docs/loki/latest/>.

- Great Expectations. 2024. *Great Expectations Documentation*. Dokumentasi. Diakses 10 November 2025. <https://docs.greatexpectations.io/docs/>.
- HashiCorp. 2025. *Vault Secrets Management Documentation*. Dokumentasi. Diakses 10 November 2025. <https://developer.hashicorp.com/vault/docs>.
- Hinder, Fabian, Valerie Vaquet, Johannes Brinkrolf, dan Barbara Hammer. 2023. "Model-Based Explanations of Concept Drift". *Neurocomputing* 555:126640. <https://doi.org/10.1016/j.neucom.2023.126640>. <https://doi.org/10.1016/j.neucom.2023.126640>.
- Hsu, C. H., P. W. Liu, Y. C. Fan, dan C. Y. Lin. 2024. "End-to-End Automation of ML Model Lifecycle Management Using Machine Learning Operations Platforms". Dalam *2024 International Conference on Consumer Electronics-Taiwan (ICCE-Taiwan)*, 1–6. IEEE. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>.
- IBM. 2024. *What is Observability? Logs, Metrics, and Distributed Tracing*. Diakses 2 November 2025. <https://www.ibm.com/topics/observability>.
- Jaeger Authors. 2024. *Jaeger: Open Source, End-to-End Distributed Tracing*. Dokumentasi. Diakses 10 November 2025. <https://www.jaegertracing.io/docs/>.
- Jain, Souratn, dan Jyotipriya Das. 2025. "Integrating Data Engineering and MLOps for Scalable and Resilient Machine Learning Pipelines: Frameworks, Challenges, and Future Trends". *World Journal of Advanced Engineering Technology and Sciences* 14 (1): 241–253. <https://doi.org/10.30574/wjaets.2025.14.1.0020>. <https://doi.org/10.30574/wjaets.2025.14.1.0020>.
- Jegou, Herve, Matthijs Douze, dan Cordelia Schmid. 2011. "Product Quantization for Nearest Neighbor Search". *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33 (1): 117–128. <https://doi.org/10.1109/TPAMI.2010.57>. <https://doi.org/10.1109/TPAMI.2010.57>.
- Jogalekar, Prasad, dan Murray Woodside. 2000. "Evaluating the Scalability of Distributed Systems". *IEEE Transactions on Parallel and Distributed Systems* 11 (6): 589–603. <https://doi.org/10.1109/71.862209>. <https://doi.org/10.1109/71.862209>.

- Johnson, Jeff, Matthijs Douze, dan Hervé Jégou. 2021. “Billion-Scale Similarity Search with GPUs”. *IEEE Transactions on Big Data* 7 (3): 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>. <https://doi.org/10.1109/TBDATA.2019.2921572>.
- Jourand, Nicolas, Tom Bayer, Tobias Biegel, dan Joachim Metternich. 2023. “Handling Concept Drift in Deep Learning Applications for Process Monitoring”. *Procedia CIRP* 120:42–47. <https://doi.org/10.1016/j.procir.2023.08.007>. <https://doi.org/10.1016/j.procir.2023.08.007>.
- Kartakis, Sokratis, Giuseppe Angelo Porcelli, Georgios Schinas, dan Shelbee Eigenbrode. June 2022. “MLOps Foundation Roadmap for Enterprises with Amazon SageMaker”. Diakses 2 November 2025, *AWS Machine Learning Blog* (). <https://aws.amazon.com/blogs/machine-learning/mlops-foundation-roadmap-for-enterprises-with-amazon-sagemaker/>.
- Keeney, Ralph L., dan Howard Raiffa. 1993. *Decisions with Multiple Objectives: Preferences and Value Trade-Offs*. Cambridge University Press. ISBN: 978-0521438834. <https://doi.org/10.1109/TSMC.1979.4310245>. <https://doi.org/10.1109/TSMC.1979.4310245>.
- Kim, Gene, Jez Humble, Patrick Debois, dan John Willis. 2016. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. 1st. IT Revolution Press. ISBN: 978-1942788003. <https://itrevolution.com/product/the-devops-handbook/>.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O’Reilly Media. ISBN: 978-1491903063. <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>.
- Kong Inc. 2025. *Kong Gateway Documentation*. Dokumentasi. Diakses 10 November 2025. <https://docs.konghq.com/gateway/latest/>.
- Kreps, Jay. 2014. *Questioning the Lambda Architecture*. Diakses 8 November 2025. O’Reilly Radar. <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>.
- Kreuzberger, Dominik, Niklas Kühl, dan Sebastian Hirschl. 2023. “Machine Learning Operations (MLOps): Overview, Definition, and Architecture”. *IEEE Access* 11:31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>. <https://doi.org/10.1109/ACCESS.2023.3262138>.

- KServe Contributors. 2024. *KServe Documentation*. Dokumentasi. Diakses 12 Oktober 2025. <https://kservice.github.io/website/>.
- Kubeflow Contributors. 2024. *Kubeflow Documentation*. Dokumentasi. Diakses 29 September 2025. <https://www.kubeflow.org/docs/>.
- Lakka, Mounika. 2025. “Kubernetes for Enterprise: Mastering Cloud-Native Container Orchestration at Scale”. *European Modern Studies Journal* 9 (3): 358–367. [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31). [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31).
- Lamer, Antoine, Chloé Saint-Dizier, Nicolas Paris, dan Emmanuel Chazard. 2024. “Data Lake, Data Warehouse, Datamart, and Feature Store: Their Contributions to the Complete Data Reuse Pipeline”. *JMIR Medical Informatics* 12:e54590. <https://doi.org/10.2196/54590>. <https://doi.org/10.2196/54590>.
- Lamsweerde, Axel van. 2009. *Requirements Engineering: From System Goals to UML Models to Software Specifications*. Wiley. ISBN: 978-0470012703. <https://www.wiley.com/en-us/Requirements+Engineering%3A+From+System+Goals+to+UML+%20to+Software+Specifications-p-9780470012703>.
- Li, Anya, Bhala Ranganathan, Feng Pan, Mickey Zhang, Qianjun Xu, Runhan Li, Sethu Raman, Shail Paragbhai Shah, dan Vivienne Tang. 2023. “Managed Geo-Distributed Feature Store: Architecture and System Design”. *arXiv preprint arXiv:2305.20077*, <https://doi.org/10.48550/arXiv.2305.20077>. <https://arxiv.org/abs/2305.20077>.
- Liang, Penghao, Wei Chen, Yifan Zhang, dan Jun Liu. 2024. “Automating the Training and Deployment of Models in MLOps by Integrating Systems with Machine Learning”. *arXiv preprint arXiv:2405.09819*, <https://doi.org/10.48550/arXiv.2405.09819>. <https://arxiv.org/abs/2405.09819>.
- Limoncelli, Thomas. 2022. “GitOps: A Path to More Self-Service IT”. *Communications of the ACM* 65 (8): 43–49. <https://doi.org/10.1145/3233241>. <https://dl.acm.org/doi/10.1145/3233241>.
- Liu, Jiageng, Igor Makarov, dan Antoinette Schoar. 2023. “Anatomy of a Run: The Terra Luna Crash”. (Cambridge, MA), no. 31160, <https://doi.org/10.3386/w31160>. <https://doi.org/10.3386/w31160>.

- Lu, Jie, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, dan Guangquan Zhang. 2019. “Learning Under Concept Drift: A Review”. Versi pracetak: arXiv:2004.05785 (2020), *IEEE Transactions on Knowledge and Data Engineering* 31 (12): 2346–2363. <https://arxiv.org/abs/2004.05785>.
- Malkov, Yury A., dan D. A. Yashunin. 2020. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>. <https://doi.org/10.1109/TPAMI.2018.2889473>.
- Marz, Nathan, dan James Warren. 2015. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications. ISBN: 978-1617290343. <https://www.manning.com/books/big-data>.
- Mazur, Mieszko, Man andg, dan Miguel Vega. 2021. “COVID-19 and the March 2020 Stock Market Crash. Evidence from S&P1500”. *Finance Research Letters* 38:101690. <https://doi.org/10.1016/j.frl.2020.101690>. <https://doi.org/10.1016/j.frl.2020.101690>.
- McKnight, William. January 2020. *Delivering on the Vision of MLOps: A Maturity-Based Approach*. Diakses 2 November 2025. GigaOm. <https://gigaom.com/report/delivering-on-the-vision-of-mlops/>.
- Microsoft Azure. 2023. *Machine Learning Operations Maturity Model*. Azure Architecture Center. Diakses 2 November 2025. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.
- MinIO, Inc. 2024. *MinIO Documentation*. Dokumentasi. Diakses 5 November 2025. <https://min.io/docs/>.
- MLflow Contributors. 2024. *MLflow Documentation*. Dokumentasi. Diakses 1 November 2025. <https://mlflow.org/docs/latest/index.html>.
- Munappy, Aiswarya Raj, Jan Bosch, Helena Holmström Olsson, Anders Arpteg, dan Björn Brinne. 2022. “Data Management for Production Quality Deep Learning Models: Challenges and Solutions”. *Journal of Systems and Software* 191:111359. <https://doi.org/10.1016/j.jss.2022.111359>. <https://doi.org/10.1016/j.jss.2022.111359>.

- Newman, Sam. 2015. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media. ISBN: 978-1491950524. <https://www.oreilly.com/library/view/building-microservices/9781491950340/>.
- Nuseibeh, Bashar, dan Steve Easterbrook. 2000. "Requirements Engineering: A Roadmap". Dalam *Proceedings of the Conference on The Future of Software Engineering*, 35–46. <https://doi.org/10.1145/336512.336523>. <https://doi.org/10.1145/336512.336523>.
- OpenAI. December 2022. *New and Improved Embedding Model*. OpenAI Blog. Diakses 2 November 2025. <https://openai.com/index/new-and-improved-embedding-model/>.
- Paley, Andrei, Raoul-Gabriel Urma, dan Neil D. Lawrence. 2022. "Challenges in Deploying Machine Learning: A Survey of Case Studies". *ACM Computing Surveys* 55 (6): 1–29. <https://doi.org/10.1145/3533378>. <https://doi.org/10.1145/3533378>.
- Pimentel, João Felipe, Leonardo Murta, Vanessa Braganholo, dan Juliana Freire. 2019. "A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks". Dalam *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, 507–517. IEEE. <https://doi.org/10.1109/MSR.2019.00077>. <https://doi.org/10.1109/MSR.2019.00077>.
- Polyzotis, Neoklis, Sudip Roy, Steven Euijong Whang, dan Martin Zinkevich. 2018. "Data Lifecycle Challenges in Production Machine Learning: A Survey". *ACM SIGMOD Record* 47 (2): 17–28. <https://doi.org/10.1145/3299887.3299891>. <https://doi.org/10.1145/3299887.3299891>.
- Prometheus Authors. 2024. *Prometheus Documentation*. Dokumentasi. Diakses 5 November 2025. <https://prometheus.io/docs/>.
- Raystack. 2025a. *Compass Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/compass>.
- . 2025b. *Dagger Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/dagger>.
- . 2025c. *Firehose Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/firehose>.

- . 2025d. *Frontier Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/frontier>.
- . 2025e. *Guardian Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/guardian>.
- Raystack. 2025f. *Meteor Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/meteor>.
- . 2025g. *Optimus Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/optimus>.
- . 2025h. *Raccoon Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/raccoon>.
- . 2025i. *Siren Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/siren>.
- . 2025j. *Stencil Documentation*. Dokumentasi. Diakses 10 November 2025. <https://github.com/raystack/stencil>.
- Redis Ltd. 2024. *Redis Documentation*. Dokumentasi. Diakses 7 November 2025. <https://redis.io/docs/>.
- Reis, Denis Moreira dos, Peter Flach, Stan Matwin, dan Gustavo Batista. 2016. “Fast Unsupervised Online Drift Detection Using Incremental Kolmogorov-Smirnov Test”. Dalam *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1545–1554. ACM. <https://doi.org/10.1145/2939672.2939836>. <https://doi.org/10.1145/2939672.2939836>.
- Rella, Bhanu Prakash Reddy. 2022. “MLOps and DataOps Integration for Scalable Machine Learning Deployment”. *International Journal for Multidisciplinary Research* 4 (1). <https://www.ijfmr.com/papers/2022/1/39278.pdf>.
- Rodrigues, Miguel G., Eduardo K. Viegas, Altair O. Santin, dan Fabricio Enembreck. 2025. “A MLOps Architecture for Near Real-Time Distributed Stream Learning Operation Deployment”. *Journal of Network and Computer Applications* 238:104169. <https://doi.org/10.1016/j.jnca.2025.104169>. <https://doi.org/10.1016/j.jnca.2025.104169>.
- Saaty, Thomas L. 2008. “Decision Making with the Analytic Hierarchy Process”. *International Journal of Services Sciences* 1 (1): 83–98. <https://doi.org/10.1504/IJSSCI.2008.017590>. <https://doi.org/10.1504/IJSSCI.2008.017590>.

- Sculley, D., Gary Holt, andiel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, dan and Den-
nison. 2015. "Hidden Technical Debt in Machine Learning Systems". Dalam
Advances in Neural Information Processing Systems 28 (NIPS 2015), disunting
oleh C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, dan R. Garnett, 2503–
2511. Red Hook, NY: Curran Associates, Inc. [https://proceedings.neurips.cc/
paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf](https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf).
- Shankar, Shreya, Rolando Garcia, Joseph M. Hellerstein, dan Aditya G. Parameswaran.
2022. *Operationalizing Machine Learning: An Interview Study*. ArXiv preprint
arXiv:2209.09125. <https://doi.org/10.48550/arXiv.2209.09125>. [https://arxiv.
org/abs/2209.09125](https://arxiv.org/abs/2209.09125).
- Sommerville, Ian. 2016. *Software Engineering*. 10th edisi. Pearson. ISBN: 978-
0133943030. [https://www.pearson.com/en-us/subject-catalog/p/software-
engineering/P200000003339](https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003339).
- Stopford, Ben. 2018. *Designing Event-Driven Systems: Concepts and Patterns for
Streaming Services with Apache Kafka*. O'Reilly Media. ISBN: 978-1-4920-
3819-5. [https://www.oreilly.com/library/view/designing-event-driven-
systems/9781492038252/](https://www.oreilly.com/library/view/designing-event-driven-systems/9781492038252/).
- Stoyanovich, Julia, Bill Howe, dan H. V. Jagadish. 2020. "Responsible Data Man-
agement". *Proceedings of the VLDB Endowment* 13 (12): 3474–3488. <https://doi.org/10.14778/3415478.3415570>. [https://doi.org/10.14778/3415478.
3415570](https://doi.org/10.14778/3415478.3415570).
- Tekton Contributors. 2024. *Tekton Pipelines Documentation*. Dokumentasi. Diakses
10 November 2025. <https://tekton.dev/docs/>.
- The Business Research Company. September 2025. *Algorithmic Trading Market
Report 2025*. Diakses 15 Oktober 2025. [https://www.researchandmarkets.
com/reports/5939167/algorithmic-trading-market-report](https://www.researchandmarkets.com/reports/5939167/algorithmic-trading-market-report).
- Triantaphyllou, Evangelos. 2000. *Multi-Criteria Decision Making Methods: A Com-
parative Study*. Springer. ISBN: 978-0792366072. [https://link.springer.com/
book/10.1007/978-1-4757-3157-6](https://link.springer.com/book/10.1007/978-1-4757-3157-6).

- Vangala, Rajesh, Priya Mehta, dan Arjun Singh. 2022. *MLOps in Practice: A Framework for Scalable AI Model Deployment, Monitoring, and Retraining*. Technical Report. Diakses 1 November 2025. https://www.researchgate.net/publication/393096121_MLOps_in_Practice_A_Framework_for_Scalable_AI_Model_Deployment_Monitoring_and_Retraining.
- Vela, andiel, Andrew Sharp, Richard Zhang, Trang Nguyen, An Hoang, dan Oleg S. Pinykh. 2022. "Temporal Quality Degradation in AI Models". *Scientific Reports* 12:11654. <https://doi.org/10.1038/s41598-022-15245-z>. <https://doi.org/10.1038/s41598-022-15245-z>.
- Wang, Weiquan, dan Johannes Ruf. 2021. "Information Leakage in Backtesting". *SSRN Electronic Journal*, <https://doi.org/10.2139/ssrn.3836631>. <https://doi.org/10.2139/ssrn.3836631>.
- Zaharia, Matei, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, dkk. 2016. "Apache Spark: A Unified Engine for Big Data Processing". *Communications of the ACM* 59 (11): 56–65. <https://doi.org/10.1145/2934664>. <https://doi.org/10.1145/2934664>.